
QUESTION 1

Select the policies below that permit you to create a new entry of environment=prod at the path /secrets/apps/my_secret (select three).

- A. path "secrets/+my_secret" { capabilities = ["create"] allowed_parameters = { "*" = [] } }
- B. path "secrets/apps/my_secret" { capabilities = ["update"] }
- C. path "secrets/apps/my_secret" { capabilities = ["create"] allowed_parameters = { "environment" = [] } }
- D. path "secrets/apps/*" { capabilities = ["create"] allowed_parameters = { "environment" = ["dev", "test", "qa", "prod"] } }

Answer: A, C, D

Explanation:

Comprehensive and Detailed in Depth

This question requires identifying Vault policies that allow creating a new entry with environment=prod at the specific path /secrets/apps/my_secret. Vault policies define permissions using paths, capabilities, and parameter constraints. Let's evaluate each option:

Option A: path "secrets/+my_secret" { capabilities = ["create"] allowed_parameters = { "*" = [] } }

The + wildcard matches any single segment in the path, so this policy applies to /secrets/apps/my_secret. The create capability permits creating new entries at this path. The allowed_parameters = { "*" = [] } means any parameter (including environment) can be set to any value. This satisfies the requirement to create an entry with environment=prod. Thus, this policy is correct.

Option B: path "secrets/apps/my_secret" { capabilities = ["update"] }

This policy targets the exact path /secrets/apps/my_secret but only grants the update capability. According to Vault's documentation, update allows modifying existing entries, not creating new ones. Since the question specifies creating a new entry, this policy does not meet the requirement and is incorrect.

Option C: path "secrets/apps/my_secret" { capabilities = ["create"] allowed_parameters = { "environment" = [] } }

This policy explicitly matches /secrets/apps/my_secret and grants the create capability, which allows new entries to be written. The allowed_parameters = { "environment" = [] } specifies that the environment parameter can take any value (an empty list means no restriction on values). This permits setting environment=prod, making this policy correct.

Option D: path "secrets/apps/*" { capabilities = ["create"] allowed_parameters = { "environment" = ["dev", "test", "qa", "prod"] } }

The * wildcard matches any path under secrets/apps/, including /secrets/apps/my_secret. The create capability allows new entries, and the allowed_parameters restricts environment to dev, test, qa, or prod. Since prod is an allowed value, this policy permits creating an entry with environment=prod and is correct.

Overall Explanation from Vault Docs:

Vault policies control access via paths and capabilities (create, read, update, delete, list). The create

capability is required to write new data. Parameter constraints (allowed_parameters) further restrict what key-value pairs can be written. An empty list ([]) allows any value, while a populated list restricts values to those specified. A deny takes precedence over any allow, but no deny is present here. Reference: <https://developer.hashicorp.com/vault/docs/concepts/policies#parameter-constraints>

QUESTION 2

Below is a list of parent and child tokens and their associated TTL. Which token(s) will be revoked first?

- A. |——hvs.y4fUERqCtUV0xsQjWLJar5qX - TTL: 4 hours
- B. |——hvs.FNiIFU14RUxxUYA14ErLfPVR - TTL: 6 hours
- C. |——hvs.Jw9LMpu7oCQgxiKbjfyzyg75 - TTL: 4 hours (child of B)
- D. |——hvs.3IrlhEvcerEGbae11YQf9FvI - TTL: 3 hours
- E. |——hvs.hOpweMVFvqfvoVnNgvZq8jLS - TTL: 5 hours (child of D)

Answer: D

Explanation:

Comprehensive and Detailed in Depth

Vault tokens have a Time-To-Live (TTL) that determines their expiration time, after which they are revoked. Parent-child relationships mean that revoking a parent token also revokes its children, regardless of their TTLs. Let's analyze:

- A: TTL 4 hours - Expires after 4 hours, no children listed.
- B: TTL 6 hours - Expires after 6 hours, parent to C.
- C: TTL 4 hours (child of B) - Expires after 4 hours or if B is revoked earlier.
- D: TTL 3 hours - Expires after 3 hours, parent to E.
- E: TTL 5 hours (child of D) - Expires after 5 hours or if D is revoked earlier.

Analysis:

Shortest TTL is D (3 hours), so it expires first unless a parent above it (none listed) is revoked sooner.

E (5 hours) is a child of D. If D is revoked at 3 hours, E is also revoked, despite its longer TTL.

A and C (4 hours) expire after D.

B (6 hours) expires last among parents.

The question asks which token(s) are revoked first based on TTL alone, not manual revocation. D has the shortest TTL (3 hours) and will be revoked first. E's revocation depends on D, but the question focuses on initial expiration. Thus, only D is revoked first based on its TTL.

Overall Explanation from Vault Docs:

Tokens form a hierarchy where child tokens inherit revocation from their parents. æWhen a parent token is revoked, all of its child tokens”and all of their leases”are revoked as well. TTL dictates automatic expiration unless overridden by manual revocation or parent revocation. Here, D's 3-hour TTL is the shortest, making it the first to expire naturally.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/tokens#token-hierarchies-and-orphan-tokens>

QUESTION 3

Your company's security policies require that all encryption keys must be rotated at least once per year. After using the Transit secrets engine for a year, the Vault admin issues the proper command to rotate the key named ecommerce that was used to encrypt your data.

What command can be used to easily re-encrypt the original data with the new version of the key?

- A. `vault write -f transit/keys/ecommerce/rotate <old data>`
- B. `vault write -f transit/keys/ecommerce/update <old data>`
- C. `vault write transit/encrypt/ecommerce v1:v2 <old data>`
- D. `vault write transit/rewrap/ecommerce ciphertext=<old data>`

Answer: D

Explanation:

Comprehensive and Detailed in Depth

The Transit secrets engine in Vault manages encryption keys and supports key rotation. After rotating the ecommerce key, existing ciphertext (encrypted with the old key version) must be re-encrypted (rewrapped) with the new key version without exposing plaintext. Let's evaluate:

A: `vault write -f transit/keys/ecommerce/rotate <old data>`

This command rotates the key, creating a new version, but does not re-encrypt existing data. It's for key management, not data rewrapping. Incorrect.

B: `vault write -f transit/keys/ecommerce/update <old data>`

There's no update endpoint in Transit for re-encrypting data. This is invalid and incorrect.

C: `vault write transit/encrypt/ecommerce v1:v2 <old data>`

The transit/encrypt endpoint encrypts new plaintext, not existing ciphertext. The v1:v2 syntax is invalid. Incorrect.

D: `vault write transit/rewrap/ecommerce ciphertext=<old data>`

The transit/rewrap endpoint takes existing ciphertext, decrypts it with the old key version, and reencrypts it with the latest key version (post-rotation). This is the correct command. For example, if <old data> is `vault:v1:cZNHVx+...`, the output might be `vault:v2:kChHZ9w4....`

Overall Explanation from Vault Docs:

æVault's Transit secrets engine supports key rotation! The rewrap endpoint allows ciphertext encrypted with an older key version to be re-encrypted with the latest key version without exposing the plaintext. This operation is secure and efficient, using the keyring internally.

Reference: <https://developer.hashicorp.com/vault/tutorials/encryption-as-a-service/eaas-transitrewrap>

QUESTION 4

From the unseal options listed below, select the options you can use if you're deploying Vault onpremises (select four).

- A. Certificates
- B. Transit
- C. AWS KMS
- D. HSM PKCS11
- E. Key shards

Answer: B, C, D, E

Explanation:

Comprehensive and Detailed in Depth

Vault requires unsealing to access encrypted data, and on-premises deployments support various unseal mechanisms. Let's assess:

A: Certificates

Certificates secure communication (e.g., TLS), not unsealing. Vault's seal/unseal process uses cryptographic keys, not certificates. Incorrect.

B: Transit

The Transit secrets engine can auto-unseal Vault by managing encryption keys internally. Ideal for onpremises setups avoiding external services. Correct.

C: AWS KMS

AWS KMS can auto-unseal Vault if the on-premises cluster has internet access to AWS APIs. Common in hybrid setups. Correct.

D: HSM PKCS11

Hardware Security Modules (HSM) with PKCS11 support secure key storage and auto-unsealing onpremises. Correct.

E: Key shards

Shamir's Secret Sharing splits the master key into shards, the default manual unseal method for all Vault clusters. Correct.

Overall Explanation from Vault Docs:

æVault supports multiple seal types| Key shards (Shamir) is the default| Auto-unseal options like Transit, AWS KMS, and HSM (PKCS11) are viable for on-premises if configured with access to required services. Certificates are not an unseal mechanism.

Reference: <https://developer.hashicorp.com/vault/docs/configuration/seal>

QUESTION 5

How does the Vault Secrets Operator (VSO) assist in integrating Kubernetes-based workloads with

Vault?

- A. By enabling a local API endpoint to allow the workload to make requests directly from the VSO
- B. By using client-side caching for KVv1 and KVv2 secrets engines
- C. By injecting a Vault Agent directly into the pod requesting secrets from Vault
- D. By watching for changes to its supported set of Custom Resource Definitions (CRD)

Answer: D

Explanation:

Comprehensive and Detailed in Depth

The Vault Secrets Operator (VSO) integrates Kubernetes workloads with Vault by syncing secrets.

Let's evaluate:

A: VSO doesn't create a local API endpoint for direct requests; it syncs secrets to Kubernetes Secrets.

Incorrect.

B: Client-side caching is a Vault Agent feature, not VSO's primary function. VSO can use caching, but it's not the main integration method. Incorrect.

C: VSO doesn't inject Vault Agents; that's a separate Vault Agent Sidecar approach. Incorrect.

D: VSO watches Custom Resource Definitions (CRDs) to sync Vault secrets to Kubernetes Secrets dynamically. This is its core mechanism. Correct.

Overall Explanation from Vault Docs:

œVSO operates by watching for changes to its supported set of CRDs| It synchronizes secrets from Vault to Kubernetes Secrets, ensuring applications access them natively.

Reference: <https://developer.hashicorp.com/vault/docs/platform/k8s/vso>

QUESTION 6

By default, what TCP port does Vault replication use?

- A. tcp00
- B. tcp00
- C. tcp01
- D. tcp01

Answer: C

Explanation:

Comprehensive and Detailed in Depth

Vault replication ensures data consistency across clusters, using a specific port:

A: 8200 - Default HTTP API port, not replication.

B: 8300 - Raft protocol port, not replication.

C: 8201 - Default replication port. Correct.

D: 8301 - Serf protocol port, not replication.

Overall Explanation from Vault Docs:

œReplication occurs on TCP port 8201 by default| distinct from the API (8200) and Raft (8300) ports.

Reference: <https://developer.hashicorp.com/vault/tutorials/day-one-raft/raft-referencearchitecture#network-connectivity>

QUESTION 7

What is the proper command to enable the AWS secrets engine at the default path?

A. vault enable aws secrets engine

B. vault secrets enable aws

C. vault secrets aws enable

D. vault enable secrets aws

Answer: B

Explanation:

Comprehensive and Detailed in Depth

Enabling a secrets engine in Vault follows a specific syntax:

A: Incorrect syntax; jumbled order.

B: Correct: vault secrets enable <type> enables the AWS engine at aws/. Correct.

C: Incorrect word order.

D: Incorrect syntax.

Overall Explanation from Vault Docs:

œThe command vault secrets enable <type> enables a secrets engine at its default path (e.g., aws/ for AWS).

Reference: <https://developer.hashicorp.com/vault/docs/commands/secrets>

QUESTION 8

In regards to the Transit secrets engine, which of the following is true given the following command and output (select three):

\$ vault write encryption/encrypt/creditcard plaintext=\$(base64 <<< "1234 5678 9101 1121")

Key: ciphertext Value:

vault:v3:cZNHVx+sxdMErXRSuDa1q/pz49fXTn1PScKfhf+PIZPvy8xKfkytpwKcbC0fF2U=

A. The Transit secrets engine is mounted at the encryption path

B. The name of the keyring used to encrypt the data is creditcard

- C. There are at least three data keys associated with this keyring
- D. The data was written to the encryption path, which is provided by default when enabling the Transit secrets engine

Answer: A, B, C

Explanation:

Comprehensive and Detailed in Depth

A: The command uses encryption/encrypt/creditcard, indicating the Transit engine is mounted at encryption/. Correct.

B: The endpoint creditcard specifies the key name used for encryption. Correct.

C: The output vault:v3: shows key version 3, implying at least three versions (v1, v2, v3) after rotations. Correct.

D: The default path for Transit is transit/, not encryption/. This is a custom mount, not default. Incorrect.

Overall Explanation from Vault Docs:

œThe Transit engine encrypts data at a specified key name| Key versions (e.g., v3) indicate rotations.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/transit>

QUESTION 9

Which of the following statements are true regarding Vault seal and unseal (select three)?

- A. By default, Vault uses the Shamir Sharing algorithm to create unseal keys during the initialization process
- B. When using Vault Auto Unseal feature, Vault returns unseal keys to the user when it is initialized
- C. Vault can use a third-party KMS solution to automatically unseal during a service restart
- D. Vault supports high availability for the Auto Unseal feature, allowing you to point to multiple keys

Answer: A, C, D

Explanation:

Comprehensive and Detailed in Depth

A: Vault uses Shamir's Secret Sharing by default for unseal keys. Correct.

B: Auto Unseal uses KMS or similar; it returns recovery keys, not unseal keys. Incorrect.

C: Third-party KMS (e.g., AWS KMS) can auto-unseal Vault. Correct.

D: Auto Unseal supports HA with multiple keys for redundancy. Correct.

Overall Explanation from Vault Docs:

œVault uses Shamir's algorithm by default| Auto Unseal with KMS supports HA and does not return

unseal keys but recovery keys.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/seal#seal-unseal>

QUESTION 10

If Bobby is currently assigned the following policy, what additional policy can be added to ensure Bobby cannot access the data stored at secret/apps/confidential but still read all other secrets?

```
path "secret/apps/*" { capabilities = ["create", "read", "update", "delete", "list"] }
```

- A. path "secret/apps/confidential" { capabilities = ["deny"] }
- B. path "secret/*" { capabilities = ["read", "deny"] }
- C. path "secret/apps/*" { capabilities = ["deny"] }
- D. path "secret/apps/confidential/*" { capabilities = ["deny"] }

Answer: A

Explanation:

Comprehensive and Detailed in Depth

A: Denies all access to secret/apps/confidential, overriding the original policy's permissions. Correct.

B: Applies to all secret/*, overly restrictive and unclear with mixed capabilities. Incorrect.

C: Denies all secret/apps/*, blocking more than required. Incorrect.

D: Denies subpaths under confidential, not the path itself. Incorrect.

Overall Explanation from Vault Docs:

œA deny capability takes precedence over any allow! Use it to restrict specific paths.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/policies#capabilities>

QUESTION 11

Tommy has written an AWS Lambda function that will perform certain tasks for the organization when data has been uploaded to an S3 bucket. Security policies for the organization do not allow Tommy to hardcode any type of credential within the Lambda code or environment variables. However, Tommy needs to retrieve a credential from Vault to write data to an on-premises database. What auth method should Tommy use in Vault to meet the requirements while not violating security policies?

- A. AWS
- B. Userpass
- C. Token
- D. AppRole

Answer: A

Explanation:

Comprehensive and Detailed in Depth

A: AWS auth uses IAM roles, avoiding hardcoded credentials. Correct for Lambda.

B: Userpass requires username/password, violating policy. Incorrect.

C: Token requires a pre-generated token, often hardcoded. Incorrect.

D: AppRole needs RoleID/SecretID, typically hardcoded. Incorrect.

Overall Explanation from Vault Docs:

œThe AWS auth method provides an automated mechanism to retrieve a Vault token for IAM principals; no manual credential provisioning required.

Reference: <https://developer.hashicorp.com/vault/docs/auth/aws#aws-auth-method>

QUESTION 12

What command would have created the token displayed below?

```
$ vault token lookup hvs.nNeZ2I64ALCxuO7dqQEJGPrO
```

Key: policies Value: [default dev], num_uses: 5, ttl: 767h59m49s

Key Value

--- -----

```
accessor mfvaVMFgOcXHleqlRasroSON
creation_time 1604610457
creation_ttl 768h
display_name token
entity_id n/a
expire_time 2024-12-07T16:07:37.7540672-05:00
explicit_max_ttl 0s
id hvs.nNeZ2I64ALCxuO7dqQEJGPrO
issue_time 2024-11-05T16:07:37.7540672-05:00
meta <nil>
num_uses 5
orphan false
path auth/token/create
policies [default dev]
renewable true
ttl 767h59m49s
type service
```

A. vault token create -policy=dev -use-limit=5

B. vault token create -policy=dev -ttl=768h

C. vault token create -policy=dev -policy=default -ttl=768h

D. vault token create -policy=dev

Answer: A

Explanation:

Comprehensive and Detailed in Depth

A: Matches dev policy and num_uses=5. TTL is system default (768h). Correct.

B: Missing num_uses. Incorrect.

C: Adds default policy explicitly, not needed as it's implicit. Incorrect.

D: Missing num_uses. Incorrect.

Overall Explanation from Vault Docs:

œvault token create with -policy and -use-limit sets specific attributes| default policy is included implicitly.

Reference: <https://developer.hashicorp.com/vault/docs/commands/token/create#command-options>

QUESTION 13

You've set up multiple Vault clusters, one on-premises intended to be the primary cluster, and the second cluster in AWS, which was deployed for performance replication. After enabling replication, developers complain that all the data they've stored in the AWS Vault cluster is missing. What happened?

A. There is a certificate mismatch after replication was enabled since Vault replication generates its own TLS certificates to ensure nodes are trusted entities

B. All of the data on the secondary cluster was deleted after replication was enabled

C. The data was automatically copied to the primary cluster after replication was enabled since all writes are always forwarded to the primary cluster

D. The data was moved to a recovery path after replication was enabled. Use the vault secrets move command to move the data back to its intended location

Answer: B

Explanation:

Comprehensive and Detailed in Depth

A: Certificate issues don't delete data. Incorrect.

B: Performance replication wipes the secondary's data to sync with the primary. Correct.

C: Data isn't copied to the primary; replication is one-way. Incorrect.

D: No recovery path exists; data is wiped. Incorrect.

Overall Explanation from Vault Docs:

œWhen replication is enabled, all of the secondary's existing storage will be wiped| This is

irrevocable.

Reference: <https://developer.hashicorp.com/vault/tutorials/enterprise/performance-replication>

QUESTION 14

Based on the screenshot below, how many auth methods have been enabled on this Vault instance?



- A. 1
- B. 2
- C. 4
- D. 3

Answer: B

Explanation:

Comprehensive and Detailed in Depth

Token is enabled by default and cannot be disabled.

Userpass is explicitly enabled.

Total: 2 auth methods.

Overall Explanation from Vault Docs:

œTokens are the default auth method! Additional methods like userpass increase the count.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/tokens>

QUESTION 15

Given the following policy, which command below would not result in a permission denied error (select two)?

```
path "secret/*" { capabilities = ["create", "update"] allowed_parameters = { "student" = ["steve", "frank", "jamie", "susan", "gerry", "damien"]} }
path "secret/apps/*" { capabilities = ["read"]}
path "secret/apps/results" { capabilities = ["deny"]} }
```

- A. vault kv put secret/apps/results student03=practice
- B. vault kv put secret/apps/app01 student=bryan
- C. vault kv put secret/common/results student=frank
- D. vault kv get secret/apps/api_key

Answer: C, D

Explanation:

Comprehensive and Detailed in Depth

A: Denied by secret/apps/results deny policy. Incorrect.

B: secret/apps/app01 only allows read, not create. Incorrect.

C: secret/common/results allows create with student=frank (allowed value). Correct.

D: secret/apps/api_key allows read. Correct.

Overall Explanation from Vault Docs:

ædeny overrides any allow| allowed_parameters restricts values.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/policies#parameter-constraints>

QUESTION 16

Which of the following Vault policies will allow a Vault client to read a secret stored at secrets/applications/app01/api_key?

- A. path "secrets/applications/" { capabilities = ["read"] allowed_parameters = { "certificate" = [] } }
- B. path "secrets/*" { capabilities = ["list"] }
- C. path "secrets/applications/+api_*" { capabilities = ["read"] }
- D. path "secrets/applications/app01/api_key/*" { capabilities = ["update", "list", "read"] }

Answer: C

Explanation:

Comprehensive and Detailed in Depth

This question requires identifying a policy that permits reading the secret at secrets/applications/app01/api_key. Vault policies use paths and capabilities to control access. Let's evaluate:

A: path "secrets/applications/" { capabilities = ["read"] allowed_parameters = { "certificate" = [] } }

This policy allows reading at secrets/applications/, but not deeper paths like secrets/applications/app01/api_key. The allowed_parameters restriction is irrelevant for reading secrets. Incorrect.

B: path "secrets/*" { capabilities = ["list"] }

The list capability allows listing secrets under secrets/, but not reading their contents. Reading requires the read capability. Incorrect.

C: path "secrets/applications/+api_*" { capabilities = ["read"] }

The + wildcard matches one segment (e.g., app01), and api_* matches api_key. This policy grants read access to secrets/applications/app01/api_key. Correct.

D: path "secrets/applications/app01/api_key/*" { capabilities = ["update", "list", "read"] }

This policy applies to subpaths under api_key/, not the exact path api_key. It includes read, but the path mismatch makes it incorrect for this specific secret.

Overall Explanation from Vault Docs:

œWildcards (*, +) allow flexible path matching| read capability is required to retrieve secret data.

Option C uses globbing to precisely target the required path.

Reference: <https://developer.hashicorp.com/vault/tutorials/policies/policies>

QUESTION 17

You want to encrypt a credit card number using the Transit secrets engine. You enter the following command and receive an error. What can you do to ensure that the credit card number is properly encrypted and the ciphertext is returned?

```
$ vault write -format=json transit/encrypt/creditcards plaintext="1234 5678 9101 1121"
```

Error: * illegal base64 data at input byte 4

- A. The plain text data needs to be encoded to base64
- B. The token used to issue the encryption request does not have the appropriate permissions
- C. Credit card numbers are not supported using the Transit secrets engine since it is considered sensitive data
- D. The credit card number should not include spaces

Answer: A

Explanation:

Comprehensive and Detailed in Depth

The error indicates a problem with the plaintext input format. Let's analyze:

A: The Transit engine requires plaintext to be base64-encoded for safe transport, as it may include non-text data. The error illegal base64 data occurs because "1234 5678 9101 1121" isn't base64-encoded. Correct: use plaintext=\$(base64 <<< "1234 5678 9101 1121").

B: Permission errors would return a 403, not a base64 error. Incorrect.

C: Transit supports encrypting sensitive data like credit card numbers. Incorrect.

D: Spaces aren't the issue; the format must be base64. Incorrect.

Overall Explanation from Vault Docs:

œWhen you send data to Vault for encryption, it must be base64-encoded plaintext| This ensures

safe transport of binary or text data.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/transit#usage>

QUESTION 18

Which of the following token attributes can be used to renew a token in Vault (select two)?

- A. TTL
- B. Token ID
- C. Identity policy
- D. Token accessor

Answer: B, D

Explanation:

Comprehensive and Detailed in Depth

Token renewal extends a token's TTL. Let's evaluate:

A: TTL - Defines expiration time, not used for renewal. Incorrect.

B: Token ID - The token's unique identifier; can be specified to renew it (e.g., vault token renew <token-id>). Correct.

C: Identity policy - Relates to access control, not renewal. Incorrect.

D: Token accessor - A unique identifier for operations like renewal without exposing the token (e.g., vault token renew -accessor <accessor>). Correct.

Overall Explanation from Vault Docs:

œTokens can be renewed with vault token renew using either the token ID or accessor. TTL is not an attribute for renewal.

Reference: <https://developer.hashicorp.com/vault/docs/commands/token/renew#token-renew>

QUESTION 19

When generating dynamic credentials, Vault also creates associated metadata, including information like time duration, renewability, and more, and links it to the credentials. What is this referred to as?

- A. Secret
- B. Token
- C. Lease
- D. Secrets engine

Answer: C

Explanation:

Comprehensive and Detailed in Depth

A: Secrets are the credentials themselves, not the metadata. Incorrect.

B: Tokens authenticate clients, not the metadata for credentials. Incorrect.

C: A lease is metadata tied to dynamic secrets, managing their lifecycle (TTL, renewability). Correct.

D: Secrets engines generate secrets, not the metadata. Incorrect.

Overall Explanation from Vault Docs:

œWith every dynamic secret, Vault creates a lease: metadata containing TTL, renewability, etc.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/lease>

QUESTION 20

You are using an orchestrator to deploy a new application. Even though the orchestrator creates a new AppRole secret ID, security requires that only the new application has the combination of the role ID and secret ID. What feature can you use to meet these requirements?

A. Have the application authenticate with the role ID to retrieve the secret ID

B. Use response wrapping and provide the application server with the unwrapping token instead

C. Use a batch token instead of a traditional service token

D. Secure the communication between the orchestrator and Vault using TLS

Answer: B

Explanation:

Comprehensive and Detailed in Depth

A: Exposes the secret ID, violating the requirement. Incorrect.

B: Response wrapping delivers the secret ID in a single-use token, ensuring only the application unwraps it. Correct.

C: Batch tokens don't address secret ID delivery security. Incorrect.

D: TLS secures communication but doesn't restrict access to the secret ID. Incorrect.

Overall Explanation from Vault Docs:

œResponse wrapping wraps the secret in a single-use token, ensuring only the intended recipient unwraps it.

Reference: <https://developer.hashicorp.com/vault/tutorials/auth-methods/approle>

QUESTION 21

True or False? When encrypting data with the Transit secrets engine, Vault always stores the ciphertext in a dedicated KV store along with the associated encryption key.

A. True

B. False

Answer: B

Explanation:

Comprehensive and Detailed in Depth

A: Incorrect. Transit doesn't store ciphertext; it returns it to the client.

B: Correct. The Transit engine performs encryption/decryption without persisting data.

Overall Explanation from Vault Docs:

œThe Vault Transit secrets engine does NOT store any data| Ciphertext is returned to the caller.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/transit>

QUESTION 22

What is the default maximum time-to-live (TTL) for a token, measured in days?

A. 32 days (768 hours)

B. 7 days (168 hours)

C. 14 days (336 hours)

D. 31 days (744 hours)

Answer: A

Explanation:

Comprehensive and Detailed in Depth

A: Vault's default max TTL is 768 hours (32 days). Correct.

B, C, D: Incorrect values per Vault's defaults.

Overall Explanation from Vault Docs:

œThe system max TTL is 768 hours (32 days) unless overridden|

Reference: <https://developer.hashicorp.com/vault/docs/concepts/tokens#token-time-to-liveperiodic-tokens-and-explicit-max-ttls>

QUESTION 23

After decrypting data using the Transit secrets engine, the plaintext output does not match the plaintext credit card number that you encrypted. Which of the following answers provides a solution?

\$ vault write transit/decrypt/creditcard ciphertext="vault:v1:cZNHVx+sxdMEr....."

Key: plaintext Value: Y3JlZGl0LWNhcmQtbmVtYmVyCg==

A. Vault is sealed, therefore the data cannot be decrypted. Unseal Vault to properly decrypt the data

B. The user doesn't have permission to decrypt the data, therefore Vault returns false data

- C. The resulting plaintext data is base64-encoded. To reveal the original plaintext, use the `base64 --decode` command
- D. The data is corrupted. Execute the encryption command again using a different data key

Answer: C

Explanation:

Comprehensive and Detailed in Depth

A: Sealing would prevent decryption, not return encoded data. Incorrect.

B: Permission issues don't return encoded data. Incorrect.

C: Transit returns base64-encoded plaintext; decoding `Y3JlZGl0LWNhcmQtbmVtYmVyCg==` yields `œcredit-card-number` . Correct.

D: No evidence of corruption; it's a format issue. Incorrect.

Overall Explanation from Vault Docs:

œAll plaintext data must be base64-encoded! Decode it to reveal the original value.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/transit>

QUESTION 24

True or False? The Vault Secrets Operator does NOT encrypt client cache, such as Vault tokens and leases, by default in Kubernetes Secrets.

- A. True
- B. False

Answer: A

Explanation:

Comprehensive and Detailed in Depth

A: VSO doesn't encrypt client cache by default; it requires extra configuration. Correct.

B: Incorrect; encryption is optional, not default.

Overall Explanation from Vault Docs:

œClient cache persistence and encryption are not enabled by default! Requires Transit engine configuration.

Reference: <https://developer.hashicorp.com/vault/docs/platform/k8s/vso/sources/vault#vaultclient-cache>

QUESTION 25

True or False? When using the Transit secrets engine, setting the `min_decryption_version` will determine the minimum key length of the data key (i.e., 2048, 4096, etc.).

- A. True
- B. False

Answer: B

Explanation:

Comprehensive and Detailed in Depth

A: Incorrect. `min_decryption_version` sets the minimum key version, not length.

B: Correct. It controls versioning, not key size.

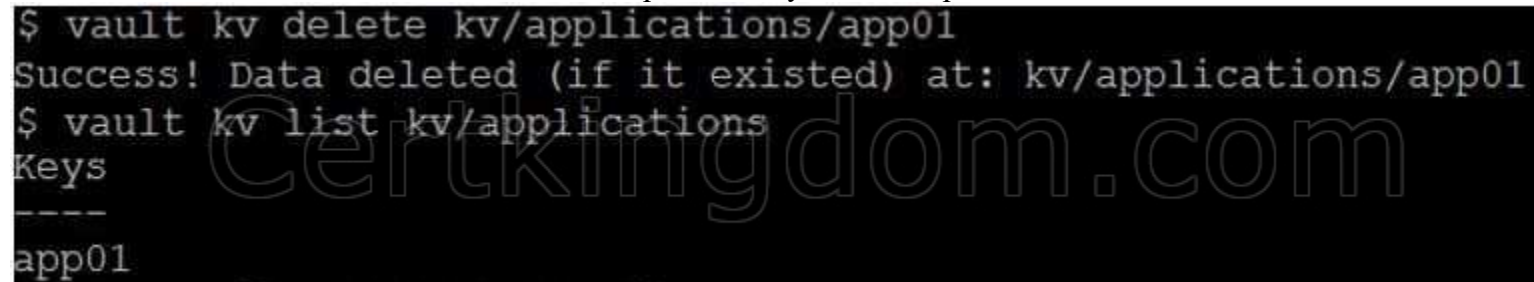
Overall Explanation from Vault Docs:

`min_decryption_version` specifies the minimum key version for decryption. Key length is a separate configuration.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/transit#usage>

QUESTION 26

After issuing the command to delete a secret, you run a `vault kv list` command, but the path to the secret still seems to exist. What command would permanently delete the path from Vault?



```
$ vault kv delete kv/applications/app01
Success! Data deleted (if it existed) at: kv/applications/app01
$ vault kv list kv/applications
Keys
----
app01
```

- A. `vault kv delete -force kv/applications/app01`
- B. `vault kv destroy -versions=1 kv/applications/app01`
- C. `vault kv metadata delete kv/applications/app01`
- D. `vault kv delete -all kv/applications/app01`

Answer: C

Explanation:

Comprehensive and Detailed in Depth

A: Soft-deletes data, not metadata. Incorrect.

B: Destroys a version, not the path. Incorrect.

C: Deletes all metadata and versions, removing the path. Correct.

D: Invalid syntax. Incorrect.

Overall Explanation from Vault Docs:

⌘kv metadata delete deletes all versions and metadata for the key, permanently removing it.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/kv/kv-v2#key-metadata>

QUESTION 27

When using the Vault Secrets Operator, where is the secret written to after being retrieved from Vault?

- A. The secret is never written to any service or persistent storage
- B. Directly to the filesystem of the pod
- C. Kubernetes Secrets
- D. To the cloud-provider's native secret manager (Azure Key Vault, AWS Secrets Manager, etc.)

Answer: C

Explanation:

Comprehensive and Detailed in Depth

A: Incorrect; VSO writes to Kubernetes Secrets.

B: Incorrect; not written to pod filesystem.

C: VSO syncs secrets to Kubernetes Secrets. Correct.

D: Incorrect; no automatic cloud provider integration.

Overall Explanation from Vault Docs:

⌘VSO synchronizes secrets from Vault to Kubernetes Secrets|

Reference: <https://developer.hashicorp.com/vault/docs/platform/k8s/vso>

QUESTION 28

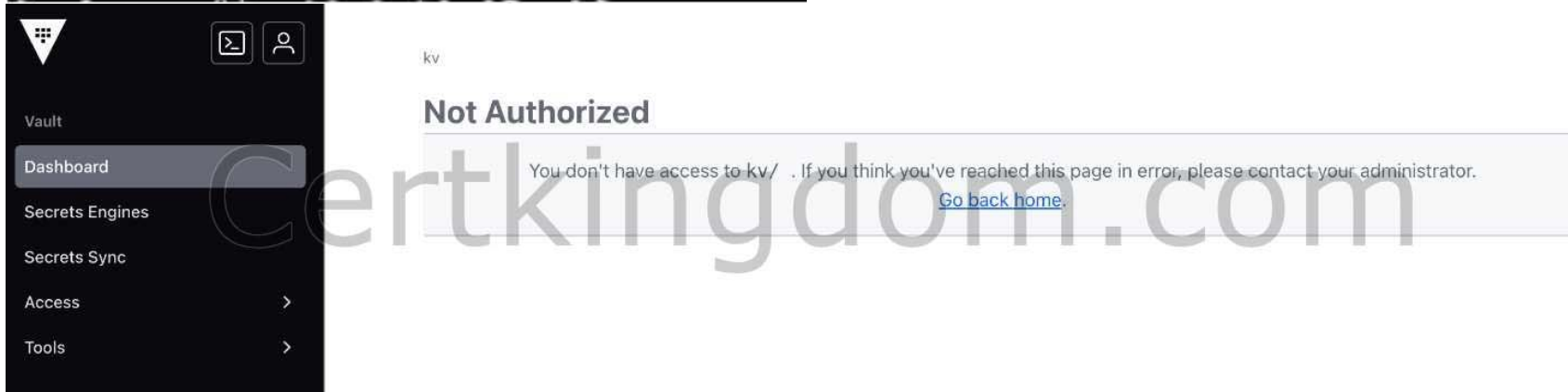
A user is assigned the following policy, and they can successfully retrieve secrets using the CLI.

However, the user reports receiving an error message in the UI. Why can't the user access the secret in the Vault UI?

```
path "kv/apps/app01" { capabilities = ["read"] }
```

Successful retrieval using the CLI

```
$ vault kv get kv/apps/app01
===== Data =====
Key      Value
-----
student01 student01
```



- A. The user doesn't know what they're doing
- B. The user doesn't have permissions to retrieve the data from the UI, only the CLI
- C. The user needs list permissions to browse the UI
- D. The user's token is invalid

Answer: C

Explanation:

Comprehensive and Detailed in Depth

A: Irrelevant to permissions. Incorrect.

B: UI and CLI use the same permissions. Incorrect.

C: UI browsing requires list on parent paths; read alone isn't enough. Correct.

D: Token works via CLI, so it's valid. Incorrect.

Overall Explanation from Vault Docs:

œTo browse the UI, users need list permissions on paths leading to the secret|

Reference: <https://developer.hashicorp.com/vault/docs/concepts/policies#list>

QUESTION 29

During a service outage, you must ensure all current tokens and leases are copied to another Vault

cluster for failover so applications don't need to authenticate. How can you accomplish this?

- A. Have Vault write all the tokens and leases to a file so you have a second copy of them
- B. Configure all applications to use the auto-auth feature of the Vault Agent
- C. Configure Disaster Recovery replication and promote the secondary cluster during an outage
- D. Replicate to another cluster using Performance Replication and promote the secondary cluster during an outage

Answer: C

Explanation:

Comprehensive and Detailed in Depth

A: Insecure and manual; not a Vault feature. Incorrect.

B: Auto-auth doesn't replicate tokens/leases. Incorrect.

C: DR replication mirrors tokens and leases; promotion enables failover. Correct.

D: Performance replication doesn't replicate tokens fully. Incorrect.

Overall Explanation from Vault Docs:

Disaster Recovery replication mirrors tokens and leases| Promote the secondary during an outage.

Reference: <https://developer.hashicorp.com/vault/docs/enterprise/replication#replicated-data>

QUESTION 30

You logged into the Vault CLI and attempted to enable an auth method, but you received this error message. What can you do to resolve the error and configure Vault?

(Error: dial tcp 127.0.0.1:8200: connect: connection refused)

```
bk~$vault secrets enable transit
Error enabling: Post "https://127.0.0.1:8200/v1/sys/mounts/transit": http: server
gave HTTP response to HTTPS client
bk~$
```

- A. Restart the Vault service on this node
- B. Ask an admin to grant you permission to enable the userpass auth method
- C. Change 'userpass' to 'username and password'
- D. Set the VAULT_ADDR environment variable to HTTP

Answer: D

Explanation:

Comprehensive and Detailed in Depth

A: Connection refused isn't a service issue here. Incorrect.

B: Permissions don't cause connection errors. Incorrect.

C: Invalid syntax change. Incorrect.

D: Default VAULT_ADDR is HTTPS; if TLS is off, set to http://0.0.0.0:8200. Correct.

Overall Explanation from Vault Docs:

œIf TLS is disabled, set VAULT_ADDR to http://0.0.0.0:8200 to avoid connection errors|

Reference: https://developer.hashicorp.com/vault/docs/commands#vault_addr

QUESTION 31

Jason has enabled the userpass auth method at the path users/. What path would Jason and other Vault operators use to interact with this new auth method?

A. users/auth/

B. authentication/users

C. auth/users

D. users/

Answer: C

Explanation:

Comprehensive and Detailed in Depth

In HashiCorp Vault, authentication methods (auth methods) are mechanisms that allow users or machines to authenticate and obtain a token. When an auth method like userpass is enabled, it is mounted at a specific path in Vault's namespace, and this path determines where operators interact with it”e.g., to log in, configure, or manage it.

The userpass auth method is enabled with the command `vault auth enable -path=users userpass`, meaning it's explicitly mounted at the `users/` path. However, Vault's authentication system has a standard convention: all auth methods are accessed under the `auth/` prefix, followed by the mount path. This prefix is a logical namespace separating authentication endpoints from secrets engines or system endpoints.

Option A: `users/auth/`

This reverses the expected order. The `auth/` prefix comes first, followed by the mount path (`users/`), not the other way around. This path would not correspond to any valid Vault endpoint for interacting with the userpass auth method. Incorrect.

Option B: `authentication/users`

Vault does not use `authentication/` as a prefix; it uses `auth/`. The term “authentication” is not part of Vault's path structure”it's a conceptual term, not a literal endpoint. This makes the path invalid and unusable in Vault's API or CLI. Incorrect.

Option C: `auth/users`

This follows Vault's standard convention: `auth/` (the authentication namespace) followed by `users`

(the custom mount path specified when enabling the auth method). For example, to log in using the userpass method mounted at users/, the command would be `vault login -method=userpass -path=users username=<user>`. The API endpoint would be `/v1/auth/users/login`. This is the correct path for operators to interact with the auth method, whether via CLI, UI, or API. Correct.

Option D: users/

While users/ is the mount path, omitting the auth/ prefix breaks Vault's structure. Directly accessing users/ would imply it's a secrets engine or other mount type, not an auth method. Auth methods always require the auth/ prefix for interaction. Incorrect.

Detailed Mechanics:

When an auth method is enabled, Vault creates a backend at the specified path under auth/. The userpass method, for instance, supports endpoints like `/login` (for authentication) and `/users/<username>` (for managing users). If mounted at users/, these become `auth/users/login` and `auth/users/users/<username>`. This structure ensures isolation and clarity in Vault's routing system. The ability to customize the path (e.g., users/ instead of the default userpass/) allows flexibility for organizations with multiple auth instances, but the auth/ prefix remains mandatory.

Overall Explanation from Vault Docs:

œWhen enabled, auth methods are mounted within the Vault mount table under the auth/ prefix. For example, enabling userpass at users/ allows interaction at auth/users. This convention ensures operators can consistently locate and manage auth methods, regardless of custom paths.

Reference: <https://developer.hashicorp.com/vault/docs/auth#enabling-disabling-auth-methods>

QUESTION 32

You want to integrate a third-party application to retrieve credentials from the HashiCorp Vault API. How can you accomplish this without having direct access to the source code?

- A. You cannot integrate a third-party application with Vault without being able to modify the source code
- B. Put in a request to the third-party application vendor
- C. Instead of the API, have the application use the Vault CLI to retrieve credentials
- D. Use the Vault Agent to obtain secrets and provide them to the application

Answer: D

Explanation:

Comprehensive and Detailed in Depth

Integrating a third-party application with Vault without modifying its source code requires a solution that handles authentication and secret retrieval externally, then delivers secrets in a way the application can consume (e.g., files or environment variables). Let's break this down:

Option A: You cannot integrate a third-party application with Vault without being able to modify the

source code

This is overly restrictive and incorrect. Vault provides tools like the Vault Agent, which can authenticate and fetch secrets on behalf of an application without requiring code changes. The agent can render secrets into a format (e.g., a file) that the application reads naturally. This option ignores Vault's flexibility for such scenarios. Incorrect.

Option B: Put in a request to the third-party application vendor

While this might eventually lead to native Vault support, it's impractical, slow, and depends on the vendor's willingness and timeline. It doesn't address the immediate need to integrate without source code access. This is a passive approach, not a technical solution within Vault's capabilities. Incorrect.

Option C: Instead of the API, have the application use the Vault CLI to retrieve credentials

The Vault CLI is designed for human operators or scripts, not seamless application integration. Thirdparty applications without source code modification can't invoke the CLI programmatically unless they're scripted to do so, which still requires external orchestration and isn't a clean solution. This approach is clunky, error-prone, and not suited for real-time secret retrieval in production. Incorrect.

Option D: Use the Vault Agent to obtain secrets and provide them to the application

The Vault Agent is a lightweight daemon that authenticates to Vault, retrieves secrets, and renders them into a consumable format (e.g., a file or environment variables) for the application. For example, if the application reads a config file, the agent can write secrets into that file using a template. This requires no changes to the application's code—just configuration of the agent and the application's environment. It's a standard, scalable solution for such use cases. Correct.

Detailed Mechanics:

The Vault Agent operates in two modes: authentication (to obtain a token) and secret rendering (via templates). For a third-party app, you'd configure the agent with an auth method (e.g., AppRole), a template (e.g., `{{ with secret "secret/data/my-secret" }}{{ .Data.data.key }}{{ end }}`), and a sink (e.g., `/path/to/app/config`). The agent runs alongside the app (e.g., as a sidecar in Kubernetes or a daemon on a VM), polls Vault for updates, and refreshes secrets as needed. The app remains oblivious to Vault, reading secrets as if they were static configs. This decoupling is key to integrating unmodified applications.

Real-World Example:

Imagine a legacy app that reads an API key from `/etc/app/key.txt`. The Vault Agent authenticates with Vault, fetches the key from `secret/data/api`, and writes it to `/etc/app/key.txt`. The app starts, reads the file, and operates normally—no code changes required.

Overall Explanation from Vault Docs:

æVault Agent¹ provides a simpler way for applications to integrate with Vault without requiring changes to application code¹. It renders templates containing secrets required by your application. This is ideal for third-party or legacy apps where source code access is unavailable.

Reference: <https://developer.hashicorp.com/vault/docs/agent-and-proxy/agent>

QUESTION 33

What API endpoint is used to manage secrets engines in Vault?

- A. /secret-engines/
- B. /sys/mounts
- C. /sys/capabilities
- D. /sys/kv

Answer: B

Explanation:

Comprehensive and Detailed in Depth

Vault's API provides endpoints for managing its components, including secrets engines, which generate and manage secrets (e.g., AWS, KV, Transit). Managing secrets engines involves enabling, disabling, tuning, or listing them. Let's evaluate:

Option A: /secret-engines/

This is not a valid Vault API endpoint. Vault uses /sys/ for system-level operations, and no endpoint named /secret-engines/ exists in the official API documentation. It's a fabricated path, possibly a misunderstanding of secrets engine management. Incorrect.

Option B: /sys/mounts

This is the correct endpoint. The /sys/mounts endpoint allows operators to list all mounted secrets engines (GET), enable a new one (POST to /sys/mounts/<path>), or tune existing ones (POST to /sys/mounts/<path>/tune). For example, enabling the AWS secrets engine at aws/ uses POST /v1/sys/mounts/aws with a payload specifying the type (aws). This endpoint is the central hub for secrets engine management. Correct.

Option C: /sys/capabilities

The /sys/capabilities endpoint checks permissions for a token on specific paths (e.g., what capabilities like read or write are allowed). It's unrelated to managing secrets engines—it's for policy auditing, not mount operations. Incorrect.

Option D: /sys/kv

There's no /sys/kv endpoint. The KV secrets engine, when enabled, lives at a user-defined path (e.g., kv/), not under /sys/. System endpoints under /sys/ handle configuration, not specific secrets engine instances. Incorrect.

Detailed Mechanics:

The /sys/mounts endpoint interacts with Vault's mount table, a registry of all enabled backends (auth methods and secrets engines). A GET request to /v1/sys/mounts returns a JSON list of mounts, e.g., {"kv": {"type": "kv", "options": {"version": "2"}}}. A POST request to /v1/sys/mounts/my-mount with {"type": "kv"} mounts a new KV engine. Tuning (e.g., setting TTLs) uses /sys/mounts/<path>/tune. This endpoint's versatility makes it the go-to for secrets engine management.

Real-World Example:

To enable the Transit engine: `curl -X POST -H "X-Vault-Token: <token>" -d '{"type": "transit"}' http://0.0.0.1:8200/v1/sys/mounts/transit`. To list mounts: `curl -X GET -H "X-Vault-Token: <token>" http://0.0.0.1:8200/v1/sys/mounts`.

Overall Explanation from Vault Docs:

œThe /sys/mounts endpoint is used to manage secrets engines in Vault. List, enable, or tune mounts via this system endpoint.

Reference: <https://developer.hashicorp.com/vault/api-docs/system/mounts>

QUESTION 34

You are deploying Vault in a local data center, but want to be sure you have a secondary Vault cluster in the event the primary cluster goes offline. In the secondary data center, you have applications that are running, as they are architected to run active/active. Which type of replication would be best in this scenario?

- A. Disaster Recovery replication
- B. Performance replication

Answer: B

Explanation:

Comprehensive and Detailed in Depth

Vault supports two replication types: Performance Replication and Disaster Recovery (DR)

Replication, each serving distinct purposes. The scenario involves an on-premises primary cluster and a secondary cluster in another data center, with active/active applications needing Vault access. Let's analyze:

Option A: Disaster Recovery replication

DR replication mirrors the primary cluster's state (secrets, tokens, leases) to a secondary cluster, which remains in standby mode until activated (promoted) during a failover. It's designed for disaster scenarios where the primary is lost, not for active/active use. The secondary doesn't serve reads or writes until promoted, which doesn't suit applications actively running in the secondary data center.

Incorrect.

Option B: Performance replication

Performance replication creates an active secondary cluster that replicates data from the primary in near real-time. It supports read operations locally, reducing latency for applications in the secondary data center, and can handle writes (forwarded to the primary). This fits an active/active architecture, providing redundancy and performance. If the primary fails, the secondary can continue serving reads (though writes need reconfiguring). Correct.

Detailed Mechanics:

Performance replication uses a primary-secondary model with log shipping via Write-Ahead Logs (WALs). The secondary maintains its own storage, synced from the primary, and can serve reads independently. Writes are forwarded to the primary, ensuring consistency. In an active/active setup, applications in both data centers can query their local Vault cluster, leveraging the secondary's read capability. DR replication, conversely, keeps the secondary dormant, requiring manual promotion, which introduces downtime unsuitable for active apps.

Real-World Example:

Primary cluster at `dc1.vault.local:8200`, secondary at `dc2.vault.local:8200`. Apps in DC2 query the secondary for secrets (e.g., `GET /v1/secret/data/my-secret`), avoiding cross-DC latency. If DC1 fails, DC2 continues serving cached reads until a new primary is established.

Overall Explanation from Vault Docs:

Performance replication allows secondary clusters to serve reads locally, ideal for active/active setups. DR replication is for failover, keeping secondaries in standby.

Reference: <https://developer.hashicorp.com/vault/docs/enterprise/replication>

QUESTION 35

How long does the Transit secrets engine store the resulting ciphertext by default?

- A. 24 hours
- B. 30 days
- C. 32 days
- D. Transit does not store data

Answer: D

Explanation:

Comprehensive and Detailed in Depth

The Transit secrets engine in Vault is designed for encryption-as-a-service, not data storage. Let's evaluate:

Option A: 24 hours

Transit doesn't store ciphertext, so no TTL applies. Incorrect.

Option B: 30 days

No storage means no 30-day retention. Incorrect.

Option C: 32 days

This aligns with token TTLs, not Transit behavior. Incorrect.

Option D: Transit does not store data

Transit encrypts data and returns the ciphertext to the caller without persisting it in Vault. Correct.

Detailed Mechanics:

When you run `vault write transit/encrypt/mykey plaintext=<base64-data>`, Vault uses the named key

(e.g., mykey) to encrypt the input and returns a response like vault:v1:< ciphertext>. This ciphertext is not stored in Vault's storage backend (e.g., Consul, Raft); it's the client's responsibility to save it (e.g., in a database). This stateless design keeps Vault lightweight and secure, avoiding data retention risks.

Real-World Example:

Encrypt a credit card: vault write transit/encrypt/creditcard plaintext=\$(base64 <<< "1234-5678-9012-3456"). Response: ciphertext=vault:v1:<data>. You store this in your app's database; Vault retains nothing.

Overall Explanation from Vault Docs:

œVault does NOT store any data encrypted via the transit/encrypt endpoint! The ciphertext is returned to the caller for storage elsewhere.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/transit>

QUESTION 36

Which of the following policies would permit a user to generate dynamic credentials on a database?

- A. path "database/creds/read_only_role" { capabilities = ["generate"] }
- B. path "database/creds/read_only_role" { capabilities = ["update"] }
- C. path "database/creds/read_only_role" { capabilities = ["list"] }
- D. path "database/creds/read_only_role" { capabilities = ["read"] }

Answer: D

Explanation:

Comprehensive and Detailed in Depth

The Database secrets engine generates dynamic credentials for database access. The endpoint database/creds/<role> (e.g., read_only_role) provides these credentials via a read operation. Let's analyze:

Option A: capabilities = ["generate"]

There's no generate capability in Vault policies. Capabilities are create, read, update, delete, list, etc. This is invalid. Incorrect.

Option B: capabilities = ["update"]

update (PUT) modifies existing data, not generates credentials. The creds endpoint uses GET. Incorrect.

Option C: capabilities = ["list"]

list retrieves metadata or paths, not credential data. Incorrect.

Option D: capabilities = ["read"]

Generating dynamic credentials involves a GET request to database/creds/<role>, mapped to the read capability. This policy allows it. Correct.

Detailed Mechanics:

For a role `read_only_role` defined with `vault write database/roles/read_only_role db_name=my-db creation_statements="CREATE USER..."`, a user with read on `database/creds/read_only_role` can run `vault read database/creds/read_only_role` to get temporary credentials. Vault's policy system aligns HTTP verbs to capabilities: GET = read, PUT = update. This counterintuitive mapping (GET for creation) is specific to dynamic secrets.

Overall Explanation from Vault Docs:

⌘Generating database credentials requires read capability on `database/creds/<role>`! Despite creating credentials, the HTTP request is a GET.

Reference: <https://developer.hashicorp.com/vault/tutorials/db-credentials/database-secrets>

QUESTION 37

Which scenario most strongly indicates a need to run a self-hosted Vault cluster instead of using HCP Vault Dedicated?

- A. Your organization doesn't require any custom security policies or intricate network topologies
- B. You want to offload all operational tasks and rely on HashiCorp to manage patching, upgrades, and infrastructure
- C. You prefer a fully managed environment that is readily scalable with minimal configuration overhead
- D. You must maintain specific compliance or custom integration requirements that demand full control over the Vault environment, including infrastructure provisioning and plugin development

Answer: D

Explanation:

Comprehensive and Detailed in Depth

HCP Vault Dedicated is a managed service, while self-hosted Vault (Community or Enterprise) requires user management. Let's evaluate:

A: Simple needs favor HCP Vault's managed simplicity. Incorrect.

B: Offloading tasks aligns with HCP Vault, not self-hosted. Incorrect.

C: Managed scalability suits HCP Vault. Incorrect.

D: Compliance, custom integrations, and plugin development need full control, only possible with self-hosted Vault. Correct.

Detailed Mechanics:

Self-hosted Vault allows custom plugins, FIPS 140-2 compliance, and specific network configs (e.g., air-gapped setups), unavailable in HCP Vault Dedicated due to its standardized, managed nature.

Overall Explanation from Vault Docs:

⌘Self-managed Vault supports custom requirements! HCP Vault Dedicated offloads operations but

limits control.

Reference: <https://developer.hashicorp.com/vault/tutorials/get-started/available-editions>

QUESTION 38

From the options below, select the benefits of using a batch token over a service token (select four).

- A. Often used for ephemeral, high-performance workloads
- B. Can be a root token
- C. Can be used on performance replication clusters (if orphan)
- D. Has accessors
- E. Lightweight and scalable
- F. No storage cost for token creation

Answer: A, C, E, F

Explanation:

Comprehensive and Detailed in Depth

Batch tokens are lightweight alternatives to service tokens, with trade-offs. Let's analyze:

A: Designed for short-lived, high-performance tasks. Correct.

B: Cannot be root tokens; root status is service-token-specific. Incorrect.

C: Orphan batch tokens work in replication. Correct.

D: No accessors; unique to service tokens. Incorrect.

E: Minimal overhead makes them scalable. Correct.

F: No disk storage reduces cost. Correct.

Overall Explanation from Vault Docs:

æBatch tokens are encrypted blobs| lightweight, scalable, no storage cost, ideal for ephemeral workloads.

Reference: <https://developer.hashicorp.com/vault/tutorials/tokens/batch-tokens>

QUESTION 39

Which of the following are accurate statements regarding the use of a KV v2 secrets engine (select three)?

- A. Issuing a vault kv destroy command permanently deletes the current version of the secret
- B. Issuing a vault kv destroy command deletes all versions of a secret
- C. Issuing a vault kv delete command performs a soft delete of the current version
- D. Issuing a vault kv metadata delete command permanently deletes the secret

Answer: A, C, D

Explanation:

Comprehensive and Detailed in Depth

KV v2 supports versioning. Let's evaluate:

A: destroy removes a specific version permanently. Correct.

B: destroy targets specified versions, not all. Incorrect.

C: delete soft-deletes the current version. Correct.

D: metadata delete removes all versions and metadata. Correct.

Overall Explanation from Vault Docs:

ækv delete soft-deletes| kv destroy permanently removes versions| kv metadata delete wipes everything.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/kv/kv-v2>

QUESTION 40

Which of the following is NOT a valid way in which a lease can be revoked in Vault?

A. Using the user interface (UI)

B. Automatically when the TTL or Max-TTL expires

C. Using the API to call the /v1/sys/leases endpoint

D. Via the CLI using the vault token command

Answer: D

Explanation:

Comprehensive and Detailed in Depth

Leases manage dynamic secrets' lifecycles. Let's check:

A: UI allows lease revocation. Valid.

B: TTL expiration auto-revokes leases. Valid.

C: API endpoint revokes leases. Valid.

D: vault token manages tokens, not leases directly. Invalid.

Overall Explanation from Vault Docs:

æLeases can be revoked via API, UI, CLI (vault lease revoke), or TTL expiry| vault token is for tokens.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/lease>

QUESTION 41

True or False? Once you create a KV v1 secrets engine and place data in it, there is no way to modify the mount to include the features of a KV v2 secrets engine.

A. True

B. False

Answer: B

Explanation:

Comprehensive and Detailed in Depth

A: Incorrect; KV v1 can be upgraded to v2.

B: Correct; vault kv enable-versioning upgrades it.

Overall Explanation from Vault Docs:

œkv enable-versioning turns on versioning for an existing KV v1 engine at its path.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/kv/kv-v2#upgrading-from-version-1>

QUESTION 42

Given the following screenshot, how many secrets engines have been enabled by a Vault user?

The screenshot displays the Vault web interface. On the left, a dark sidebar lists navigation options, with 'Secrets Engines' highlighted. The main panel shows the 'Secrets Engines' configuration page. At the top of this panel are two search boxes: 'Filter by engine type' and 'Filter by engine name'. Below these, a list of enabled secrets engines is shown. The list includes: 'certs/' (pki_5f4e8adf), 'cubbyhole/' (cubbyhole_f333a76e, per-token private secret storage), 'kv/' (kv_e8bacb66), and 'transit/' (transit_0e77e4be). Each entry is preceded by a small icon representing its type (lock, list, or double-headed arrow). A large, semi-transparent watermark 'Certkingdom.com' is visible across the center of the image.

A. 2

B. 3

- C. 4
- D. 5

Answer: B

Explanation:

Comprehensive and Detailed in Depth

cubbyhole is default; kv and transit are user-enabled. Total: 3.

Overall Explanation from Vault Docs:

œcubbyhole is enabled by default| User-enabled engines add to this.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/cubbyhole>

QUESTION 43

When configuring Vault replication and monitoring its status, you keep seeing something called 'WALs'. What are WALs?

- A. Warning of allocated logs
- B. Write along logging
- C. Write-ahead logs
- D. Wake after LAN

Answer: C

Explanation:

Comprehensive and Detailed in Depth

C: WALs (Write-Ahead Logs) ensure data consistency in replication. Correct.

Overall Explanation from Vault Docs:

œReplication uses Write-Ahead Logs (WALs) for log shipping between clusters|

Reference: <https://developer.hashicorp.com/vault/docs/internals/replication>

QUESTION 44

A Jenkins server is using the following token to access Vault. Based on the lookup shown below, what type of token is this?

\$ vault token lookup hvs.FGP1A77Hxa1Sp6Pkp1yURcZB

Key Value

--- -----

accessor RnH8jtgrxBrYanizlyJ7Y8R

creation_time 1604604512

creation_ttl 24h

display_name token
entity_id n/a
expire_time 2025-11-06T14:28:32.8891566-05:00
explicit_max_ttl 0s
id hvs.FGP1A77Hxa1Sp6KRau5eNB
issue_time 2025-11-06T14:28:32.8891566-05:00
meta <nil>
num_uses 0
orphan false
path auth/token/create
period 24h
policies [admin default]
renewable true
ttl 23h59m50s
type service

- A. Periodic token
- B. Batch token
- C. Orphaned token
- D. Secondary token

Answer: A

Explanation:

Comprehensive and Detailed in Depth

A: period indicates a renewable periodic token. Correct.

Overall Explanation from Vault Docs:

œA periodic token has a period, renewable without a max TTL.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/tokens#token-time-to-liveperiodic-tokens-and-explicit-max-ttls>

QUESTION 45

After encrypting data using the Transit secrets engine, you've received the following output. Which of the following is true based on the output displayed below?

Key: ciphertext Value:

vault:v2:45f9zW6cglbrzCjI0yCyC6DBYtSBSxnMgUn9B5aHcGEit71xefPEmmjMbrk3

- A. The original encryption key has been rotated at least once
- B. The data is stored in Vault using a KV v2 secrets engine

C. This is the second version of the encrypted data

D. Similar to the KV secrets engine, the Transit secrets engine was enabled using the transit v2 option

Answer: A

Explanation:

Comprehensive and Detailed in Depth

A: v2 shows the key was rotated once. Correct.

B: Transit doesn't store data. Incorrect.

C: v2 is the key version, not data version. Incorrect.

D: No transit v2 option exists. Incorrect.

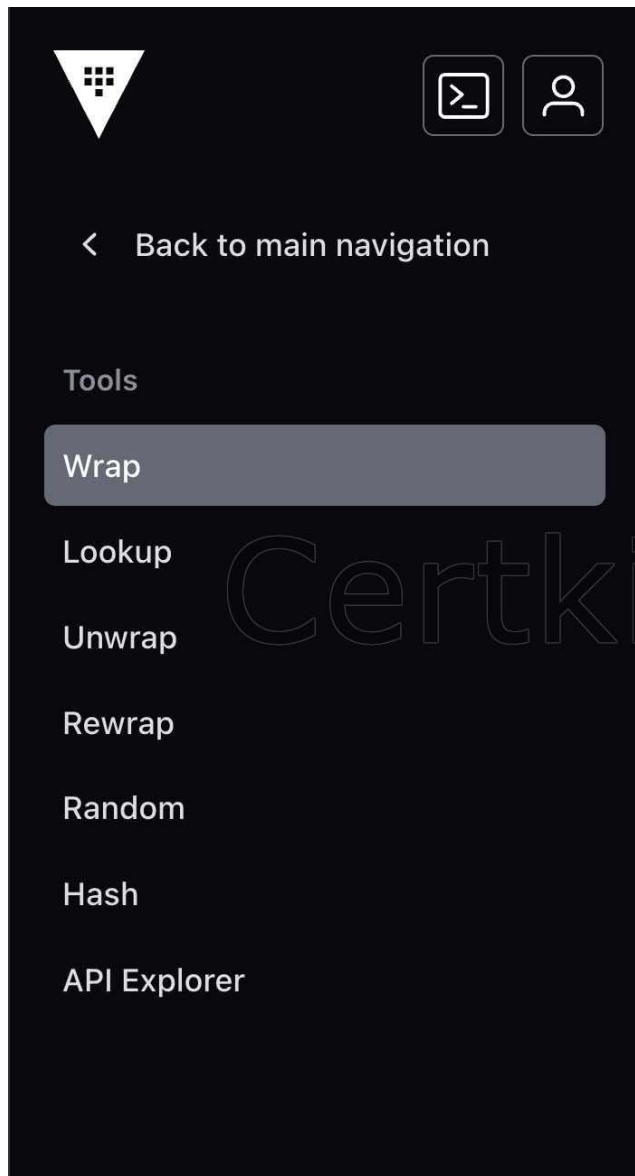
Overall Explanation from Vault Docs:

œCiphertext is prepended with the key version (e.g., v2)| Indicates rotation.

Reference: <https://developer.hashicorp.com/vault/tutorials/encryption-as-a-service/eastransit#rotate-the-encryption-key>

QUESTION 46

What could you do with the feature found in the screenshot below (select two)?



Wrap Data

Data to wrap (json-formatted)

```
1 {  
2 }
```

☐ Wrap TTL

Vault will use the default (30m)

- A. Using a short TTL, you could encrypt data in order to place only the encrypted data in Vault
- B. Encrypt the Vault master key that is stored in memory
- C. Encrypt sensitive data to send to a colleague over email
- D. Use response-wrapping to protect data

Answer: C, D

Explanation:

Comprehensive and Detailed in Depth

The screenshot highlights Vault's response wrapping feature, accessible via the UI's "Wrap" option. This feature wraps a Vault response (e.g., a secret or token) in a single-use token with a configurable TTL, ensuring secure delivery to an intended recipient. Let's evaluate each option against this capability:

Option A: Using a short TTL, you could encrypt data in order to place only the encrypted data in Vault. This misinterprets response wrapping. Wrapping doesn't encrypt data for storage in Vault; it secures a response for transmission outside Vault. Encryption for storage would involve the Transit secrets engine, not wrapping. The TTL in wrapping limits the wrapped token's validity, not the data's encryption lifecycle. This option conflates two unrelated features and is incorrect.

Vault Docs Insight: "Response wrapping does not store data in Vault; it delivers it securely to a recipient. (No direct storage implication.)"

Option B: Encrypt the Vault master key that is stored in memory

The master key in Vault is already encrypted at rest (in storage) and decrypted in memory during operation using the unseal process (e.g., Shamir shares or auto-unseal). Response wrapping doesn't interact with the master key—it's a client-facing feature for secret delivery, not an internal encryption mechanism. This is a fundamental misunderstanding of Vault's architecture and wrapping's purpose. Incorrect.

Vault Docs Insight: "The master key is managed by the seal mechanism, not client-facing features like wrapping. (See seal/unseal docs.)"

Option C: Encrypt sensitive data to send to a colleague over email

This aligns perfectly with response wrapping. You can retrieve a secret (e.g., `vault read secret/data/my-secret`), wrap it with a short TTL (e.g., 5 minutes), and receive a token (e.g., `hvs.<token>`). You email this token to a colleague, who unwraps it with `vault unwrap <token>` to access the secret. The data is encrypted within the token, secure during transit, and expires after the TTL. This is a textbook use case for wrapping. Correct.

Vault Docs Insight: "Response wrapping¹ can be used to securely send sensitive data to another party, such as over email, with a limited lifetime. (Directly supported use case.)"

Option D: Use response-wrapping to protect data

This is the essence of the feature. Wrapping protects data by encapsulating it in a single-use token, accessible only via an unwrap operation. For example, `vault write -wrap-ttl=60s secret/data/mysecret` returns a wrapped token, protecting the secret until unwrapped. This ensures confidentiality and controlled access, making it a core benefit of the feature. Correct.

Vault Docs Insight: "Vault can wrap a response in a single-use token¹ protecting the data until unwrapped by the recipient. (Core definition.)"

Detailed Mechanics:

Response wrapping works by taking a Vault API response (e.g., a secret's JSON payload) and storing it in the cubbyhole secrets engine under a newly generated single-use token. The token's TTL (e.g.,

60s) limits its validity. The API call POST /v1/sys/wrapping/wrap with a payload (e.g., {"ttl": "60s", "data": {"key": "value"}}) returns {"wrap_info": {"token": "hvs.<token>"} }. The recipient uses vault unwrap hvs.<token> (or POST /v1/sys/wrapping/unwrap) to retrieve the original data. Once unwrapped, the token is revoked, ensuring one-time use. This leverages Vault's encryption and token system for secure data exchange.

Real-World Example:

You generate an API key in Vault: vault write secret/data/api key=abc123. In the UI, you click æWrap with a 5-minute TTL, getting hvs.XYZ. You email hvs.XYZ to a colleague, who runs vault unwrap hvs.XYZ within 5 minutes to get key=abc123. After unwrapping, the token is invalid, and the secret is safe from interception.

Overall Explanation from Vault Docs:

æVault includes a feature called response wrapping. When requested, Vault can take the response it would have sent to an HTTP client and instead insert it into the cubbyhole of a single-use token, returning that token instead. This is useful for securely delivering sensitive data. The feature excels at protecting data in transit (e.g., email) and enforcing one-time access, not internal key management or storage encryption.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/response-wrapping>

Additional Reference: <https://developer.hashicorp.com/vault/docs/secrets/cubbyhole>

QUESTION 47

From the options below, select the benefits of using the PKI (x.509 certificates) secrets engine (select three):

- A. TTLs on Vault certs are longer to ensure certificates are valid for a longer period of time
- B. Reducing, or eliminating certificate revocations
- C. Reduces time to get a certificate by eliminating the need to generate a private key and CSR
- D. Vault can act as an intermediate CA

Answer: B, C, D

Explanation:

Comprehensive and Detailed in Depth

The PKI secrets engine in Vault generates dynamic X.509 certificates, acting as a certificate authority (CA) to streamline certificate management. Let's assess each option based on its documented benefits:

Option A: TTLs on Vault certs are longer to ensure certificates are valid for a longer period of time
This is misleading. Vault's PKI engine allows configurable TTLs, but the recommendation is for short TTLs (e.g., hours or days) to reduce the need for revocation and enhance security. Long TTLs increase exposure if a certificate is compromised, requiring revocation and larger Certificate Revocation Lists

(CRLs). The engine's benefit isn't longer validity—it's flexibility and automation, not extended lifetimes. Incorrect.

Vault Docs Insight: By keeping TTLs relatively short, revocations are less likely, helping scale to large workloads. (Short TTLs are preferred.)

Option B: Reducing, or eliminating certificate revocations

A key advantage of the PKI engine is issuing short-lived certificates. With short TTLs (e.g., 24h), certificates expire naturally before revocation is needed, minimizing CRL maintenance. For example, an app can fetch a new cert daily, reducing revocation events compared to traditional multi-year certs. This aligns with Vault's ephemeral certificate model. Correct.

Vault Docs Insight: By keeping TTLs relatively short, revocations are less likely to be needed, keeping CRLs short. (Direct benefit.)

Option C: Reduces time to get a certificate by eliminating the need to generate a private key and CSR. Traditionally, obtaining a certificate involves generating a private key, creating a Certificate Signing Request (CSR), and submitting it to a CA—a manual, time-consuming process. The PKI engine automates this: vault write pki/issue/my-role common_name=app.example.com instantly generates a private key and signed certificate. This eliminates manual steps, speeding up issuance significantly. Correct.

Vault Docs Insight: Services can get certificates without generating a private key and CSR, submitting to a CA, and waiting. (Automation reduces time.)

Option D: Vault can act as an intermediate CA

The PKI engine can be configured as an intermediate CA, signed by a root CA (internal or external). For example, vault write pki/intermediate/generate/internal common_name="Intermediate CA" creates an intermediate, which can issue certificates under a trust chain. This supports hierarchical PKI setups, a major feature. Correct.

Vault Docs Insight: The PKI secrets engine can act as an intermediate CA, issuing certificates on behalf of a root CA. (Explicit capability.)

Detailed Mechanics:

The PKI engine operates at paths like pki/ (root) or pki_int/ (intermediate). Roles (e.g., my-role) define parameters like TTL and allowed domains. Issuing a cert (vault write pki/issue/my-role) returns a JSON payload with certificate, private_key, and issuing_ca. Short TTLs leverage Vault's lease system, auto-revoking certs on expiry. As an intermediate CA, it signs certificates with its key, validated against a root, enhancing trust management.

Real-World Example:

An app needs a cert: vault write pki/issue/web common_name=web.example.com ttl=24h. Vault returns a cert and key instantly, valid for 24 hours. No CSR, no revocation needed—expires tomorrow. Another PKI mount at pki_int/ issues certs under a corporate root CA.

Overall Explanation from Vault Docs:

The PKI secrets engine generates dynamic X.509 certificates. Services can get certificates without the usual manual process. By keeping TTLs short, revocations are less likely. Vault can act as an

intermediate CA, issuing certificates efficiently. These benefits”automation, reduced revocation, and CA flexibility”define its value.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/pki>

QUESTION 48

According to the screenshot below, what auth method did this client use to log in to Vault?

(Screenshot shows a lease path: auth/userpass/login/student01)

- A. Userpass
- B. Auth
- C. Root token
- D. Child token

Answer: A

Explanation:

Comprehensive and Detailed in Depth

The screenshot provides a lease path: auth/userpass/login/student01, which reveals the authentication method used to generate the token tied to this lease. Vault's auth methods create tokens at specific paths, and the path structure indicates the method.

Option A: Userpass

The path auth/userpass/login/student01 explicitly includes userpass, matching the userpass auth method. This method authenticates users with a username (e.g., student01) and password, typically via vault login -method=userpass username=student01. The /login endpoint confirms a login operation, and the lease ties to the resulting token. This is the clear, correct answer based on the path. Correct.

Vault Docs Insight: æThe userpass auth method allows users to authenticate with a username and password| mounted at auth/userpass by default. (Matches the path.)

Option B: Auth

æAuth isn't an auth method”it's the namespace prefix (auth/) for all auth methods in Vault (e.g., auth/token, auth/userpass). The screenshot specifies userpass within auth/, not a generic æauth method. This option is a misnomer and incorrect.

Vault Docs Insight: æAll auth methods are mounted under auth/| æauth' itself is not a method. (Clarifies structure.)

Option C: Root token

A root token is a privileged token type, not an auth method. It's created during Vault initialization or via auth/token/create with root privileges, not through a login path like auth/userpass/login. The screenshot's path indicates a userpass login, not a root token usage. Incorrect.

Vault Docs Insight: æRoot tokens are created at initialization| not tied to a specific auth method login

path. (Distinct from userpass.)

Option D: Child token

A child token is a token created by a parent token (e.g., via vault token create), not an auth method.

The path auth/userpass/login/student01 shows a login event, not a token creation event (which would be auth/token/create). This option confuses token hierarchy with authentication. Incorrect.

Vault Docs Insight: æChild tokens are created by parent tokens| not directly via login endpoints.

(Different mechanism.)

Detailed Mechanics:

When a user logs in with vault login -method=userpass -path=userpass username=student01, Vault hits the endpoint POST /v1/auth/userpass/login/student01 with a password payload. Success generates a token, and a lease is created at auth/userpass/login/student01 with a TTL. The screenshot's lease path directly reflects this process, pinpointing userpass as the method.

Real-World Example:

Enable userpass: vault auth enable userpass. Add user: vault write auth/userpass/users/student01 password=secret. Login: vault login -method=userpass username=student01. The token's lease appears as auth/userpass/login/student01.

Overall Explanation from Vault Docs:

æThe lease shown lives at auth/userpass/login/<username> and indicates the userpass auth method was used to obtain a token| The userpass method authenticates via username/password at its mount path. The path structure is a definitive indicator.

Reference: <https://developer.hashicorp.com/vault/docs/auth/userpass>

QUESTION 49

What is the difference between the TTL and the Max TTL (select two)?

- A. The TTL defines when the token will expire and be revoked
- B. The TTL defines when another token will be generated
- C. The Max TTL defines the timeframe for which a token cannot be used
- D. The Max TTL defines the maximum timeframe for which a token can be renewed

Answer: A, D

Explanation:

Comprehensive and Detailed in Depth

Vault tokens have two key time attributes: TTL (Time-To-Live) and Max TTL (Maximum Time-To-Live), governing their lifecycle. Let's dissect each option:

Option A: The TTL defines when the token will expire and be revoked

The TTL is the current lifespan of a token before it expires. For example, a token with a TTL of 24h (vault token create -ttl=24h) expires 24 hours from creation unless renewed. Upon expiry, Vault

revokes it automatically. This is a fundamental property of TTL, making this statement accurate.

Correct.

Vault Docs Insight: æThe TTL defines when the token will expire| if it reaches its TTL, it will be revoked by Vault. (Core definition.)

Option B: The TTL defines when another token will be generated

TTL governs expiration, not token generation. New tokens are created explicitly (e.g., vault token create) or via auth methods, not automatically by TTL. This misunderstands TTL's role—it's about expiry, not regeneration. Incorrect.

Vault Docs Insight: æTTL is the duration until expiration| New tokens are not generated by TTL. (No generation link.)

Option C: The Max TTL defines the timeframe for which a token cannot be used

This is backwards. Max TTL sets the upper limit a token can exist through renewals, not a period of inactivity or unusability. A token with a Max TTL of 72h can be renewed up to 72 hours from creation, after which it's revoked. This option inverts the concept. Incorrect.

Vault Docs Insight: æMax TTL defines the maximum timeframe for which the token can be renewed| not a usage restriction. (Opposite meaning.)

Option D: The Max TTL defines the maximum timeframe for which a token can be renewed

Max TTL caps the total lifespan of a token, including renewals. For example, a token with TTL=24h and Max TTL=72h (vault token create -ttl=24h -explicit-max-ttl=72h) can be renewed twice (24h + 24h + 24h = 72h) before hitting the limit. Beyond 72h, renewal fails, and it expires. This is the precise definition of Max TTL. Correct.

Vault Docs Insight: æThe Max TTL defines the maximum timeframe for which the token can be renewed| Once reached, it cannot be renewed further. (Exact match.)

Detailed Mechanics:

TTL is dynamic, decreasing as time passes (e.g., vault token lookup shows ttl: 23h59m50s after 10 seconds). Renewal (vault token renew) resets TTL to its original value (e.g., 24h), but only up to Max TTL from creation. System defaults (768h days) apply unless overridden. Periodic tokens (-period=24h) renew indefinitely within their period, ignoring Max TTL unless explicitly set.

Real-World Example:

Create: vault token create -ttl=1h -explicit-max-ttl=3h. After 1h, TTL=0, renewable. Renew at 2h total, TTL=1h again. At 3h total, Max TTL hits—revoked. Contrast with TTL-only: vault token create -ttl=1h, renewable up to system Max TTL (768h).

Overall Explanation from Vault Docs:

æThe TTL defines when the token will expire| If it reaches its TTL, it will be immediately revoked by Vault. The Max TTL defines the maximum timeframe for which the token can be renewed| Once the Max TTL is reached, the token cannot be renewed any longer and will be revoked. These attributes ensure controlled token lifecycles.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/tokens#token-time-to-liveperiodic-tokens-and-explicit-max-ttls>

QUESTION 50

What is true about the output of the following command (select three)?

```
$ vault operator init -key-shares=3 -key-threshold=2 \
> -pgp-keys="keybase:janedoe,keybase:john doe,keybase:student01"
```

- A. The admin never sees all the unseal keys and cannot unseal Vault by themselves
- B. All three users, Jane/John/Student01, will receive all unseal keys and can unseal Vault
- C. The admin will receive the unseal keys and be able to unseal Vault themselves
- D. The keys will be returned encrypted
- E. Each individual can only decrypt their own unseal key using their private PGP key

Answer: A, D, E

Explanation:

Comprehensive and Detailed in Depth

The command initializes Vault, splitting the master key into 3 shares (threshold 2) and encrypting each with PGP keys for Jane, John, and Student01. Let's analyze:

Option A: The admin never sees all the unseal keys and cannot unseal Vault by themselves

With -pgp-keys, Vault encrypts each share with a user's public PGP key. The admin (initializer) sees only encrypted outputs (e.g., Key 1: <encrypted>), not plaintext keys. Since 2 shares are needed and no single entity gets all, the admin can't unseal alone. Correct.

Vault Docs Insight: "The initializer receives encrypted keys| never sees all plaintext keys, enhancing security. (Directly stated.)"

Option B: All three users, Jane/John/Student01, will receive all unseal keys and can unseal Vault

Each user gets one encrypted share (e.g., Jane gets Key 1, John Key 2). No user receives all shares" only one, decryptable with their private key. Unsealing requires collaboration (2 of 3), so this is false. Incorrect.

Vault Docs Insight: "Each PGP key encrypts one share| No single user gets all keys. (Distribution is per-user.)"

Option C: The admin will receive the unseal keys and be able to unseal Vault themselves

Without PGP, the admin gets plaintext keys. With -pgp-keys, they get encrypted keys they can't decrypt (lacking private keys). Threshold=2 means collaboration is required. Incorrect.

Vault Docs Insight: "Using PGP keys ensures the initializer cannot unseal alone| (Security feature.)"

Option D: The keys will be returned encrypted

The -pgp-keys flag encrypts each share with the corresponding public key. Output shows encrypted blobs (e.g., base64-encoded PGP ciphertext), not plaintext. Correct.

Vault Docs Insight: "Vault will generate the unseal keys and encrypt them using the given PGP keys|"

(Explicit behavior.)

Option E: Each individual can only decrypt their own unseal key using their private PGP key. Each share is encrypted with one user's public key (e.g., Jane's key encrypts Key 1). Only Jane's private key decrypts it. This ensures secure distribution. Correct.

Vault Docs Insight: "Only the owner of the corresponding private key can decrypt the value" (PGP security.)

Detailed Mechanics:

Command: `vault operator init -key-shares=3 -key-threshold=2 -pgpkeys="jane.pgp,john.pgp,student01.pgp"`. Vault generates 3 shares via Shamir's Secret Sharing, encrypts each (Key 1 with jane.pgp, etc.), and outputs encrypted strings. Unsealing requires 2 decrypted shares combined via `vault operator unseal`. PGP ensures the admin can't access plaintext, enforcing split knowledge.

Real-World Example:

Output: Key 1: <encrypted-jane>, Key 2: <encrypted-john>, Key 3: <encrypted-student01>. Jane decrypts Key 1 with `gpg -d`, John decrypts Key 2. They submit via UI or CLI to unseal.

Overall Explanation from Vault Docs:

"Vault can optionally be initialized using PGP keys. In this mode, Vault will generate the unseal keys and immediately encrypt them using the given users' public PGP keys. Only the owner of the corresponding private key is able to decrypt the value. The initializer never sees all plaintext keys and cannot unseal Vault alone. This enhances security by distributing trust."

Reference: <https://developer.hashicorp.com/vault/docs/commands/operator/init#pgp-keys>

QUESTION 51

How can Vault be used to programmatically obtain a generated code for MFA, somewhat similar to Google Authenticator?

- A. Cubbyhole
- B. The random byte generator
- C. TOTP secrets engine
- D. The identity secrets engine

Answer: C

Explanation:

Comprehensive and Detailed in Depth

Vault can generate time-based one-time passwords (TOTP) for multi-factor authentication (MFA), mimicking apps like Google Authenticator. Let's evaluate:

Option A: Cubbyhole

Cubbyhole is a per-token secret store, not a TOTP generator. It's for temporary secret storage, not

MFA code generation. Incorrect.

Vault Docs Insight: æCubbyhole stores secrets tied to a token| no TOTP functionality. (Different purpose.)

Option B: The random byte generator

Vault's /sys/tools/random endpoint generates random bytes, not time-based codes synced with a clock (TOTP requirement). It's for generic randomness, not MF

A. Incorrect.

Vault Docs Insight: æRandom bytes are not time-based| unsuitable for TOTP. (Unrelated feature.)

Option C: TOTP secrets engine

The TOTP engine generates and validates TOTP codes (e.g., 6-digit codes every 30s) using a shared secret, just like Google Authenticator. You create a key (vault write totp/keys/my-key) and fetch codes (vault read totp/code/my-key). Perfect for programmatic MF

A. Correct.

Vault Docs Insight: æThe TOTP secrets engine can act as a TOTP code generator| replacing traditional generators like Google Authenticator. (Exact match.)

Option D: The identity secrets engine

The Identity engine manages user/entity identities and policies, not TOTP codes. It's for identity management, not MFA generation. Incorrect.

Vault Docs Insight: æIdentity engine handles identity data| no TOTP generation. (Different scope.)

Detailed Mechanics:

Enable: vault secrets enable totp. Create key: vault write totp/keys/my-key issuer=Vault. Get code: vault read totp/code/my-key returns {"data":{"code":"123456"}}. Codes sync with time (RFC 6238), usable in APIs or apps.

Overall Explanation from Vault Docs:

æThe TOTP secrets engine can act as a TOTP code generator| It provides an added layer of security since the ability to generate codes is guarded by policies and audited.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/totp>

QUESTION 52

From the options below, select the auth methods that are better suited for machine-to-machine authentication (select five):

- A. Kubernetes
- B. GitHub
- C. TLS
- D. Token
- E. AppRole
- F. AWS
- G. LDAP

H. OIDC

Answer: A, C, D, E, F

Explanation:

Comprehensive and Detailed in Depth

Machine-to-machine (M2M) auth methods in Vault enable automated systems to authenticate without human interaction. Let's assess:

A: Kubernetes - Uses service account tokens for pods. Correct.

Vault Docs Insight: æKubernetes auth| ideal for workloads in Kubernetes clusters.

B: GitHub - User-focused, requires human GitHub login. Incorrect.

Vault Docs Insight: æGitHub auth| typically for human users.

C: TLS - Certificate-based, perfect for M2M. Correct.

Vault Docs Insight: æTLS auth uses certificates| suited for machine authentication.

D: Token - Pre-generated tokens for automation. Correct.

Vault Docs Insight: æToken auth| can be used by machines with proper management.

E: AppRole - RoleID/SecretID for apps. Correct.

Vault Docs Insight: æAppRole is designed for machine-to-machine authentication|

F: AWS - IAM roles for AWS resources. Correct.

Vault Docs Insight: æAWS auth| automated for AWS-based machines.

G: LDAP - User directory-based, human-oriented. Incorrect.

Vault Docs Insight: æLDAP| commonly for human user authentication.

H: OIDC - User SSO, not M2M. Incorrect.

Vault Docs Insight: æOIDC| for human single sign-on.

Overall Explanation from Vault Docs:

æExamples of machine auth methods include AppRole, AWS, Kubernetes, TLS, and Token| Human auth methods include LDAP, GitHub, OIDC.

Reference: <https://developer.hashicorp.com/vault/docs/auth>

QUESTION 53

You've hit the URL for the Vault UI, but you're presented with this screen. Why doesn't Vault present you with a way to log in?

Key shares

The number of key shares to split the master key into

Key threshold

The number of key shares required to reconstruct the master key

✓ Encrypt output with PGP

✓ Encrypt root token with PGP

Initialize



- A. The Consul storage backend was not configured correctly
- B. Vault needs to be initialized before it can be used
- C. A Vault policy is preventing you from logging in
- D. The Vault configuration file has an incorrect configuration

Answer: B

Explanation:

Comprehensive and Detailed in Depth

The initialization page means Vault is new or reset. Let's evaluate:

- A: Storage issues don't trigger this screen; they'd cause errors post-init. Incorrect.
- B: Vault requires initialization (vault operator init) to set up keys and enable login. Correct.
- C: Policies apply post-login, not pre-init. Incorrect.
- D: Config errors would prevent Vault from starting, not show this screen. Incorrect.

Overall Explanation from Vault Docs:

œBefore Vault can be used, it must be initialized and unsealed| This screen indicates Vault has not

been initialized yet.

Reference: <https://developer.hashicorp.com/vault/docs/commands/operator/init>

QUESTION 54

Which of the following secrets engines does NOT issue a lease upon a read request?

- A. KV
- B. Consul
- C. Database
- D. AWS

Answer: A

Explanation:

Comprehensive and Detailed in Depth

Leases tie to dynamic secrets with TTLs. Let's check:

A: KV - Static secrets, no lease on read. Correct.

B: Consul - Dynamic creds with leases. Incorrect.

C: Database - Dynamic creds with leases. Incorrect.

D: AWS - Dynamic creds with leases. Incorrect.

Overall Explanation from Vault Docs:

αThe Key/Value Backend¹ does not issue leases although it may return a lease duration.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/lease#lease-renew-and-revoke>

QUESTION 55

Which of the following statements best describes the difference in cluster strategies between selfmanaged Vault and HashiCorp-managed Vault?

- A. Self-managed clusters require users to handle setup, maintenance, and scaling, whereas HCP Vault Dedicated is fully managed by HashiCorp and offloads most operational tasks
- B. Neither self-managed clusters nor HCP Vault Dedicated include enterprise security features such as replication or disaster recovery
- C. Both self-managed clusters and HCP Vault Dedicated require manual patching and upgrades, but only self-managed clusters are hosted in the user's cloud
- D. In self-managed clusters, HashiCorp is responsible for scaling, upgrades, and patching, while HCP Vault Dedicated requires the user to handle all operational overhead

Answer: A

Explanation:

Comprehensive and Detailed in Depth

A: Correctly contrasts self-managed (user responsibility) with HCP Vault (HashiCorp-managed).

Correct.

B: Both support replication; false. Incorrect.

C: HCP Vault doesn't require manual upgrades. Incorrect.

D: Reverses responsibilities; false. Incorrect.

Overall Explanation from Vault Docs:

œHCP Vault Dedicated is operated by HashiCorp| Self-managed Vault requires users to handle setup, maintenance, and scaling.

Reference: <https://developer.hashicorp.com/hcp/docs/vault/what-is-hcp-vault>

QUESTION 56

What are the primary benefits of running Vault in a production deployment over dev server mode (select two)?

A. Faster deployment

B. Persistent storage

C. Ability to enable auth methods

D. Encryption via TLS

Answer: B, D

Explanation:

Comprehensive and Detailed in Depth

A: Dev mode is faster to deploy; incorrect.

B: Production uses persistent storage vs. dev's in-memory. Correct.

C: Auth methods work in both modes. Incorrect.

D: Production enables TLS; dev uses plaintext. Correct.

Overall Explanation from Vault Docs:

œDev server mode stores data in memory| Production mode supports persistent storage and TLS encryption.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/dev-server>

QUESTION 57

True or False? All dynamic secrets in Vault are required to have a lease.

A. True

B. False

Answer: A

Explanation:

Comprehensive and Detailed in Depth

A: All dynamic secrets (e.g., database creds) have leases for lifecycle management. Correct.

B: Incorrect; leases are mandatory for dynamic secrets.

Overall Explanation from Vault Docs:

œAll dynamic secrets in Vault are required to have a lease| forcing consumers to check in routinely.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/lease>

New Topic: Topic 2, Exam Pool B

QUESTION 58

You have a long-running app that cannot handle a regeneration of a token or secret. What type of token should be created for this application in order to authenticate and interact with Vault?

- A. Service Token with Use Limit
- B. Periodic Service Token
- C. Batch Token
- D. Orphan Token

Answer: B

Explanation:

Comprehensive and Detailed in Depth

For a long-running application that cannot handle token or secret regeneration, the Periodic Service Token is the most suitable choice. According to HashiCorp Vault documentation, periodic service tokens are renewable tokens that do not have a maximum Time-to-Live (TTL), meaning they can be renewed indefinitely by the client without requiring manual intervention or regeneration. This is ideal for applications needing continuous access to Vault over an extended period. The documentation states: "Periodic tokens have a TTL, but no max TTL. Periodic tokens may live for an infinite amount of time, so long as they are renewed within their TTL." This feature ensures uninterrupted operation for long-running processes, aligning perfectly with the scenario described. In contrast, a Service Token with Use Limit has a finite number of uses before expiration, making it unsuitable for continuous access without regeneration. A Batch Token is designed for short-lived, one-time operations or batch processes, not persistent access, as it lacks renewability and has a fixed TTL. An Orphan Token, while not tied to a parent token, does not inherently address the regeneration issue and is less secure for long-term use due to its lack of association with policies or identity. Thus,

the periodic service token stands out as the best fit.

Reference:

HashiCorp Vault Documentation - Tokens: Periodic Tokens

QUESTION 59

An application requires a specific key/value pair to be updated in order to process a batch job. The value should be either "true" or "false." However, when developers have been updating the value, sometimes they mistype the value or capitalize the value, causing the batch job not to run. What feature of a Vault policy can be used to restrict entry to the required values?

- A. Add a deny statement for all possible misspellings of the value
- B. Add an `allowed_parameters` value to the policy
- C. Change the policy to include the list capability
- D. Use a `*` wildcard at the end of the policy

Answer: B

Explanation:

Comprehensive and Detailed in Depth

To restrict the values of a key/value pair to only "true" or "false" and prevent mistyping or capitalization errors, the `allowed_parameters` feature in a Vault policy is the most effective solution. The HashiCorp Vault documentation explains that `allowed_parameters` can be used to "permit a list of keys and values that are permitted on the given path." By specifying `allowed_parameters` with the exact values "true" and "false," the policy ensures that only these values are accepted, rejecting any deviations (e.g., "True," "TRUE," or "flase"). This provides fine-grained control and eliminates the risk of human error impacting the batch job.

Adding a deny statement for all possible misspellings is impractical and error-prone, as it requires anticipating every potential mistake, which is neither scalable nor efficient. The list capability allows listing and reading values but does not restrict what can be written, failing to address the problem of enforcing specific values. Using a wildcard (`*`) at the end of the policy permits unrestricted values, which directly contradicts the need to limit entries to "true" or "false." Thus, `allowed_parameters` is the precise tool for this use case.

Reference:

HashiCorp Vault Documentation - Policies: Fine-Grained Control

QUESTION 60

Your application cannot manage authentication with Vault, but it can communicate with a local service to retrieve secrets. What solution can enable your app to generate dynamic credentials from Vault?

- A. Vault Proxy with caching feature enabled
- B. Vault Agent with environment variable secret injection
- C. Vault Proxy with Auto-Auth feature enabled
- D. Vault Agent with the templating feature configured

Answer: C

Explanation:

Comprehensive and Detailed in Depth

For an application that cannot manage authentication with Vault but can communicate with a local service, the Vault Proxy with Auto-Auth feature enabled is the optimal solution. The HashiCorp Vault documentation states that Vault Proxy can "act as a proxy between Vault and the application, optionally simplifying the authentication process." The Auto-Auth feature allows the proxy to handle authentication on behalf of the application, enabling it to generate dynamic credentials without the application needing to manage the authentication process directly. This aligns perfectly with the requirement of delegating authentication to a local service.

Vault Proxy with caching improves performance by caching responses but does not inherently handle authentication, missing the core need. Vault Agent with environment variable secret injection injects secrets into the application's environment but assumes the agent manages authentication, which the application cannot do. Vault Agent with templating generates credentials based on templates but still requires authentication management, which the application cannot handle. Vault Proxy with Auto-Auth uniquely addresses this by offloading authentication responsibilities.

Reference:

HashiCorp Vault Documentation - Vault Agent and Proxy

QUESTION 61

Which statement best describes the process of sealing a Vault instance?

- A. Disable the TLS certificates on the Vault server by running `vault secrets disable pki`, blocking all requests.
- B. Run `vault operator rotate` to rotate the Vault tokens for all clients, causing them to reauthenticate with the Vault.
- C. Run the `vault operator seal` command, which securely discards the master key from memory and prevents further operations until unsealed.
- D. Revoke all leases so no secrets can be accessed using `vault lease revoke`, but keep the master key in memory for quick recovery.

Answer: C

Explanation:

Comprehensive and Detailed in Depth

Sealing a Vault instance is a critical security operation that involves locking down the system to prevent access until it is explicitly unsealed. The HashiCorp Vault documentation states: "Sealing a Vault will throw away the root key in memory and require another unseal process to restore it." This is achieved by running the vault operator seal command, which "securely discards the master key from memory and prevents further operations until unsealed." This action ensures that no data can be accessed without the unseal process, making it the correct description of sealing.

Disabling TLS certificates via vault secrets disable pki affects the PKI secrets engine, not the sealing process. Running vault operator rotate rotates encryption keys, not seals the Vault. Revoking leases with vault lease revoke terminates secret access but keeps the master key in memory, unlike sealing, which discards it. Thus, only option C accurately reflects the sealing process.

Reference:

HashiCorp Vault Documentation - Seal and Unseal

HashiCorp Vault Documentation - Vault Operator Seal Command

QUESTION 62

The Vault Agent provides which of the following benefits? (Select three)

- A. Token renewal
- B. Authentication to Vault
- C. Client-side caching of responses
- D. Automatically creates secrets in the desired storage backend

Answer: A, B, C

Explanation:

Comprehensive and Detailed in Depth

The Vault Agent is a client daemon designed to simplify integration with Vault by providing several key benefits. According to the HashiCorp Vault documentation, these include:

Token Renewal: "Vault Agent automatically renews tokens issued by Vault," ensuring continuous access without manual intervention.

Authentication to Vault: "Vault Agent provides authentication to Vault," allowing applications to authenticate using their identity without managing tokens directly.

Client-side caching of responses: "Vault Agent offers client-side caching of responses," improving performance by reducing server requests.

However, automatically creating secrets in the desired storage backend is not a function of Vault Agent. Secret creation is handled by Vault's secrets engines, not the agent, which focuses on

authentication, token management, and caching. Thus, A, B, and C are the correct benefits.

Reference:

HashiCorp Vault Documentation - Vault Agent

QUESTION 63

You have deployed an application that needs to encrypt data before writing to a database. What secrets engine should you use?

- A. Transit
- B. SSH
- C. PKI
- D. TOTP

Answer: A

Explanation:

Comprehensive and Detailed in Depth

For encrypting data before writing it to a database, the Transit secrets engine is the appropriate choice. The HashiCorp Vault documentation describes it as handling "cryptographic functions on data in-transit" and notes that it "can be viewed as 'cryptography as a service' or 'encryption as a service.'" It is designed to encrypt data without storing it, making it ideal for applications needing to secure data before storage in an external database. The primary use case is "to encrypt data from applications while still storing that encrypted data in some primary data store."

The SSH secrets engine manages SSH keys and authentication, not data encryption. The PKI secrets engine handles certificate management, not general data encryption. The TOTP secrets engine generates time-based one-time passwords, unrelated to data encryption. Thus, Transit is the correct choice.

Reference:

HashiCorp Vault Documentation - Transit Secrets Engine

QUESTION 64

Which of the following statements best describes the difference between static and dynamic credentials in a secrets management system?

- A. They are functionally identical—the only difference is what secrets engine creates them.
- B. Static credentials only apply to specific use cases, while dynamic credentials can be used everywhere.
- C. Static credentials often remain persistent for long periods of time, while dynamic are short-lived and auto-rotated.

D. Static credentials are ephemeral and rotated frequently, while dynamic credentials remain unchanged indefinitely.

Answer: C

Explanation:

Comprehensive and Detailed in Depth

The key difference between static and dynamic credentials lies in their lifecycle and management.

The HashiCorp Vault documentation explains: "Static credentials are typically assigned and remain valid for long stretches, requiring manual rotation. By contrast, dynamic credentials are issued on demand, designed to be short-lived, and automatically rotated or revoked when they expire." This reduces exposure risk and simplifies management, making C the correct statement: "Static credentials often remain persistent for long periods of time, while dynamic are short-lived and autorotated."

Option A is incorrect as static and dynamic credentials differ in function, not just origin. Option B misstates their applicability—static credentials aren't limited to specific use cases, and dynamic credentials have specific purposes. Option D reverses the definitions entirely. Thus, C aligns with Vault's design.

Reference:

HashiCorp Vault Documentation - Static and Dynamic Secrets

QUESTION 65

Select the two default policies created in Vault. (Select two)

- A. root
- B. user
- C. admin
- D. default
- E. base
- F. vault

Answer: A, D

Explanation:

Comprehensive and Detailed in Depth

Vault creates two default policies upon initialization: root and default. The HashiCorp Vault documentation states: "Vault creates two default policies, root and default. The root policy cannot be deleted or modified. The default policy is attached to all tokens, by default, however, this action can be modified if needed." The root policy grants unrestricted access for administrative tasks, while the default policy provides basic permissions for all tokens unless overridden.

Policies like user, admin, base, and vault are not default; they must be explicitly created by users if needed. Thus, A (root) and D (default) are the correct selections.

Reference:

HashiCorp Vault Documentation - Policies: Built-in Policies

QUESTION 66

Holly has discovered that a highly privileged dynamic credential with a very long lease time was created, which could negatively impact the organization's security. What command can Holly use to invalidate the credential so it can't be used without affecting other credentials?

- A. vault lease revoke aws/creds/admine1b9a1-27b8-83d9-9fe0-d99d786bdc83
- B. Holly would need to delete the credential on the cloud platform directly
- C. vault lease revoke -all
- D. vault lease revoke aws/creds/admin/*

Answer: A

Explanation:

Comprehensive and Detailed in Depth

To invalidate a specific dynamic credential without affecting others, Holly should use the vault lease revoke command with the exact lease ID. The HashiCorp Vault documentation states: "The lease revoke command revokes the lease on a secret, invalidating the underlying secret. To revoke a lease, you can specify the path and lease ID attached to the creds." The command vault lease revoke aws/creds/admine1b9a1-27b8-83d9-9fe0-d99d786bdc83 targets the specific credential by its unique lease ID, ensuring precision without broader impact.

Deleting the credential on the cloud platform (B) doesn't guarantee Vault recognizes it as revoked. vault lease revoke -all (C) revokes all leases, affecting unrelated credentials. vault lease revoke aws/creds/admin/* (D) revokes all leases under that path, potentially impacting other valid credentials. Thus, A is the correct command.

Reference:

HashiCorp Vault Documentation - Lease Revoke Command

QUESTION 67

You need to write a Vault operator policy and give the users access to perform administrative actions in Vault. What path is used for Vault backend functions?

- A. /security
- B. /admin
- C. /vault

- D. /system
- E. /sys
- F. /backend

Answer: E

Explanation:

Comprehensive and Detailed in Depth

The correct path for Vault backend functions, which include administrative actions, is /sys. The HashiCorp Vault documentation confirms: "All backend system functions live in the /sys backend. Policies should take /sys into account when users need to administer Vault configurations." This path hosts endpoints for system-level operations like mounting secrets engines, managing policies, and sealing/unsealing Vault.

Paths like /security, /admin, /vault, /system, and /backend are not standard for Vault's system backend. Only /sys provides the necessary administrative capabilities, making E the correct answer.

Reference:

HashiCorp Vault Documentation - System Backend

QUESTION 68

Your organization wants to set up human-based authentication for AzureAD. What authentication method should you enable and configure for Vault?

- A. OIDC/JWT
- B. Okta
- C. Active Directory
- D. UserPass

Answer: A

Explanation:

Comprehensive and Detailed in Depth

For human-based authentication with Azure Active Directory (AzureAD), the OIDC/JWT authentication method is the best choice. The HashiCorp Vault documentation explains: "The OIDC/JWT auth method is the best choice here. The organization should configure Vault to send authentication requests to AzureAD, which can then validate credentials on behalf of the user." OIDC (OpenID Connect) leverages AzureAD as an identity provider, allowing users to authenticate via their AzureAD credentials in a secure, human-friendly manner.

Okta is a separate identity provider, not directly tied to AzureAD. Active Directory auth is deprecated and less suitable for cloud-based AzureAD integration. UserPass uses a local Vault-managed

username/password, not external AzureAD authentication. Thus, A (OIDC/JWT) is correct.

Reference:

HashiCorp Vault Documentation - JWT/OIDC Auth Method

QUESTION 69

You need to connect to and manage a new HCP Vault cluster using the Vault CLI on your laptop. What environment variables should you set to establish connectivity?

- A. VAULT_CLIENT_KEY=<path-to-key-file>, VAULT_TOKEN=<token-here>
- B. VAULT_NAMESPACE=root, VAULT_REDIRECT_ADDR=<cluster-address>
- C. VAULT_ADDR=https://<cluster-address>:8200, VAULT_NAMESPACE=admin
- D. VAULT_TOKEN=<token-here>, VAULT_CLUSTER_ADDR=https://<cluster-address>:8200

Answer: C

Explanation:

Comprehensive and Detailed in Depth

To connect to an HCP Vault cluster using the Vault CLI, you need to set VAULT_ADDR and VAULT_NAMESPACE. The HashiCorp Vault documentation states: "You can use environment variables to configure the CLI globally. For example, export VAULT_ADDR='http://localhost:8200' sets the address of your Vault server globally." For HCP Vault, the default port is 8200, and the default namespace is "admin," so VAULT_ADDR=https://<cluster-address>:8200 and VAULT_NAMESPACE=admin are required. A token (via VAULT_TOKEN) is also needed for authentication but is typically set after initial connectivity.

VAULT_CLIENT_KEY isn't a standard variable for CLI connectivity. VAULT_REDIRECT_ADDR and VAULT_CLUSTER_ADDR are not used for this purpose. Thus, C provides the correct variables.

Reference:

HashiCorp Vault Documentation - CLI Environment Variables

QUESTION 70

You have a legacy application that requires secrets from Vault that must be written to a local configuration file. However, you cannot refactor the application to communicate directly with Vault. What solution should you implement to satisfy the requirements?

- A. Run the Vault Agent and use the templating feature
- B. Use the Vault Proxy with Auto-Auth to authenticate with Vault
- C. Use the Vault Proxy to act as a proxy for the Vault API
- D. Use the Vault Agent and cache the newly created tokens and leases

Answer: A

Explanation:

Comprehensive and Detailed in Depth

For a legacy application that cannot communicate directly with Vault but needs secrets in a local configuration file, the Vault Agent with templating feature is the best solution. The HashiCorp Vault documentation notes: "Vault Agent can obtain secrets and provide them to applications," and its templating feature "generates dynamic credentials based on predefined templates" and writes them to files. This allows secrets to be automatically fetched and rendered into a configuration file, meeting the requirement without refactoring.

Vault Proxy with Auto-Auth simplifies authentication but doesn't inherently write secrets to files.

Vault Proxy as an API proxy secures API access but doesn't address file writing. Vault Agent with caching improves performance but doesn't solve the file output need. Thus, A is correct.

Reference:

HashiCorp Vault Documentation - Vault Agent and Proxy

QUESTION 71

True or False? The userpass auth method has the ability to access external services in order to provide authentication to Vault.

A. True

B. False

Answer: B

Explanation:

Comprehensive and Detailed in Depth

The statement is False. The HashiCorp Vault documentation clarifies: "The userpass auth method uses a local database that cannot interact with any services outside of the Vault instance." It relies solely on credentials stored within Vault, lacking the ability to integrate with external services for authentication, unlike methods like OIDC or LDAP.

Thus, B (False) is the correct answer.

Reference:

HashiCorp Vault Documentation - Userpass Auth Method

QUESTION 72

Based on the following output, what command can Steve use to determine if the KV store is configured for versioning?

text

CollapseWrapCopy

\$ vault secrets list

Path Type Accessor Description

automation/ kv kv_56f991b9 Automation team for CI/CD
cloud/ kv kv_4426c541 Cloud team for static secrets
cubbyhole/ cubbyhole cubbyhole_9bd538e per-token priv secret storage
data_team/ kv kv_96d57692 Data warehouse KV for certs
identity/ identity identity_0042595e identity store
network/ kv kv_3e53aaab Network team secret storage
secret/ kv kv_d66e2adc key/value secret storage
sys/ system system_d6f218a9 system endpoints

- A. vault secrets list -all
- B. vault kv get automation
- C. vault secrets list -detailed
- D. vault kv list

Answer: C

Explanation:

Comprehensive and Detailed in Depth

To determine if a KV store is configured for versioning (i.e., KV v1 or v2), Steve needs detailed information about the secrets engines. The HashiCorp Vault documentation states: "To list all enabled secrets engines with detailed output, use the command vault secrets list -detailed. This will provide additional information about each secrets engine, including the version of the KV secrets engines." The -detailed flag reveals configuration details, such as the options field indicating version=2 for KV v2, which supports versioning.

vault secrets list -all is not a valid command. vault kv get automation retrieves a specific secret, not engine configuration. vault kv list lists keys in a path, not engine details. Thus, C is correct.

Reference:

HashiCorp Vault Documentation - Secrets Engines (Note: Specific command details are from CLI help and tutorials)

QUESTION 73

True or False? To prepare for day-to-day operations, the root token should be safely saved outside of Vault in order to administer Vault.

- A. True

B. False

Answer: B

Explanation:

Comprehensive and Detailed in Depth

The statement is False. Saving the root token outside of Vault for day-to-day operations is not a recommended practice and contradicts Vault's security principles. The HashiCorp Vault documentation explicitly states: "For day-to-day operations, the root token should be revoked after configuring other auth methods, which admins and Vault clients will use." This is because the root token has unrestricted access to all Vault operations, posing a significant security risk if stored externally and used routinely. Instead, Vault encourages the use of less-privileged tokens or alternative authentication methods post-initialization.

The documentation further elaborates under the "Root Tokens" section: "Root tokens are tokens with an infinite TTL that have the 'root' policy attached to them. Because of their power, it is strongly recommended that they be used only as necessary and then immediately revoked when no longer needed." Storing the root token outside Vault increases the risk of compromise, and Vault's design assumes it is used sparingly—typically only during initial setup—and then replaced with more secure, limited-privilege mechanisms. Thus, the correct operational approach is to revoke the root token after setup, not save it externally, making B (False) the correct answer.

Reference:

HashiCorp Vault Documentation - Tokens: Root Tokens

QUESTION 74

When generating a dynamic secret, what value is returned that a user can use to renew or revoke the lease?

- A. renewable
- B. token_ttl
- C. lease_max
- D. lease_id

Answer: D

Explanation:

Comprehensive and Detailed in Depth

When Vault generates a dynamic secret, it returns a `lease_id`, which is the value a user can use to renew or revoke the lease. The HashiCorp Vault documentation states: "When creating a dynamic secret, Vault always returns a `lease_id`. This `lease_id` can be used to do a vault lease renew or a vault

lease revoke command to manage the lease of a secret." The lease_id uniquely identifies the lease associated with the dynamic secret, enabling precise management of its lifecycle.

The documentation under the "Lease Renew and Revoke" section explains: "Every secret in Vault is associated with a lease. When that lease expires, Vault revokes the secret and removes access to it. Associated with every lease is a unique lease_id. This identifier can be used to renew the lease before it expires or revoke it manually." In contrast, renewable is a boolean indicating if the lease can be renewed, not a value for management. token_ttl relates to token duration, not lease management. lease_max is not a standard term in Vault's lease system. Thus, D (lease_id) is the correct answer.

Reference:

HashiCorp Vault Documentation - Leases: Lease Renew and Revoke

QUESTION 75

You are using Azure Key Vault for the auto-unseal configuration on your cluster. After the Vault service restarts, what command must you run to unseal Vault?

- A. You don't need to run a command when using auto-unseal
- B. vault operator members
- C. vault operator unseal
- D. vault operator init

Answer: A

Explanation:

Comprehensive and Detailed in Depth

When using Azure Key Vault for auto-unseal, no manual command is required to unseal Vault after a service restart. The HashiCorp Vault documentation states: "Vault supports opt-in automatic unsealing via cloud technologies: AliCloud KMS, AWS KMS, Azure Key Vault, Google Cloud KMS, and OCI KMS. This feature enables operators to delegate the unsealing process to trusted cloud providers to ease operations in the event of partial failure and to aid in the creation of new or ephemeral clusters." Specifically, for Azure Key Vault, "the auto-unseal feature automatically handles the unsealing process," eliminating the need for manual intervention.

The documentation further explains: "When configured with auto-unseal, Vault will automatically unseal itself upon startup using the configured key management service, provided the necessary permissions and credentials are in place." Options like vault operator unseal are for manual unsealing, vault operator members lists cluster members, and vault operator init initializes Vault" none apply to auto-unseal scenarios. Thus, A is correct.

Reference:

QUESTION 76

True or False? After initializing Vault or restarting the Vault service, each individual node in the cluster needs to be unsealed.

- A. True
- B. False

Answer: A

Explanation:

Comprehensive and Detailed in Depth

The statement is True. In a Vault cluster, each node must be individually unsealed after initialization or a restart unless auto-unseal is configured. The HashiCorp Vault documentation states: "Since the encryption key is stored in memory, Vault nodes do not share or replicate the encryption key to other nodes. Therefore, each node needs to individually unseal itself upon Vault initialization or anytime the Vault service is restarted on that node." This is due to Vault's design, where the master key (root key) is held in memory and lost on restart, requiring the unseal process to reconstruct it.

The documentation elaborates: "When a Vault server is started, it starts in a sealed state. In this state, Vault is configured to know where and how to access the physical storage, but doesn't know how to decrypt any of it. Unsealing is the process of obtaining the plaintext root key necessary to read the decryption key to decrypt the data." Without auto-unseal, this process is manual for each node, making A (True) correct in the default scenario.

Reference:

HashiCorp Vault Documentation - Seal and Unseal: Unsealing

HashiCorp Vault Documentation - Vault Concepts: Seal

QUESTION 77

What is the default method of authentication after first initializing Vault?

- A. TLS certificates
- B. GitHub
- C. Admin account
- D. Tokens
- E. AppRole
- F. Userpass

Answer: D

Explanation:

Comprehensive and Detailed in Depth

After initializing Vault, the default authentication method is Tokens, specifically the root token. The HashiCorp Vault documentation states: "After initializing, Vault provides the user the root token, which is the only way to log in to Vault in order to configure additional auth methods." This root token is generated during initialization and serves as the initial means of authentication until other methods are configured.

The documentation further explains under the "Token Authentication" section: "Tokens are the core method for authentication within Vault. Upon initialization, a root token is created which can be used to configure Vault further." TLS certificates, GitHub, AppRole, and Userpass require additional setup, and there's no default Admin account method. Thus, D (Tokens) is correct.

Reference:

HashiCorp Vault Documentation - Token Authentication

QUESTION 78

Which auth method is ideal for machine-to-machine authentication?

- A. Okta
- B. UserPass
- C. GitHub
- D. AppRole

Answer: D

Explanation:

Comprehensive and Detailed in Depth

For machine-to-machine authentication, AppRole is the ideal method. The HashiCorp Vault documentation states: "Although it's not the only method for applications, the ideal method for machine-to-machine authentication is AppRole. The other options are frequently reserved for human access." AppRole allows machines or services to authenticate using a role ID and secret ID, providing a secure, automated approach without human intervention.

The documentation elaborates: "The AppRole auth method provides a workflow tailored to machine-to-machine authentication. It allows applications to authenticate with Vault-defined roles and retrieve a token." Okta, UserPass, and GitHub are better suited for human users, not automated systems. Thus, D (AppRole) is correct.

Reference:

HashiCorp Vault Documentation - AppRole Auth Method

QUESTION 79

By default, what happens to child tokens when a parent token is revoked?

- A. The child tokens are revoked
- B. The child tokens are renewed
- C. The child tokens are converted to parent tokens
- D. The child tokens create their own child tokens to be used

Answer: A

Explanation:

Comprehensive and Detailed in Depth

By default, when a parent token is revoked, all child tokens are also revoked. The HashiCorp Vault documentation (via support article) states: "When a parent token is revoked, all of its child tokens" and all of their leases"are revoked as well. This ensures that a user cannot escape revocation by simply generating a never-ending tree of child tokens." This hierarchical revocation ensures security by terminating all derived access when the parent is invalidated.

The documentation on tokens adds: "Tokens in Vault are part of a hierarchy. Child tokens inherit properties from their parents, and revoking a parent token cascades to its children." Options like renewal, conversion to parent tokens, or creating new child tokens do not occur by default. Thus, A is correct.

Reference:

HashiCorp Support - Parent-Child Token Hierarchy

HashiCorp Vault Documentation - Tokens

QUESTION 80

Using the Vault CLI, there are several ways to create a new policy. Select the valid commands (Select three)

A. vault policy write my-policy - << EOF

```
path "secret/data/*" {  
  capabilities = ["create", "update"]  
}
```

EOF

B. vault policy create my-policy /tmp/policy.hcl

C. vault policy write my-policy /tmp/policy.hcl

D. \$ cat user.hcl | vault policy write my-policy -

Answer: A, C, D

Explanation:

Comprehensive and Detailed in Depth

Vault provides multiple valid ways to create a policy via the CLI using the vault policy write command. The HashiCorp Vault documentation states: "To write a policy, use the vault policy write command." The valid methods are:

A: "vault policy write my-policy - << EOF ... EOF uses heredoc syntax to inline policy content, which Vault accepts directly."

C: "vault policy write my-policy /tmp/policy.hcl writes a policy from a file, a standard method per the docs: 'The policy can be read from a file or piped from stdin.'"

D: "cat user.hcl | vault policy write my-policy - pipes policy content from a file via stdin, another documented approach: 'You can pipe the policy content to the command using -.'"

Option B, vault policy create, is invalid as no such command exists"only vault policy write is used.

Thus, A, C, and D are correct.

Reference:

HashiCorp Vault Documentation - Policies: Write a Policy

QUESTION 81

An application is trying to use a dynamic secret in which the lease has expired. What can be done in order for the application to successfully request data from Vault?

- A. Try the expired secret in hopes it hasn't been deleted yet
- B. Perform a lease renewal
- C. Request a new secret and associated lease
- D. Request the TTL be extended for the secret lease

Answer: C

Explanation:

Comprehensive and Detailed in Depth

Once a dynamic secret's lease expires, it cannot be renewed or reused; a new secret must be requested. The HashiCorp Vault documentation states: "A lease must be renewed before it has expired. Once it has expired, it is permanently revoked and a new secret must be requested." This means that after expiration, the secret is invalidated, and the application must obtain a new secret with a new lease to regain access.

Trying an expired secret (A) is futile as it's revoked. Performing a lease renewal (B) is impossible postexpiration, as the docs note: "Renewal must occur before the lease expires." Extending the TTL (D) isn't an option for an expired lease. Thus, C is the correct action.

Reference:

HashiCorp Vault Documentation - Leases: Lease Renew and Revoke

QUESTION 82

You are trying to create a new orphan token but receiving a Permission Denied error. What capabilities are required to create this token without using a root token?

- A. write privileges on the path auth/token
- B. write privileges on the path sys/mounts
- C. sudo privileges on the path auth/token/create
- D. sudo privileges on the path sys/mounts/token

Answer: C

Explanation:

Comprehensive and Detailed in Depth

Creating an orphan token without a root token requires sudo privileges on the path auth/token/create. The HashiCorp Vault documentation states: "The following paths require a root token or sudo capability in the policy: auth/token/create POST Create a periodic or an orphan token (period or no_parent) option." Orphan tokens are not tied to a parent, requiring elevated permissions due to their standalone nature.

The docs further note: "Certain endpoints, such as creating orphan tokens, are root-protected and require either a root token or a policy with sudo capability on the specific path." write on auth/token (A) is insufficient without sudo. write on sys/mounts (B) and sudo on sys/mounts/token (D) are unrelated to token creation. Thus, C is correct.

Reference:

HashiCorp Vault Documentation - Policies: Root-Protected API Endpoints

QUESTION 83

What is the correct order that Vault uses to protect data?

- A. root key --> encryption key --> data
- B. unseal keys --> root key --> data
- C. root key --> data
- D. encryption key --> root key --> data

Answer: A

Explanation:

Comprehensive and Detailed in Depth

Vault protects data using a layered encryption process: root key --> encryption key --> data. The HashiCorp Vault documentation explains: "The data stored by Vault is encrypted. Vault needs the encryption key to decrypt it. The key is also stored with the data (in the keyring), but it is encrypted with another key known as the root key. Therefore, to decrypt the data, Vault must decrypt the encryption key, which requires the root key." This sequence ensures data security through multiple encryption layers.

The docs further clarify: "Unsealing is the process of accessing this root key. The root key is stored alongside all Vault data but is encrypted by yet another mechanism: the unseal key. To recap: most Vault data is encrypted using the encryption key in the keyring; the keyring is encrypted by the root key; and the root key is encrypted by the unseal key." Option B includes unseal keys but omits the encryption key's role. C and D misrepresent the order. Thus, A is correct.

Reference:

HashiCorp Vault Documentation - Seal Concepts

QUESTION 84

Which statement most accurately describes how the response wrapping feature functions in Vault?

- A. Vault takes the response it would have sent to an HTTP client and instead inserts it into the cubbyhole of a single-use token, returning that single-use token instead.
- B. Vault encrypts the response with a dedicated key and sends it directly to the client, never storing it on the server or using single-use tokens for additional security.
- C. Vault divides the response into separate parts and stores each part in different tokens, requiring all tokens to be combined before disclosing the secret to the requesting client.
- D. Vault duplicates the response within a persistent token and allows multiple unwraps, ensuring that any user with the correct token can retrieve the secret repeatedly without time restrictions.

Answer: A

Explanation:

Comprehensive and Detailed in Depth

The response wrapping feature in Vault functions by securing responses in a single-use token's cubbyhole. The HashiCorp Vault documentation states: "To help address this problem, Vault includes a feature called response wrapping. When requested, Vault can take the response it would have sent to an HTTP client and instead insert it into the cubbyhole of a single-use token, returning that single-use token instead." This ensures the response is accessible only once by the intended recipient.

The docs further explain: "Logically speaking, the response is wrapped by the token, and retrieving it requires an unwrap operation against this token. Functionally speaking, the token provides authorization to use an encryption key from Vault's keyring to decrypt the data." Options B, C, and D

misrepresent this process”no dedicated key encryption, no splitting into multiple tokens, and no persistent multi-use tokens occur. Thus, A is correct.

Reference:

HashiCorp Vault Documentation - Response Wrapping

QUESTION 85

Which of the following auth methods is the best choice for human interaction with Vault (as opposed to machine/system authentication)?

- A. Kubernetes
- B. AppRole
- C. TLS
- D. OIDC

Answer: D

Explanation:

Comprehensive and Detailed in Depth

For human interaction with Vault, OIDC (OpenID Connect) is the best choice. The HashiCorp Vault documentation states: "Out of the selections provided, OIDC is the best choice since OIDC authentication uses the user's web browser to complete the authentication request. This is not well suited for machine-to-machine authentication." OIDC leverages identity providers (e.g., AzureAD, Google) for user-friendly authentication via browser-based flows.

The docs add: "The other options of Kubernetes, AppRole, and TLS are more geared towards application/machine/system authentication since they aren't human-friendly." Kubernetes suits cluster workloads, AppRole is for machines, and TLS secures communication, not human logins. Thus, D (OIDC) is correct.

Reference:

HashiCorp Vault Documentation - Authentication Methods

QUESTION 86

True or False? The command `vault lease revoke -prefix aws/` will revoke all leases associated with the secret engine mounted at `/aws`.

- A. True
- B. False

Answer: A

Explanation:

Comprehensive and Detailed in Depth

The statement is True. The vault lease revoke -prefix aws/ command revokes all leases under the specified prefix. The HashiCorp Vault documentation states: "The vault lease revoke command is used to revoke leases. Using the -prefix flag allows you to revoke entire trees of secrets." When applied to aws/, it targets all leases associated with the secrets engine mounted at that path.

The docs further explain under "Prefix-Based Revocation": "The -prefix option allows revocation of all leases that share a common prefix, effectively cleaning up all secrets under a mount point or path."

Thus, A (True) is correct.

Reference:

HashiCorp Vault Documentation - Leases: Prefix-Based Revocation

QUESTION 87

Which of the following features are not available in the Vault Community version?

- A. Cloud KMS auto-unseal
- B. Single sign-on support
- C. Event notifications and filtering
- D. Multi-factor authentication (auth)
- E. Dynamic secrets engines
- F. HSM auto-unseal

Answer: F

Explanation:

Comprehensive and Detailed in Depth

The HSM auto-unseal feature is not available in the Vault Community version; it is exclusive to the Enterprise edition. The HashiCorp Vault documentation states: "Within the Vault family of products, there are two editions offered by HashiCorp - Vault Community Edition and Vault Enterprise.

Enterprise offers other features to enable use cases such as disaster recovery, sync secrets from Vault to other cloud service providers, and advanced event management." Specifically, HSM (Hardware Security Module) auto-unseal is listed as an Enterprise-only feature, requiring additional licensing.

The docs clarify: "Both community and enterprise editions offer similar capabilities to enable secrets management," including Cloud KMS auto-unseal, single sign-on support, event notifications and filtering, multi-factor authentication, and dynamic secrets engines. However, "HSM auto-unseal provides a higher level of security by using Hardware Security Modules (HSMs) to automatically unseal the Vault," exclusive to Enterprise. Thus, F is correct.

Reference:

HashiCorp Vault Documentation - Available Vault Editions

QUESTION 88

When Vault is sealed, which are the only two operations available to a Vault administrator? (Select two)

- A. View the status of Vault
- B. Configure policies
- C. View data stored in the key/value store
- D. Rotate the encryption key
- E. Unseal Vault
- F. Author security policies

Answer: A, E

Explanation:

Comprehensive and Detailed in Depth

When Vault is sealed, its functionality is severely restricted to protect encrypted data. The HashiCorp Vault documentation states: "While Vault is sealed, the only two options available are viewing the vault status (vault status) and unsealing Vault (vault operator unseal). All the other actions require Vault to be unsealed and the user to be authenticated." This limitation ensures that no operations can access or modify data until the Vault is unsealed, enhancing security.

The documentation under "Shamir Seals" further elaborates: "When Vault is sealed, it knows where its encrypted data is stored but cannot decrypt it because the master key is not in memory. The only available operations are checking the seal status and initiating the unseal process." Thus:

A (View the status of Vault): The vault status command works when sealed, providing details like seal state.

E (Unseal Vault): The vault operator unseal command allows administrators to begin unsealing.

Options like configure policies (B), view data in the key/value store (C), rotate the encryption key (D), and author security policies (F) require an unsealed Vault and authentication, making A and E the correct selections.

Reference:

HashiCorp Vault Documentation - Seal Concepts: Shamir Seals

HashiCorp Vault Documentation - Vault Status Command

QUESTION 89

Which two interfaces automatically assume the token for subsequent requests after successfully authenticating? (Select two)

- A. CLI

- B. API
- C. UI

Answer: A, C

Explanation:

Comprehensive and Detailed in Depth

After successful authentication, the CLI and UI interfaces in Vault automatically assume the token for subsequent requests, simplifying user interaction. The HashiCorp Vault documentation states: "After authenticating, the UI and CLI automatically assume the token for all subsequent requests. The API, however, requires the user to extract the token from the server response after authenticating in order to send with subsequent requests." This is facilitated by Vault's token helper mechanism for CLI and session management in the UI.

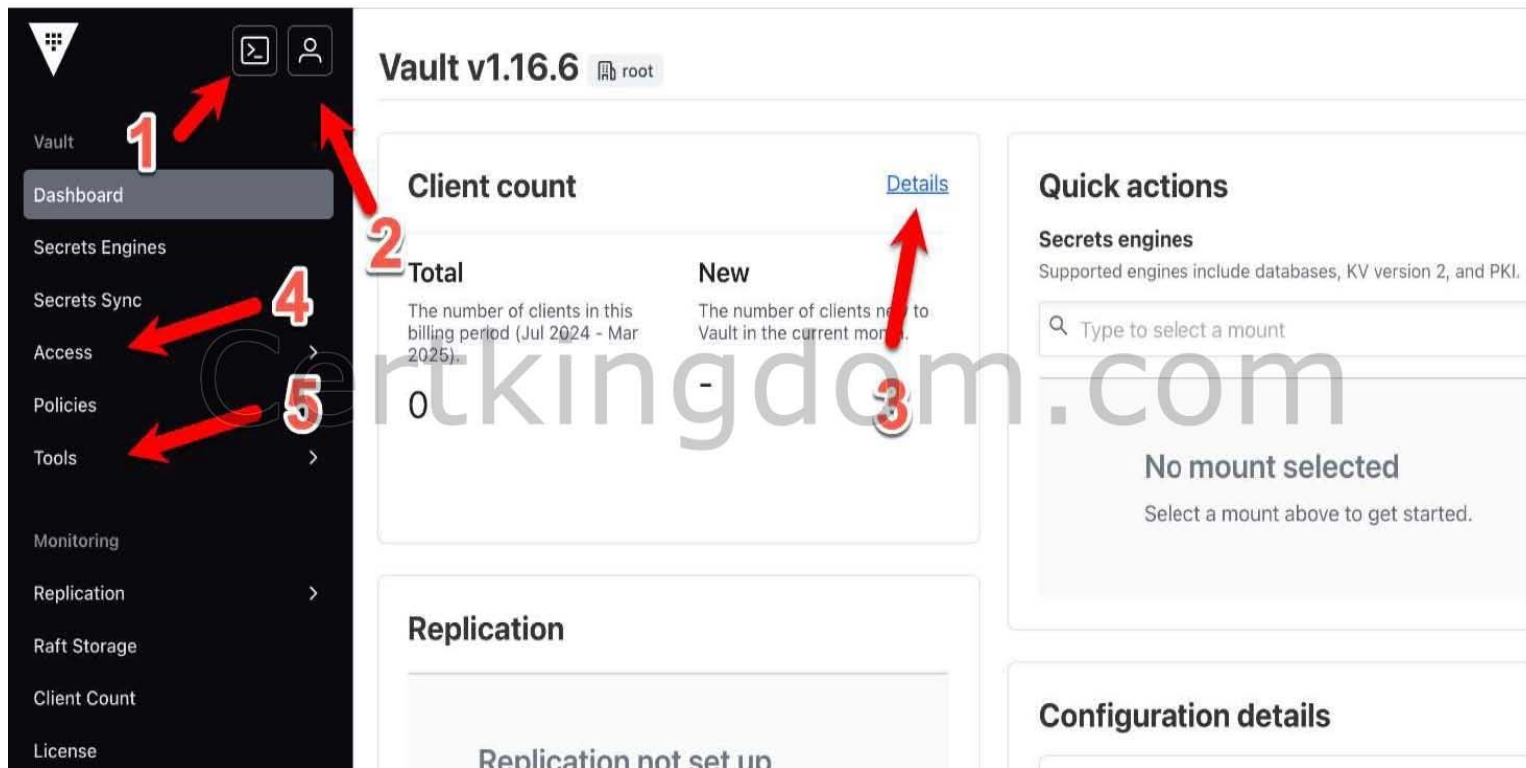
The documentation under "Token Helper" explains: "The Vault CLI uses a token helper to store the token locally after login (e.g., vault login), and future commands automatically use this token without requiring it to be specified each time." Similarly, the UI stores the token in the browser session postlogin. In contrast, the API requires explicit inclusion of the token in each request header (e.g., XVault-Token), making manual token management necessary. Thus, A (CLI) and C (UI) are correct.

Reference:

HashiCorp Vault Documentation - Commands: Token Helper

QUESTION 90

Without logging into another interface, what feature can Chad use to execute a simple CLI command to enable a new secrets engine?



- A. CLI emulation in the Vault UI (Feature 1)
- B. User information button (Feature 2)
- C. Client count details (Feature 3)
- D. Access management link (Feature 4)

Answer: A

Explanation:

Comprehensive and Detailed in Depth

The Vault UI includes a feature allowing CLI commands to be executed directly within the interface, known as the CLI emulation or REPL (Read-Eval-Print Loop) terminal. The HashiCorp Vault documentation states: "The Vault GUI includes an advanced mode that uses a read-eval-print loop (REPL) terminal to mimic basic create/read/update/delete/list (CRUDL) commands for users who are more familiar with the Vault CLI than the GUI." This feature enables Chad to run a command like `vault secrets enable <engine>` without switching to a separate CLI, fulfilling the requirement. The documentation under "Explore the Vault UI" adds: "This terminal allows users to execute Vault CLI commands directly from the web interface, enhancing usability for those accustomed to CLI workflows." Options like user information (B), client count details (C), and access management (D) do

not provide CLI execution capabilities. Thus, A is correct.

Reference:

HashiCorp Vault Documentation - Getting Started UI: Explore the Vault UI

QUESTION 91

Which of the following best describes a token accessor?

- A. A value that describes which clients have access to the attached token
- B. Describes the value associated with the token's TTL
- C. A token used for clients to access Vault secrets engines
- D. A value that acts as a reference to a token which can be used to perform limited actions against the token

Answer: D

Explanation:

Comprehensive and Detailed in Depth

A token accessor is a unique identifier linked to a token, used for management purposes. The HashiCorp Vault documentation states: "A token accessor is created alongside of each token, and the accessor can be used to perform limited actions against the token, including looking up the token's properties, renewing the token, and even revoking the token." It acts as a reference, not the token itself, enabling specific operations without exposing the token's value.

The docs further clarify: "Token accessors provide a way to interact with a token without needing the token itself, enhancing security by limiting direct exposure." Option A misattributes access control, B ties it to TTL (unrelated), and C confuses it with the token. Thus, D accurately describes its role.

Reference:

HashiCorp Vault Documentation - Tokens: Token Accessors

QUESTION 92

Which is not a capability that can be used when writing a Vault policy?

- A. delete
- B. modify
- C. create
- D. list
- E. read
- F. update

Answer: B

Explanation:

Comprehensive and Detailed in Depth

When writing a Vault policy, the valid capabilities are predefined, and modify is not among them. The HashiCorp Vault documentation states: "When writing a policy in Vault, permissions which can be applied to paths include create, read, update, delete, list, deny, and sudo." These capabilities dictate what actions a token can perform on a path.

The docs elaborate: "Capabilities are specific permissions assigned to paths in a policy. For example, create allows creating new resources, update modifies existing ones, delete removes them, list retrieves listings, and read accesses data." Modify is not a recognized capability; it's likely a misnomer for update. Thus, B is the correct answer.

Reference:

HashiCorp Vault Documentation - Policies: Capabilities

QUESTION 93

Which statement best explains the role and usage of storage backends in HashiCorp Vault?

- A. They store Vault's persistent data, affecting the scalability and performance of managing Vault.
- B. They handle the encryption of all secrets so that Vault remains completely stateless.
- C. They store only ephemeral tokens, ensuring no persistent data is ever saved.
- D. They store only unseal keys, while all secret data remains in Vault's memory.

Answer: A

Explanation:

Comprehensive and Detailed in Depth

Storage backends in Vault are responsible for storing persistent data, impacting its operation. The HashiCorp Vault documentation states: "The storage stanza configures the storage backend, which represents the location for the durable storage of Vault's information. Each backend has pros, cons, advantages, and trade-offs. For example, some backends support high availability while others provide a more robust backup and restoration process." This includes secrets, policies, and audit logs, affecting scalability and performance.

The docs add: "Vault uses a storage backend to persist its encrypted data, configuration, and metadata." Option B is incorrect as encryption is handled by Vault, not the backend. C and D are wrong"backends store all persistent data, not just tokens or unseal keys. Thus, A is correct.

Reference:

HashiCorp Vault Documentation - Storage Backends

QUESTION 94

Which of the following are benefits of using the Vault Secrets Operator (VSO)? (Select three)

- A. Support for syncing from multiple secret sources
- B. Bi-directional sync between Vault and Kubernetes Secrets
- C. Automatic secret drift and remediation
- D. Automatic secret rotation for multiple Kubernetes resource types

Answer: A, C, D

Explanation:

Comprehensive and Detailed in Depth

The Vault Secrets Operator (VSO) enhances secrets management in Kubernetes. The HashiCorp Vault documentation lists its benefits: "The following features are supported by the Vault Secrets Operator:

Support for syncing from multiple secret sources.

Automatic secret drift and remediation.

Automatic secret rotation for Deployment, ReplicaSet, StatefulSet Kubernetes resource types."

The docs explain: "VSO watches for changes to its supported Custom Resource Definitions (CRDs) and synchronizes secrets from Vault to Kubernetes Secrets, ensuring consistency (A). It detects and corrects unauthorized changes (C) and rotates secrets for specified resource types (D)." Bi-directional sync (B) is not supported" sync is one-way from Vault to Kubernetes. Thus, A, C, and D are correct.

Reference:

HashiCorp Vault Documentation - Vault Secrets Operator

QUESTION 95

True or False? Once the minimum decryption version is set on an encryption key, older versions of the key are removed from Vault and are no longer available for decryption operations.

- A. True
- B. False

Answer: B

Explanation:

Comprehensive and Detailed in Depth

The statement is False. Setting the minimum decryption version does not remove older key versions.

The HashiCorp Vault documentation states: "Key versions that are earlier than a key's specified min_decryption_version get archived, and the rest of the key versions belong to the working set. In an emergency, the min_decryption_version can be moved back to allow for legitimate decryption."

Older versions remain available for decryption if needed.

The docs add: "Archiving a key version does not delete it; it simply marks it as outside the active working set, but Vault retains it for potential use." Thus, older versions are not removed, making B correct.

Reference:

HashiCorp Vault Documentation - Transit Secrets Engine: Working Set Management

QUESTION 96

A new Vault administrator is writing a CURL command (shown below) to retrieve a secret stored in a KV v2 secrets engine at secret/audio/soundbooth but is receiving an error. What could be the cause of the error?

```
$ curl \  
--header "X-Vault-Token: hvs.rffHw0iXqkRo19b2cjf93DM39WjpbN3J" \  
https://vault.unlimited.com:8200/v1/secret/audio/soundbooth
```

A. The VAULT_ADDR environment variable wasn't set, so it should be configured: export VAULT_ADDR="https://vault.unlimited.com:8200"

B. The request is being made on the incorrect endpoint and should be:

```
$ curl \  
--header "X-Vault-Token: hvs.rffHw0iXqkRo19b2cjf93DM39WjpbN3J" \  
https://vault.unlimited.com:8200/v1/secret/data/audio/soundbooth
```

C. The user's token doesn't permit access to the Vault API, only the UI

D. The endpoint should point to v2 since this is a KV v2 secrets engine:

```
$ curl \  
--header "X-Vault-Token: hvs.rffHw0iXqkRo19b2cjf93DM39WjpbN3J" \  
https://vault.unlimited.com:8200/v2/secret/audio/soundbooth
```

Answer: B

Explanation:

Comprehensive and Detailed in Depth

The error occurs because the CURL command uses the wrong endpoint for a KV v2 secrets engine.

The HashiCorp Vault documentation states: "The KVV2 store uses a prefixed API, which is different from the version 1 API. Writing and reading versions are prefixed with the data/ path." For KV v2, the correct endpoint to retrieve a secret is /v1/secret/data/audio/soundbooth, not /v1/secret/audio/soundbooth, which applies to KV v1.

The docs explain: "In KV v2, the data/ prefix is required when accessing secrets via the API to distinguish data operations from metadata or versioning tasks." Option A (VAULT_ADDR) is irrelevant for API calls, as it's CLI-specific. Option C (token UI restriction) is incorrect—tokens apply universally.

Option D misinterprets v1 as the API version, not the engine version. Thus, B is correct.

Reference:

HashiCorp Vault Documentation - KV v2: ACL Rules

QUESTION 97

Which of the following is not an action associated with the Transit secrets engine when interacting with data?

- A. encrypt
- B. decrypt
- C. rewrap
- D. update

Answer: D

Explanation:

Comprehensive and Detailed in Depth

The Transit secrets engine focuses on cryptographic operations, not data storage or modification. The HashiCorp Vault documentation states: "The transit secrets engine handles cryptographic functions on data in-transit. Vault doesn't store the data sent to the secrets engine. It can also be viewed as 'cryptography as a service' or 'encryption as a service'. The transit secrets engine can also sign and verify data; generate hashes and HMACs of data; and act as a source of random bytes."

It further notes: "You can, however, rewrap data when the key has been rotated to ensure data is encrypted with the latest version." Supported actions include encrypt, decrypt, and rewrap, but update is not a function, as Transit doesn't store or modify data. Thus, D is correct.

Reference:

HashiCorp Vault Documentation - Transit Secrets Engine

QUESTION 98

After a client has authenticated to Vault, what security feature is used to make all subsequent calls?

- A. ldap
- B. pgp
- C. path
- D. key shard
- E. listener
- F. token

Answer: F

Explanation:

Comprehensive and Detailed in Depth

After authentication, Vault uses tokens for all subsequent calls. The HashiCorp Vault documentation states: "After authenticating, a client is issued a service token which is associated with a policy. That token is used to make all subsequent requests to Vault." Tokens serve as the primary security feature for authorizing and authenticating requests.

The docs elaborate: "Tokens are the core method for authentication within Vault. Once authenticated, the client uses this token to access secrets and perform operations according to the attached policies." Other options like ldap, pgp, path, key shard, and listener are unrelated to this role. Thus, F is correct.

Reference:

HashiCorp Vault Documentation - Authentication Concepts

QUESTION 99

Which two characters can be used when writing a policy to reflect a wildcard or path segment?
(Select two)

- A. The ampersand &
- B. The at symbol @
- C. The splat character *
- D. A dollar sign \$
- E. The pound symbol #
- F. The plus symbol +

Answer: C, F

Explanation:

Comprehensive and Detailed in Depth

Vault policies use specific characters for wildcards and path segments. The HashiCorp Vault documentation states: "The plus sign (+) can be used to denote a path segment and can be used in the middle of a path. The splat (*) can be used as a wildcard but can only be used at the very end of a path." These are the only characters designated for such purposes in policy syntax.

The docs add: "For example, secret/data/* matches all paths under secret/data/, while secret/+/foo matches a single segment like secret/bar/foo." &, @, \$, and # have no special meaning in Vault policies. Thus, C (*) and F (+) are correct.

Reference:

HashiCorp Vault Documentation - Policies: Policy Syntax

QUESTION 100

Beyond encryption and decryption of data, which of the following is not a function of the Transit secrets engine?

- A. Generate hashes and HMACs of data
- B. Sign and verify data
- C. Store the encrypted data securely in Vault for retrieval
- D. Act as a source of random bytes

Answer: C

Explanation:

Comprehensive and Detailed in Depth

The Transit secrets engine focuses on cryptographic operations, not storage. The HashiCorp Vault documentation states: "The transit secrets engine handles cryptographic functions on data in-transit. Vault doesn't store the data sent to the secrets engine. It can also be viewed as ~cryptography as a service' or ~encryption as a service'. The transit secrets engine can also sign and verify data; generate hashes and HMACs of data; and act as a source of random bytes."

It emphasizes: "Vault does not store the data sent to the secrets engine," making store the encrypted data (C) incorrect. Generate hashes/HMACs (A), sign/verify (B), and random bytes (D) are all supported functions. Thus, C is correct.

Reference:

HashiCorp Vault Documentation - Transit Secrets Engine

QUESTION 101

Compared to service tokens, batch tokens are ideal for what type of action?

- A. Generating dynamic credentials
- B. Renewing other tokens
- C. For daily batch jobs requesting secrets from Vault
- D. Short-lived, high-volume, or ephemeral tasks

Answer: D

Explanation:

Comprehensive and Detailed in Depth

Batch tokens are designed for specific, transient use cases. The HashiCorp Vault documentation states: "Batch tokens are lightweight and scalable and include just enough information to be used with Vault. They are generally used for ephemeral, high-performance workloads, such as encrypting

data." This makes them ideal for short-lived, high-volume, or ~ephemeral' tasks (D).

The docs contrast: "Unlike service tokens, which are renewable and suited for long-lived processes, batch tokens have a fixed TTL and cannot be renewed." Options like generating dynamic credentials (A) and daily batch jobs (C) align more with service tokens, while renewing tokens (B) isn't a batch token function. Thus, D is correct.

Reference:

HashiCorp Vault Documentation - Batch Tokens

QUESTION 102

True or False? All Vault policies are deny by default.

- A. True
- B. False

Answer: A

Explanation:

Comprehensive and Detailed in Depth

The statement is True. Vault operates on a default-deny model for policies. The HashiCorp Vault documentation states: "Vault policies implicitly deny all actions that are not explicitly permitted in the Vault policy." This ensures that access must be explicitly granted, enhancing security.

The docs elaborate: "By default, a token has no policies attached beyond the default policy (which grants minimal permissions), and any action not explicitly allowed by an attached policy is denied."

This principle underpins Vault's access control, making A correct.

Reference:

HashiCorp Vault Documentation - Policies Tutorial

QUESTION 103

Which of the following statements are true about HCP Vault Dedicated? (Select three)

- A. Provides 100% feature parity compared to Vault self-managed clusters
- B. Helps reduce operational overhead for organizations with push-button deployment and fully managed upgrades
- C. Increases reliability and ease of use so you can onboard applications and teams easily
- D. Increases security across clouds and machines through a single interface

Answer: B, C, D

Explanation:

Comprehensive and Detailed in Depth

HCP Vault Dedicated is a managed cloud service offering specific benefits over self-managed Vault. The HashiCorp Vault documentation outlines its advantages: "Vault Enterprise running on the HashiCorp Cloud Platform (HCP) enables users to secure, store, and tightly control access to tokens, passwords, certificates, and encryption keys within one unified cloud-based platform." It lists the following benefits relevant to the options:

B (Helps reduce operational overhead for organizations with push-button deployment and fully managed upgrades): The documentation states, "Reduce operational overhead: Push-button deployment, fully managed upgrades, and backups mean organizations can focus on adoption and integration instead of operational overhead." This reflects HCP Vault Dedicated's managed nature, automating deployment and maintenance tasks.

C (Increases reliability and ease of use so you can onboard applications and teams easily): It notes, "Ease of use: HCP Vault Dedicated is built around making cloud security automation simple. Get up and running quickly so that you can onboard applications and teams easily," and "Reliability: HashiCorp has experience supporting thousands of commercial Vault Enterprise clusters and HCP Vault Dedicated brings that expertise directly to users." This simplifies onboarding and ensures dependable operation.

D (Increases security across clouds and machines through a single interface): The docs confirm, "Increase security across clouds and machines: Secure your infrastructure across all your environments through a single interface and globally control and restrict access to sensitive data and systems," highlighting centralized security management.

However, A (Provides 100% feature parity compared to Vault self-managed clusters) is false. The documentation clarifies under "Feature Parity": "HCP Vault Dedicated does not provide 100% feature parity compared to Vault self-managed clusters. While it offers many of the same features and capabilities, there may be some differences or limitations in functionality between the two deployment options." Thus, B, C, and D are true.

Reference:

HashiCorp Vault Documentation - What is HCP Vault: Feature Parity

QUESTION 104

What is the result of the following Vault command?

```
$ vault auth enable kubernetes
```

- A. Allows Vault to access usernames and passwords stored in a Kubernetes cluster
- B. Mounts the Kubernetes auth method to the default path of kubernetes/
- C. Imports Kubernetes secrets to the local KV database
- D. Enables Vault to host an IdP for Kubernetes workloads

Answer: B

Explanation:

Comprehensive and Detailed in Depth

The command `vault auth enable kubernetes` enables the Kubernetes authentication method in Vault. The HashiCorp Vault documentation states: "In order to enable auth methods, the command should be `vault auth <enable/disable>` followed by the name of the auth method." Specifically, for Kubernetes, it explains: "The `vault auth enable kubernetes` command mounts the Kubernetes auth method to the default path of `kubernetes/`." This allows Vault to authenticate Kubernetes workloads using their service account tokens at the path `auth/kubernetes/`.

The documentation elaborates: "Once enabled, the Kubernetes auth method allows clients running in Kubernetes to authenticate with Vault using a Kubernetes Service Account Token. The default mount path is `kubernetes/`, though additional parameters can specify a different path." Option A is incorrect"Vault doesn't access usernames/passwords in Kubernetes; it uses tokens. Option C is wrong"it doesn't import secrets, only enables authentication. Option D is false"Vault doesn't become an Identity Provider (IdP); it authenticates against Kubernetes. Thus, B is correct.

Reference:

HashiCorp Vault Documentation - Secrets Enable Command

HashiCorp Vault Documentation - Kubernetes Auth Method

QUESTION 105

Which statement best explains how Vault handles data encryption?

- A. Vault uses encryption to secure data at rest and in transit, using an encryption key protected by the root key.
- B. Vault encrypts data using a root key stored in plain text on the server's filesystem.
- C. Vault stores data in plaintext on disk but encrypts it only when transmitting it over the network.
- D. Vault offloads all encryption to third-party services, so no secret data is ever processed by Vault.

Answer: A

Explanation:

Comprehensive and Detailed in Depth

Vault's encryption mechanism is a core security feature. The HashiCorp Vault documentation states: "When a Vault server is started, it starts in a sealed state. In this state, Vault is configured to know where and how to access the physical storage, but doesn't know how to decrypt any of it. Unsealing is the process of obtaining the plaintext root key necessary to read the decryption key to decrypt the data, allowing access to the Vault." It further explains: "Vault uses encryption to secure data at rest and in transit, using an encryption key protected by the root key."

The documentation details: "The data stored by Vault is encrypted using an encryption key in the

keyring. This keyring is itself encrypted by the root key, which is protected by the unseal process (e.g., Shamir's Secret Sharing or auto-unseal). Vault ensures data is encrypted both at rest in the storage backend and in transit over the network using TLS." Option B is false"the root key is never stored in plaintext. Option C is incorrect"data is encrypted at rest, not just in transit. Option D is wrong"Vault performs encryption internally, not via third-party services. Thus, A is correct.

Reference:

HashiCorp Vault Documentation - Seal Concepts

QUESTION 106

Which of the following unseal options can automatically unseal Vault upon the start of the Vault service? (Select four)

- A. HSM
- B. Azure KMS
- C. AWS KMS
- D. Transit
- E. Key Shards

Answer: A, B, C, D

Explanation:

Comprehensive and Detailed in Depth

Vault supports auto-unseal to simplify operations. The HashiCorp Vault documentation states: "Vault supports opt-in automatic unsealing via cloud technologies: AliCloud KMS, AWS KMS, Azure Key Vault, Google Cloud KMS, and OCI KMS," and includes HSM and Transit as additional options. It explains: "Auto unseal is used to automatically unseal Vault using an HSM or cloud HSM service." The valid options are:

A (HSM): "HSM (Hardware Security Module) can automatically unseal Vault by securely storing and managing the master key used for encryption and decryption operations."

B (Azure KMS): "Azure KMS can automatically unseal Vault by utilizing Azure Key Management Service to manage the master key."

C (AWS KMS): "AWS KMS can automatically unseal Vault upon the start of the service by using AWS Key Management Service to manage the master key."

D (Transit): "Transit can automatically unseal Vault by using a pre-configured encryption key stored in Vault itself to encrypt the unseal key."

The documentation clarifies: "Key Shards require the user to provide unseal keys to reconstruct the master key," making E (Key Shards) a manual process, not auto-unseal. Thus, A, B, C, and D are correct.

Reference:

QUESTION 107

What type of Vault token does not have a TTL (Time to Live)?

- A. Child tokens
- B. Parent tokens
- C. Service tokens
- D. Root tokens
- E. Batch tokens

Answer: D

Explanation:

Comprehensive and Detailed in Depth

Root tokens in Vault are unique in lacking a TTL. The HashiCorp Vault documentation states: "Non-root tokens are associated with a TTL, which determines for how long a token is valid. Root tokens are not associated with a TTL, and therefore, do not expire." It provides an example: "For example, notice that the value for token_duration is the infinity symbol, meaning it lives forever," as seen in a vault login output for a root token.

The docs elaborate: "Root tokens are tokens with an infinite TTL that have the '~root' policy attached to them. Because of their power, it is strongly recommended that they be used only as necessary and then immediately revoked when no longer needed." In contrast:

Child tokens (A) inherit TTLs from parents.

Parent tokens (B) typically have TTLs unless they are root.

Service tokens (C) have configurable TTLs for ongoing use.

Batch tokens (E) have fixed TTLs for ephemeral tasks.

Thus, D (Root tokens) is correct.

Reference:

HashiCorp Vault Documentation - Tokens: Token Time to Live

QUESTION 108

Before the following command can be run to encrypt data, what (three) commands must be run to enable and configure the transit secrets engine in Vault? (Select three)

text

CollapseWrapCopy

```
$ vault write transit/encrypt/vendor \
```

plaintext="aGFzaGljb3JwIGNlcnRpZmlZA=="

- A. base64 <<< "hashicorp certified"
- B. vault write transit/encrypt/vendor
- C. vault secrets list
- D. vault secrets enable transit
- E. vault write -f transit/keys/vendor

Answer: A, D, E

Explanation:

Comprehensive and Detailed in Depth

To encrypt data using the Transit secrets engine, it must be enabled and configured. The HashiCorp Vault documentation states: "Enable the Transit secrets engine at the default path of ~transit' using the command vault secrets enable transit. Create an encryption key called ~vendor' using the command vault write -f transit/keys/vendor. Encode the string using base-64 encoding by using the command base64 <<< ~hashicorp certified'." These steps are prerequisites for the given vault write transit/encrypt/vendor command:

A (base64 <<< "hashicorp certified"): The docs note, "All plaintext data must be base64-encoded. The reason for this requirement is that Vault does not require that the plaintext is ~text'. It could be a binary file such as a PDF or image. The easiest safe transport mechanism for this data as part of a JSON payload is to base64-encode it." The provided plaintext aGFzaGljb3JwIGNlcnRpZmlZA== is the base64 encoding of "hashicorp certified."

D (vault secrets enable transit): "Before you can use the transit secrets engine, it must be enabled with vault secrets enable transit at the default path ~transit/'."

E (vault write -f transit/keys/vendor): "An encryption key must be created before encryption can occur. Use vault write -f transit/keys/vendor to generate a key named ~vendor'."

B is the target command, not a prerequisite. C (vault secrets list) lists engines but doesn't configure Transit. Thus, A, D, and E are correct.

Reference:

HashiCorp Vault Documentation - Transit Secrets Engine

QUESTION 109

Mike's Cereal Shack uses Vault to encrypt customer data to ensure it is always stored securely. They are developing a new application integration to send new customer data to be encrypted using the following API request:

text

CollapseWrapCopy

\$ curl \

```
--header "X-Vault-Token: hv.ssf4vj1rFV5PvQSV3M9dcv832brxQFsfbXA" \  
--request POST \  
--data @data.json \  
https://vault.meshtack.com:8200/v1/transit/encrypt/customer-data  
What would be contained within the data.json file?
```

- A. Transit secrets engine configuration file
- B. Ciphertext to be decrypted
- C. The encryption key to be used for encrypting the data
- D. Cleartext customer data to be encrypted

Answer: D

Explanation:

Comprehensive and Detailed in Depth

The data.json file in this API request contains the data to be encrypted by the Transit secrets engine. The HashiCorp Vault documentation states: "When executing any call to the Vault API, data can be sent using an external file as shown above. In this case, the contents of the file would be cleartext customer data that needs to be encrypted by the transit secrets engine." Specifically, for the /transit/encrypt/ endpoint, it explains: "The API expects a JSON payload with a plaintext field containing the base64-encoded data to encrypt."

The documentation elaborates under "Encrypt Data": "The request body must include the plaintext parameter, which is the base64-encoded version of the data you want to encrypt. For example: {"plaintext": "base64-encoded-data"}." Here, D (Cleartext customer data to be encrypted) fits this requirement"customer data in cleartext, base64-encoded, sent for encryption. A (Transit config) is managed in Vault, not sent. B (Ciphertext) is the output, not input. C (Encryption key) is stored in Vault, not provided by the client. Thus, D is correct.

Reference:

HashiCorp Vault Documentation - Transit API: Encrypt Data

QUESTION 110

Which of the following best describes the function of the Vault Secrets Operator in a Kubernetes environment?

- A. It replaces the Kubernetes secrets API entirely and operates purely as a certificate authority for all workloads.
- B. It is a standalone Vault server that automatically applies security policies and rotates root tokens.
- C. It continuously reconciles and synchronizes secrets from Vault to Kubernetes, ensuring secrets are always updated

D. It provides an interface to dynamically provision Kubernetes clusters through Vault's infrastructure secrets.

Answer: C

Explanation:

Comprehensive and Detailed in Depth

The Vault Secrets Operator (VSO) enhances secrets management in Kubernetes. The HashiCorp Vault documentation states: "The Vault Secrets Operator operates by watching for changes to its supported set of Custom Resource Definitions (CRD). Each CRD provides the specification required to allow the operator to synchronize from one of the supported sources for secrets to a Kubernetes Secret. The operator writes the source secret data directly to the destination Kubernetes Secret, ensuring that any changes made to the source are replicated to the destination over its lifetime."

It further explains: "In this way, an application only needs to have access to the destination secret in order to make use of the secret data contained within." This aligns with C: "It continuously reconciles and synchronizes secrets from Vault to Kubernetes, ensuring secrets are always updated." Option A is false—it augments, not replaces, the Kubernetes Secrets API and isn't a C

A. Option B is incorrect—it

it's not a Vault server but an operator. Option D is wrong—it syncs secrets, not provisions clusters.

Thus, C is correct.

Reference:

HashiCorp Vault Documentation - Vault Secrets Operator

QUESTION 111

After creating a dynamic credential on a database, the DBA accidentally deletes the credentials on the database itself. When attempting to remove the lease, Vault returns an error stating that the credential cannot be found. What command can be run to make Vault remove the secret?

A. vault lease revoke -force -prefix <lease_path>

B. vault lease -renew

C. vault lease revoke -enforce

D. vault revoke -apply

Answer: A

Explanation:

Comprehensive and Detailed in Depth

When a dynamic credential is deleted externally, Vault may fail to revoke the lease due to the missing backend secret. The HashiCorp Vault documentation states: "The -force flag is meant for

recovery situations where the secret in the target platform was manually removed." The command `vault lease revoke -force -prefix <lease_path>` allows Vault to forcibly revoke all leases under the specified prefix, bypassing the error.

The docs elaborate: "Using `-force` with `-prefix` will revoke all leases that match the given prefix, even if the underlying secrets cannot be found or revoked on the target system. This is useful for cleaning up Vault's lease table when external changes disrupt normal revocation." Here, `<lease_path>` would be the path like `database/creds/role/`. B (`vault lease -renew`) renews leases, not removes them. C (`-enforce`) is not a valid flag. D (`vault revoke -apply`) is incorrect syntax. Thus, A is correct.

Reference:

HashiCorp Vault Documentation - Lease Revoke Command: Force

QUESTION 112

What command is used to extend the TTL of a token, if permitted?

- A. `vault token revoke <token-id>`
- B. `vault capabilities <token-id>`
- C. `vault token lookup <token-id>`
- D. `vault token renew <token-id>`

Answer: D

Explanation:

Comprehensive and Detailed in Depth

To extend a token's TTL, the `vault token renew` command is used. The HashiCorp Vault documentation states: "In order to renew a token, a user can issue a `vault token renew` command to extend the TTL. The token can also be renewed using the API." It adds: "The `vault token renew` command extends the Time To Live (TTL) of a token if the policy associated with the token permits renewal."

The docs detail: "Tokens have a TTL that determines their validity period. If renewable, the `renew` command can be used before expiration to extend this duration, subject to any max TTL limits." A (`revoke`) invalidates tokens. B (`capabilities`) shows permissions, not TTL. C (`lookup`) displays token info, not extends it. Thus, D is correct.

Reference:

HashiCorp Vault Documentation - Token Renew Command

QUESTION 113

An application has authenticated to Vault and has obtained dynamic database credentials with a lease of 4 hours. Four hours later, the credentials expire, and the application can no longer communicate with the backend database, so the application goes down. What should the developers

instruct the application to do to prevent this from happening again while maintaining the same level of security?

- A. Go back to using static credentials
- B. Renew the lease before expiration
- C. Revoke the lease before expiration
- D. Use a different auth method

Answer: B

Explanation:

Comprehensive and Detailed in Depth

To prevent application downtime due to expired dynamic credentials while maintaining security, the application should renew the lease before it expires. The HashiCorp Vault documentation states:

"The application should frequently ~check-in' with Vault and renew the lease to prevent the lease from expiring." It adds: "A lease must be renewed before it has expired. Once it has expired, it is permanently revoked and a new secret must be requested."

The docs elaborate: "Dynamic secrets are designed to be short-lived and automatically rotated or revoked when their lease expires. Renewing the lease extends its validity, ensuring continuous access without compromising the security benefits of short-lived credentials." A (Static credentials) reduces security by eliminating rotation. C (Revoke) ends access early. D (Different auth method) doesn't address lease management. Thus, B is correct.

Reference:

HashiCorp Vault Documentation - Leases: Lease Renew and Revoke

QUESTION 114

Christy has created a token and needs to use that token to access Vault. What command can she use to authenticate and access secrets stored in Vault?

```
$ vault token create -policy=christy
```

Key Value

--- -----

```
token hvs.hxDIPd8RPVtxu4AzSGS11ArP
```

```
token_accessor AxwxpDs6LbdfQbWGmBDnwIK3
```

```
token_duration 24h
```

```
token_renewable true
```

```
token_policies ["christy" "default"]
```

```
identity_policies []
```

```
policies ["christy" "default"]
```

- A. vault login hvs.hxDIPd8RPVtxu4AzSGS11ArP
- B. vault login -method=password
- C. vault login -method=token christy
- D. vault login -accessor=AxwxdDs6LbdfQbWGmBDnwIK3

Answer: A

Explanation:

Comprehensive and Detailed in Depth

To authenticate with a specific token, Christy should use the vault login command with the token value. The HashiCorp Vault documentation states: "To login with a token, you can use vault login <token> or even vault login -method=token <token> if you like typing more." For the given token hvs.hxDIPd8RPVtxu4AzSGS11ArP, the command vault login hvs.hxDIPd8RPVtxu4AzSGS11ArP authenticates Christy and stores the token for subsequent CLI use.

The docs provide an example: ```

```
$ vault login s.sf4vj1rFV5PvQSbrxQFsfbXA
```

Success! You are now authenticated. The token information displayed below is already stored in the token helper. You do NOT need to run ~vault login' again. Future Vault requests will automatically use this token.

Key Value

token s.sf4vj1rFV5PvQSbrxQFsfbXA

New Topic: Topic 3, Exam Pool C

QUESTION 115

Vault operators can create two types of groups in Vault. What are the two types?

- A. External groups
- B. Security groups
- C. Policy groups
- D. Internal groups

Answer: AD

Explanation:

Comprehensive and Detailed In-Depth

In HashiCorp Vault, operators can create two distinct types of groups within the Identity secrets engine: external groups and internal groups. These groups are used to manage and organize users and policies, facilitating access control and permissions management.

External Groups: These groups are designed to integrate with external identity providers or systems, such as LDAP or OIDC (OpenID Connect). External groups allow Vault to map groups from these external systems to Vault policies, enabling seamless access control for users authenticated via external auth methods. They can be created manually or automatically mapped (e.g., from LDAP group memberships to Vault policies). This is particularly useful when managing users who exist outside of Vault's internal identity store but need access to Vault resources. The documentation states: "External groups are usually associated with an auth method, such as LDAP or OIDC."

Internal Groups: These are created and managed directly within Vault's identity store. Internal groups are used to organize Vault entities (representing users or machines) and assign policies to them manually. They are ideal for scenarios where user management is entirely within Vault's ecosystem, without reliance on external identity providers. The documentation explains: "Internal groups are created in the identity store and map to other groups or entities."

Incorrect Options:

Security Groups: This term is not used in Vault's context for group types. While security is a core concern, "security groups" do not represent a specific category of groups in Vault.

Policy Groups: Policies in Vault define permissions, but there is no concept of "policy groups" as a distinct group type. Policies are attached to groups, not grouped themselves in this manner.

The distinction between external and internal groups enhances flexibility in managing authentication and authorization, aligning with Vault's design to support both internal and federated identity systems.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/identity#external-vs-internal-groups>

QUESTION 116

True or False? A token can be renewed up until the max TTL, even if the TTL has been reached.

A. True

B. False

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

This statement is false. In HashiCorp Vault, a token's ability to be renewed is governed by its TTL (Time To Live) and max TTL (Maximum Time To Live). The TTL represents the current validity period of the token, while the max TTL is the absolute upper limit beyond which the token cannot be extended.

Token Renewal Mechanics: A token can be renewed only if it has not yet expired (i.e., its TTL has not reached zero). Renewal extends the TTL, but this extension cannot exceed the max TTL configured for the token. The documentation clarifies: "A token can be renewed up until the max TTL as long as the

token has not expired. If the token expires (hitting the TTL), the token is revoked and is no longer valid." Once the TTL reaches zero, Vault automatically revokes the token, rendering it unusable and ineligible for renewal.

Why False?: The phrase "even if the TTL has been reached" implies that renewal is possible after expiration, which contradicts Vault's behavior. After the TTL expires, there is no active token to renew because it has been revoked. Renewal must occur within the active TTL window, and the total lifetime (including renewals) cannot exceed the max TTL.

Practical Implication: This ensures that tokens have a finite lifecycle, enhancing security by preventing indefinite use of compromised credentials. For example, a token with a TTL of 1 hour and a max TTL of 24 hours can be renewed multiple times within that 24-hour period, but only if renewed before the 1-hour TTL expires each time.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/tokens#token-time-to-liveperiodic-tokens-and-explicit-max-ttls>

QUESTION 117

A DevOps engineer has set up LDAP and GitHub auth methods. The engineer must ensure user Sarah, who authenticates via either method, has consistent access permissions. Which approach correctly describes how to achieve this in Vault?

- A. Create an entity for Sarah and map both her LDAP and GitHub identities as entity aliases to this single entity
- B. Create an external group and add the LDAP and GitHub providers as members of the group
- C. Create separate policies for each auth method and manually ensure they remain synchronized
- D. Configure a trust relationship between the LDAP and GitHub providers to ensure Sarah's account is synced

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

To ensure consistent access permissions for Sarah across multiple authentication methods (LDAP and GitHub), the correct approach in Vault is to create an entity for Sarah and map both her LDAP and GitHub identities as entity aliases to this single entity.

Entities and Aliases in Vault: Vault's Identity secrets engine allows the creation of entities, which are logical representations of users or machines. Each entity can have multiple aliases, where an alias corresponds to an identity from a specific auth method. By mapping Sarah's LDAP identity (e.g., her LDAP username) and GitHub identity (e.g., her GitHub username) as aliases to a single entity, Vault associates both identities with one set of policies. The documentation states: "Vault clients can be mapped as entities and their corresponding accounts with authentication providers can be mapped

as aliases."

Why This Works: Assigning policies to the entity ensures that Sarah's permissions remain consistent regardless of whether she logs in via LDAP or GitHub. This centralizes policy management and eliminates discrepancies.

Incorrect Options:

B. External Group Approach: Creating an external group and adding LDAP and GitHub providers as members does not inherently synchronize permissions for a single user like Sarah. External groups are better suited for mapping group memberships from external systems to Vault policies, not individual identity unification.

C. Separate Policies: Managing separate policies per auth method is error-prone and inefficient. Manual synchronization risks inconsistencies, undermining security and manageability.

D. Trust Relationship: Vault does not support configuring trust relationships between auth methods like LDAP and GitHub to sync accounts. This is a misunderstanding of Vault's architecture.

This entity-based approach leverages Vault's identity system to unify Sarah's access, simplifying administration and ensuring consistency.

Reference: <https://developer.hashicorp.com/vault/tutorials/auth-methods/identity>

QUESTION 118

Which of the following storage backends support high availability? (Select four)

- A. Consul
- B. etcd
- C. DynamoDB
- D. Integrated Storage (raft)
- E. Amazon S3
- F. In-Memory

Answer: A, B, C, D

Explanation:

Comprehensive and Detailed In-Depth

Vault supports various storage backends, but only some are designed to provide high availability (HA), ensuring data consistency and fault tolerance across multiple nodes. The four backends that support HA are:

A. Consul: Consul uses a distributed key-value store with a consensus protocol, enabling HA by replicating data across nodes. The documentation notes: "Consul's distributed nature and fault-tolerant design make it a suitable option for ensuring high availability in Vault deployments."

B. etcd: etcd employs the Raft consensus algorithm for distributed coordination, ensuring data

consistency and availability. It's explicitly supported for HA in Vault: "etcd's design ensures data consistency and fault tolerance."

C. DynamoDB: Amazon's managed NoSQL service, DynamoDB, offers replication and fault tolerance, making it HA-capable. Vault leverages these features: "DynamoDB's replication and fault tolerance mechanisms make it a robust choice."

D. Integrated Storage (raft): Vault's built-in storage backend uses the Raft consensus algorithm, providing HA without external dependencies. "Integrated Storage (raft) supports high availability by ensuring data consistency and fault tolerance."

Incorrect Options:

E. Amazon S3: While S3 offers durability, it's an object store not optimized for HA in Vault's context due to latency and lack of native consensus. "It may not be the best choice for ensuring high availability of Vault data."

F. In-Memory: This stores data in volatile memory, losing it on restart, and does not support H

A. "In-Memory storage backend does not support high availability as it is volatile."

These HA-capable backends ensure Vault remains operational and consistent in multi-node setups.

Reference: <https://developer.hashicorp.com/vault/docs/configuration/storage>

QUESTION 119

Short-lived, dynamically generated secrets provide organizations with many benefits. Select the benefits from the options below. (Select four)

A. Each application instance can generate its own credentials, rather than using a shared credential across all application instances

B. Credentials only exist when needed

C. Applications only have access to privileged accounts when needed

D. Credentials accidentally checked into a code repo or discovered in a text file are likely to be invalid

E. Dynamic credentials do not change, so legacy applications can easily take advantage of them

Answer: A, B, C, D

Explanation:

Comprehensive and Detailed In-Depth

Dynamic secrets in Vault are generated on-demand and have short lifespans, offering significant security and operational benefits:

A. Unique Credentials per Instance: "Each application instance can generate its own credentials" isolates access, reducing the blast radius of a compromise. The documentation highlights: "This improves security by isolating access."

B. On-Demand Existence: "Credentials only exist when needed" minimizes exposure time. Vault's design ensures "dynamic secrets do not exist until they are read," reducing theft risk.

C. Least Privilege Enforcement: "Applications only have access to privileged accounts when needed" aligns with security best practices. "This helps enforce the principle of least privilege," per the docs.

D. Invalidation of Leaked Credentials: "Credentials accidentally checked into a code repo or discovered in a text file are likely to be invalid" due to their short lifespan and revocation. "Dynamic secrets can be revoked immediately after use."

Incorrect Option:

E. Static Nature Misconception: "Dynamic credentials do not change" is false. The documentation counters: "Dynamic secrets change," enhancing security, but this may challenge legacy apps, not ease their use.

These benefits collectively enhance security by limiting credential exposure and scope.

Reference: <https://developer.hashicorp.com/vault/tutorials/getting-started/getting-started-dynamicsecrets>

QUESTION 120

What is the default TTL for tokens in Vault if one is not specified?

A. 24 hours (1 day)

B. 15 minutes

C. 768 hours (32 days)

D. 60 minutes (1 hour)

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

In HashiCorp Vault, the default TTL (Time To Live) for tokens, when not explicitly specified, is 768 hours, equivalent to 32 days. This applies to both the initial TTL and the maximum TTL unless overridden.

Default Configuration: The documentation states: "When no specific TTL is provided, a generated token will inherit the default TTL which is 768 hours (32 days)." This long default ensures usability in many scenarios while allowing customization.

Customization Option: Operators can adjust this using commands like `vault write sys/mounts/auth/token/tune default_lease_ttl=1h max_lease_ttl=24h`, but without such tuning, 768 hours applies.

Incorrect Options:

A. 24 hours: Too short for Vault's default; it's a common custom setting instead.

B. 15 minutes: Far too brief and not aligned with Vault's defaults.

D. 60 minutes: Another common custom value, not the default.

This default balances usability with security, encouraging explicit configuration for shorter-lived tokens when needed.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/tokens>

QUESTION 121

Select the two paths below that would be permitted for read access based on the following Vault policy:

```
path "secret/+training/*" {  
  capabilities = ["create", "read"]  
}
```

- A. secret/business/training
- B. secret/cloud/training/test/exam
- C. secret/departments/certification/api
- D. secret/departments/training/vault

Answer: BD

Explanation:

Comprehensive and Detailed In-Depth

Vault policies use path-based syntax with wildcards (+ for one segment, * for zero or more) to define permissions. The policy path "secret/+training/*" { capabilities = ["create", "read"] } grants "create" and "read" access to paths matching this pattern.

Path Analysis:

The + wildcard matches exactly one segment after "secret/".

"training/" must follow that segment.

The * wildcard allows any number of subsequent segments (including none).

Correct Paths:

B. secret/cloud/training/test/exam: Matches as "cloud" fits +, followed by "training/", and "test/exam" fits *. "Permitted since + allows for cloud and * allows for test/exam."

D. secret/departments/training/vault: Matches with "departments" as +, "training/" as +, and "vault" as *. "Permitted since + allows for departments and vault is in place of *."

Incorrect Paths:

A. secret/business/training: Fails because there's no trailing segment after "training/" to match *. "Not permitted since the wildcard is AFTER training."

C. secret/departments/certification/api: Fails because "certification" replaces "training/", which is required. "Not permitted since certification does not equal training."

This policy targets paths with a specific structure, ensuring precise access control.
Reference: <https://developer.hashicorp.com/vault/docs/concepts/policies#policy-syntax>

QUESTION 122

Which of the following features in Vault will replicate service tokens between clusters?

- A. Disaster Recovery Replication
- B. Performance Replication
- C. Vault Agent
- D. Integrated Storage

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

Vault Enterprise supports replication to synchronize data across clusters, with two main types:

Disaster Recovery (DR) Replication and Performance Replication. Only one replicates service tokens:

A. Disaster Recovery Replication: This feature replicates critical data, including service tokens, between clusters for warm-standby failover. "DR clusters are essentially a warm-standby and do replicate tokens from the primary cluster," per the documentation. This ensures continuity in disaster scenarios.

Incorrect Options:

B. Performance Replication: Focuses on scaling read performance, not token replication.

"Performance clusters create and maintain their own tokens. These tokens are NOT replicated."

C. Vault Agent: A client-side tool for token management, not cluster replication. "It does not specifically replicate service tokens between clusters."

D. Integrated Storage: A storage backend, not a replication mechanism. "It does not directly replicate service tokens between clusters."

DR Replication is designed for full data consistency, including tokens, across clusters.

Reference: <https://developer.hashicorp.com/vault/docs/enterprise/replication>

QUESTION 123

What command can be used to update a Vault policy named web-app-1 using the command line?

- A. vault policy create web-app-1 web.hcl
- B. vault policy fmt web.hcl
- C. vault policy update web-app-1 web.hcl
- D. vault policy write web-app-1 web.hcl

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

To update an existing Vault policy via the CLI, the correct command is vault policy write:

D. vault policy write web-app-1 web.hcl: This command updates (or creates if it doesn't exist) the policy named "web-app-1" with the contents of "web.hcl". The documentation states: "The write keyword is used to update an existing policy with the contents of the specified file."

Incorrect Options:

A. vault policy create: No such subcommand exists; create is invalid. "The create keyword is not a valid subcommand."

B. vault policy fmt: Formats the HCL file but doesn't update Vault. "It is used to format a policy file."

C. vault policy update: Incorrect syntax; Vault uses write, not update. "There is no update command, only write."

The write command's dual purpose (create or update) simplifies policy management.

Reference: <https://developer.hashicorp.com/vault/docs/commands/policy/write>

QUESTION 124

Your organization has many applications needing heavy read access to Vault. As these applications integrate with Vault, the primary Vault cluster's performance is negatively impacted. What feature can you use to scale the cluster and improve performance?

A. Add additional standby nodes

B. Enable multiple secrets engines for the applications

C. Enable control groups

D. Add performance standby nodes

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

To address performance issues from heavy read access, Vault Enterprise offers performance standby nodes:

D. Add performance standby nodes: These nodes handle read-only requests locally, offloading the primary cluster. "Vault Enterprise offers additional features that allow HA nodes to service read-only requests on the local standby node," improving scalability and performance.

Incorrect Options:

A. Additional Standby Nodes: Standard HA standby nodes focus on failover, not read scaling. "May help with high availability, but not directly address performance."

B. Multiple Secrets Engines: Organizes secrets but doesn't scale read performance. "Does not directly address performance issues."

C. Control Groups: A resource management feature, not for scaling Vault. "Not directly related to scaling the Vault cluster."

Performance standby nodes distribute read workloads effectively in Vault Enterprise.

Reference: <https://developer.hashicorp.com/vault/docs/enterprise/performance-standby>

QUESTION 125

When a lease is created, what actions can be performed by using only the lease ID? (Choose two)

- A. Renew the lease
- B. Revoke the lease
- C. Extend the max TTL for the lease
- D. Authenticate using the lease ID

Answer: AB

Explanation:

Comprehensive and Detailed In-Depth

A lease ID in Vault identifies a lease associated with dynamic secrets, allowing specific management actions:

- A. Renew the lease: "Using the lease ID, the lease can be renewed up until the maximum TTL," extending its duration without altering other properties.
- B. Revoke the lease: "It is possible to revoke the lease, which immediately invalidates the lease and any associated resources." This terminates the lease instantly.

Incorrect Options:

- C. Extend the max TTL: Requires configuration changes beyond the lease ID. "This operation typically involves modifying the configuration."
- D. Authenticate: Lease IDs are for lease management, not authentication. "The lease ID does not have any direct relationship to authentication processes."

Lease IDs enable precise control over dynamic secret lifecycles.

Reference: <https://developer.hashicorp.com/vault/docs/commands/lease>

QUESTION 126

Based on the output below, how many policies have been added to Vault?

```
$ vault policy list
base
default
root
web-app-1
automation-team
```

- A. 3
- B. 4
- C. 1
- D. 2

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

The vault policy list command displays all policies in Vault. The output lists five policies: "base", "default", "root", "web-app-1", and "automation-team". However, "root" and "default" are built-in policies:

Built-in Policies:

"root": A superuser policy created by default.

"default": Provides common permissions, also default. "Vault has two default policies, root and default."

Added Policies:

"base", "web-app-1", "automation-team" are not built-in, meaning they were added. Thus, 3 policies were added.

Incorrect Options:

B. 4: Overcounts by including one built-in.

C. 1, D. 2: Undercounts the added policies.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/policies#built-in-policies>

QUESTION 127

Although batch and service tokens share many characteristics, which of the following are true only about batch tokens? (Select three)

- A. Can create child tokens
- B. Are renewable up until the max TTL
- C. Maintain a single fixed TTL
- D. They are valid for either the primary or any secondary clusters

E. They are not persisted to disk

Answer: C, D, E

Explanation:

Comprehensive and Detailed In-Depth

Batch and service tokens differ in key ways, with these unique to batch tokens:

C. Maintain a single fixed TTL: "Batch tokens maintain a single fixed TTL," non-renewable, unlike service tokens.

D. Valid across clusters: "They are valid for either the primary or any secondary clusters," enhancing flexibility in replicated setups.

E. Not persisted to disk: "Batch tokens are not persisted to disk," reducing exposure risk.

Incorrect Options:

A. Can create child tokens: "Batch tokens cannot create child tokens," unlike service tokens.

B. Renewable: "Batch tokens are not renewable," a key distinction from service tokens.

Batch tokens prioritize lightweight, ephemeral use.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/tokens#token-type-comparison>

QUESTION 128

What features are offered by the Vault Agent? (Select three)

A. Auditing

B. Templating

C. Auto-auth

D. Secret caching

Answer: B, C, D

Explanation:

Comprehensive and Detailed In-Depth

The Vault Agent is a client-side daemon with these features:

B. Templating: "Allows rendering of user-supplied templates by Vault Agent," integrating secrets into configs.

C. Auto-auth: "Automatically authenticate to Vault and manage token renewal," simplifying auth workflows.

D. Secret caching: "Allows client-side caching of responses," reducing Vault load.

Incorrect Option:

A. Auditing: Handled by Vault's audit devices, not Agent. "Auditing is typically handled by enabling audit devices."

Reference: <https://developer.hashicorp.com/vault/docs/v1.16.x/agent-and-proxy/agent>

QUESTION 129

You need to decrypt customer data to provide it to an application. When you run the decryption command, you get the output below. Why does the response not directly reveal the cleartext data?

```
$ vault write transit/decrypt/phone_number ciphertext="vault:v1:tgx2vsxtlQRfyLSKvem..."
```

Key Value

--- -----

```
plaintext aGFzaGljb3JwIGNlcnRpZmllZDogdmF1bHQgYXNzb2NpYXRl
```

- A. The user does not have permission to view the cleartext data
- B. The output is base64 encoded
- C. The output is actually a response wrapped token that needs to be unwrapped
- D. The original data must have been encrypted

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

The Vault Transit secrets engine returns decrypted data in base64-encoded format:

B. The output is base64 encoded: "All plaintext data must be base64-encoded before being encrypted by Vault. As a result, decrypted data is always base64 encoded." Users must decode it (e.g., using `base64 -d`) to see cleartext.

Incorrect Options:

- A. Permission Issue: Permissions would cause an error, not encoded output. "Not because the user lacks permission."
- C. Wrapped Token: The output is plaintext, not a token. "Not a response wrapped token."
- D. Original Encryption: Irrelevant; the issue is encoding, not encryption state.

This encoding ensures safe transmission of binary data.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/transit#usage>

QUESTION 130

You are considering using HCP Vault Dedicated but are concerned about differences between a hosted version and a self-hosted deployment. Which of the following statements is true about HCP Vault Dedicated?

- A. HCP Vault Dedicated provides a similar experience to self-hosted Vault Enterprise because it uses the same Vault binary
- B. HCP Vault Dedicated can be deployed on any cloud provider, including AWS, Azure, and Google Cloud, with full multi-cloud support
- C. HCP Vault Dedicated requires different CLI commands and APIs compared to self-hosted Vault Enterprise
- D. HCP Vault Dedicated is currently limited to a single region and cannot be deployed across multiple regions

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

HCP Vault Dedicated is a managed Vault service provided by HashiCorp, designed to mirror the selfhosted Vault Enterprise experience while simplifying deployment:

A. Same Vault Binary: "HCP Vault Dedicated provides a similar experience to self-hosted Vault Enterprise because it uses the same Vault binary." This ensures consistency in functionality, CLI commands, APIs, and UI interactions, making it familiar to users of self-hosted Vault. The documentation confirms: "HCP Vault Dedicated uses the same binary as self-hosted Vault Enterprise, which means you will have a consistent user experience."

Incorrect Options:

B. Multi-Cloud Deployment: HCP Vault Dedicated is a HashiCorp-managed service, not deployable by users on any cloud provider. "It is specifically offered as a hosted solution by HashiCorp and does not support deployment on other cloud platforms." It currently supports AWS and Azure, but not full multi-cloud flexibility.

C. Different CLI/APIs: The use of the same binary ensures identical CLI and API interfaces. "Does not require different CLI commands and APIs compared to self-hosted Vault Enterprise."

D. Single Region Limitation: It supports multiple regions (e.g., North America, Asia, Europe). "Not limited to a single region and can be deployed across multiple regions."

This consistency aids adoption for organizations transitioning to a managed solution.

Reference: <https://developer.hashicorp.com/hcp/docs/vault/what-is-hcp-vault#why-hcp-vaultdedicated>

QUESTION 131

What command can be used to revoke all leases associated with a database role named prod-mysql?

- A. vault lease revoke database/role/prod-mysql
- B. vault lease revoke -prefix database/creds/prod-mysql
- C. vault revoke database/role/prod-mysql

D. vault lease revoke database/creds/prod-mysql

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

To revoke all leases tied to a specific database role like prod-mysql, the correct command leverages the -prefix flag:

B. vault lease revoke -prefix database/creds/prod-mysql: This command revokes all leases with the prefix database/creds/prod-mysql, which corresponds to credentials generated by the prod-mysql role in the database secrets engine. "To immediately revoke all leases associated with a specific role, the user can run the command vault lease revoke -prefix database/creds/prod-mysql," ensuring targeted revocation without affecting other roles.

Incorrect Options:

A. vault lease revoke database/role/prod-mysql: Incorrect path; roles are at database/roles/, not leases. "Does not specify the correct path for revoking leases."

C. vault revoke: Missing lease subcommand; incorrect syntax. "Does not follow the correct syntax for revoking leases."

D. vault lease revoke database/creds/prod-mysql: Targets a single lease, not all, without -prefix. "Does not include the -prefix flag to revoke all leases."

The -prefix approach ensures comprehensive lease cleanup for the role.

Reference: <https://developer.hashicorp.com/vault/docs/commands/lease/revoke#examples>

QUESTION 132

Jarrad is an AWS engineer and has provisioned a new EC2 instance running MySQL since his application requires a specific MySQL version. He wants to integrate Vault into his workflow but is new to Vault. What secrets engine should Jarrad use to integrate this new database running in AWS?

- A. azure
- B. database
- C. kv
- D. aws

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

For integrating a MySQL database on an EC2 instance with Vault, the database secrets engine is the

appropriate choice:

B. database: "The 'database' secrets engine in Vault is specifically designed for integrating with databases like MySQL." It generates dynamic credentials, manages rotations, and supports MySQL plugins, ideal for Jarrad's use case. "To manage the database resource, the database secrets engine should be used, specifically with the MySQL plugin."

Incorrect Options:

A. azure: For Azure-specific credential management, not databases. "Used for generating Azure service principal credentials."

C. kv: Stores static secrets, not dynamic database credentials. "Used for storing arbitrary secrets in a key-value pair format."

D. aws: Manages AWS credentials, not database integration. "Used for generating AWS access keys." The database engine's MySQL support is agnostic to the hosting platform (EC2 vs. RDS), focusing on the database itself.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/databases/mysql-maria>

QUESTION 133

Suzy is a Vault user that needs to create and replace values at the path `secrets/automation/apps/chef`. Does the following policy permit her the permissions to do so?

text

CollapseWrapCopy

```
path "secrets/automation/apps/chef" {
  capabilities = ["create", "read", "list"]
}
```

A. No, the policy would deny Suzy from performing certain actions

B. Yes, the policy has appropriate permissions

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

The policy grants specific capabilities, but not all required for Suzy's needs:

A. No, Denied Actions: The policy allows "create", "read", "list" at `secrets/automation/apps/chef`. "Create" permits adding new key-value pairs, but "replace" (updating existing values) requires the "update" capability, which is missing. "If Suzy needs to create AND replace values (update), she needs both create and update capabilities."

Incorrect Option:

B. Yes: Incorrect, as "update" is omitted. "Does not include the update capability, which is required for replacing values."

Without "update", Suzy can create but not replace values, limiting her ability.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/policies>

QUESTION 134

What API endpoint is used to enable and configure a secrets engine?

- A. /v1/sys/init
- B. /v1/sys/mounts
- C. /v1/sys/config
- D. /v1/sys/plugins/catalog

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

The correct API endpoint for managing secrets engines is:

B. /v1/sys/mounts: "The /sys/mounts endpoint is used to manage secrets engines in Vault." It enables and configures secrets engines by mounting them at specific paths, per the documentation.

Incorrect Options:

- A. /v1/sys/init: For Vault initialization, not secrets engines. "Used for initializing the Vault server."
- C. /v1/sys/config: Manages system-wide settings, not engines. "Used for system-wide configurations."
- D. /v1/sys/plugins/catalog: Manages plugins, not engine mounting. "Related to managing plugins and their catalog."

This endpoint is central to Vault's secrets engine lifecycle.

Reference: <https://developer.hashicorp.com/vault/api-docs/system/mounts>

QUESTION 135

Tanner manages a data processing application and needs to be sure the data being processed is encrypted so it is securely stored post-processing. Which secrets engines can encrypt data? (Select three)

- A. transit
- B. KMIP
- C. SSH
- D. transform

Answer: A, B, D

Explanation:

Comprehensive and Detailed In-Depth

Vault offers secrets engines for encryption:

A. transit: "Designed specifically for encryption and decryption operations," ideal for securing data at rest.

B. KMIP: "Integrates with external Key Management Systems that support the KMIP protocol," enabling encryption via external keys.

D. transform: "Used for data transformation operations, including encryption and decryption," with custom pipelines.

Incorrect Option:

C. SSH: "Used for dynamic SSH key generation and management," not general data encryption. "Only the Transit and Transform secrets engines can encrypt/decrypt data," with KMIP adding external key support.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/transit>,

<https://developer.hashicorp.com/vault/docs/secrets/transform>

QUESTION 136

You have TBs of data encrypted by Vault stored in a database and are worried about Vault becoming unavailable and not being able to decrypt the data.

a. Is it possible to export the encryption key to store it somewhere else in the event Vault becomes unavailable?

A. Yes, as long as the key was configured to be exportable when it was created

B. No, you cannot export the encryption key from Vault

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

Vault allows key export under specific conditions:

A. Yes, if Exportable: "When creating the key, the exportable flag must be set as true. By default, it is false." If set, "this enables the keys to be exportable," allowing retrieval for external storage. "Once set, this cannot be disabled."

Incorrect Option:

B. No: Incorrect if the key is exportable. "You cannot export the encryption key from Vault if it was not configured to be exportable."

This feature, while not best practice, supports disaster recovery scenarios.

Reference: <https://developer.hashicorp.com/vault/api-docs/secret/transit#exportable>

QUESTION 137

True or False? To encrypt existing encrypted data with the latest version of the encryption key, you need to first decrypt it and then request Vault to re-encrypt it with the latest version of the encryption key.

A. True

B. False

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

This statement is false due to Vault's rewrap feature:

B. False: "You can use the rewrap feature of the transit secrets engine to rewrap the data with the latest version of the key. This process does not reveal the plaintext data." Rewrapping updates the encryption key version without decryption.

Incorrect Option:

A. True: Incorrect; rewrapping avoids the decrypt-re-encrypt cycle.

This enhances security and efficiency in key rotation.

Reference: <https://developer.hashicorp.com/vault/docs/secrets/transit>

QUESTION 138

Elijah manages a legacy application that requires strict control over when its service account credentials change. Which type of credential should be used for this legacy application?

A. static

B. dynamic

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

For strict control over credential changes:

A. static: "Static credentials should be used here so they can be controlled outside of Vault." They remain constant until manually updated, suiting legacy needs. "Stored within Vault using the KV secrets engine."

Incorrect Option:

B. dynamic: "Designed to change automatically," which conflicts with strict control requirements. Static credentials offer predictability for legacy systems.

Reference: <https://developer.hashicorp.com/vault/tutorials/secrets-management/static-secrets>

QUESTION 139

What occurs when a Vault cluster cannot maintain a quorum while using the Integrated Storage backend?

- A. Vault continues to operate in read-only mode until quorum is restored
- B. The cluster becomes unavailable and cannot commit new logs
- C. Vault automatically promotes a standby node to a leader to restore quorum
- D. Vault temporarily switches to local storage until quorum is regained

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

Integrated Storage (Raft) requires a quorum:

B. Unavailable: "If a cluster cannot achieve quorum, the cluster becomes unavailable and cannot commit new logs." Quorum is "a majority of members from a peer set," e.g., 3 of 5 nodes.

Incorrect Options:

- A. Read-Only: "Does not continue to operate in read-only mode."
- C. Auto-Promotion: "Does not automatically promote a standby node."
- D. Local Storage: "Does not temporarily switch to local storage."

Quorum loss halts operations to ensure consistency.

Reference: <https://developer.hashicorp.com/vault/docs/v1.16.x/internals/integrated-storage>

QUESTION 140

Assuming default configurations, which of the following operations require a threshold of key shares to perform? (Select three)

- A. Rotating the Vault encryption key to adhere to internal security policies
- B. Unsealing Vault after a scheduled maintenance to install patches

- C. Generating a new root token as a break-glass procedure
- D. Creating a new set of recovery keys due to an employee leaving the organization

Answer: B, C, D

Explanation:

Comprehensive and Detailed In-Depth

Certain operations require unseal keys:

- B. Unsealing: "Unsealing the Vault requires a threshold of unseal keys."
- C. Root Token: "Generating a new root token requires a threshold of unseal keys."
- D. Recovery Keys: "Changing the unseal/recovery keys requires the current threshold."

Incorrect Option:

- A. Key Rotation: "An online operation and does not cause downtime," no shares needed.

Reference: <https://developer.hashicorp.com/vault/docs/commands/operator/rekey>

QUESTION 141

Which of the following best describes response wrapping?

- A. The response is Base64 encoded, and the user must decode the response to retrieve the cleartext data
- B. Rather than provide a direct response, Vault returns a token and an accessor
- C. Vault responds with an encrypted version of the response, decrypted via transit
- D. Vault inserts the response into a single-use token's cubbyhole

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

Response wrapping secures responses:

- D. Cubbyhole: "Vault takes the response and inserts it into the cubbyhole of a single-use token."

Incorrect Options:

- A. Base64: "Not directly related to response wrapping."
- B. Token/Accessor: "Describes token use, not wrapping."
- C. Transit: "Not involved in response wrapping."

Reference: <https://developer.hashicorp.com/vault/docs/concepts/response-wrapping#overview>

QUESTION 142

You have ciphertext stored in an Amazon S3 bucket encrypted by the key named prod-customer. Will Vault decrypt this data with the command `vault write transit/decrypt/prod-customer ciphertext="vault:v4:xa1f9FIJtn13em/Wb7QCsXsU/kCOn7..."` given this output?

```
$ vault read transit/keys/prod-customer
```

```
Key Value
```

```
--- -----
```

```
...
```

```
keys map[4:1549347108 5:1549347109 6:1549347110]
```

```
latest_version 6
```

```
min_available_version 0
```

```
min_decryption_version 4
```

```
min_encryption_version 0
```

Will Vault decrypt this data for you by running the following command?

```
$ vault write transit/decrypt/prod-customer
```

```
ciphertext="vault:v4:xa1f9FIJtn13em/Wb7QCsXsU/kCOn7..."
```

A. Yes, because the minimum decryption key configuration is set to 4

B. No, since the latest version of the key is 6

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

Vault can decrypt if the key version is available:

A. Yes: "The minimum decryption version set to 4 indicates that Vault will be able to decrypt data encrypted with version 4 of the key."

Incorrect Option:

B. No: "The latest version being 6 does not impact Vault's ability to decrypt earlier versions."

Reference: <https://developer.hashicorp.com/vault/docs/secrets/transit#usage>

QUESTION 143

True or False? You can create and update Vault policies using the UI.

A. True

B. False

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

The Vault UI supports policy management:

A. True: "You can indeed create and update Vault policies within the UI."

Incorrect Option:

B. False: Incorrect; UI functionality exists.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/policies>

QUESTION 144

What is the default value of the VAULT_ADDR environment variable?

A. http://0.0.0.0:8200

B. https://vault.example.com:8200

C. https://0.0.0.0:8200

D. http://vault.example.com:8200

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

The default address is:

C. https://0.0.0.0:8200: "Vault assumes the value of https://0.0.0.0:8200 when you make requests to Vault."

Incorrect Options:

A, B, D: Non-default values requiring manual setting.

Reference: https://developer.hashicorp.com/vault/docs/commands#vault_addr

QUESTION 145

True or False? The root and default policies can be deleted if they are not needed or being used.

A. True

B. False

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

In HashiCorp Vault, the root and default policies are built-in and cannot be deleted:

B. False: "The default and root policy cannot be deleted. You don't have to use them, but you can't delete them." The root policy grants superuser privileges, while the default policy provides common permissions assigned to new tokens unless explicitly excluded (e.g., via vault token create -nodefault-policy). Their permanence ensures baseline functionality and security.

Incorrect Option:

A. True: Incorrect; these policies are immutable in terms of deletion. "The root and default policies cannot be deleted."

This design choice maintains Vault's operational integrity and security model.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/policies#built-in-policies>

QUESTION 146

What header must be included in an API request in order to provide authentication validation?

- A. X-Token-Vault
- B. X-Vault-Token
- C. X-Token-Creds
- D. X-Vault-Creds

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

For Vault API authentication:

B. X-Vault-Token: "The token for authentication is set directly as a header for the HTTP API. The header should be either X-Vault-Token: <token> or Authorization: Bearer <token>." This header carries the client token required to validate the request's authenticity and permissions.

Incorrect Options:

A. X-Token-Vault: Incorrect naming convention. "Does not follow the standard naming conventions."

C. X-Token-Creds: Not recognized by Vault. "Does not align with standard authentication headers."

D. X-Vault-Creds: Invalid for authentication. "Does not correspond to the standard mechanism."

The X-Vault-Token header is critical for secure API interactions.

Reference: <https://developer.hashicorp.com/vault/docs/auth/token#authentication>

QUESTION 147

Hanna is working with Vault and has been assigned a namespace called integration, where she stores all her secrets. Hanna configured her application to use the following API request, but the request is

failing. What changes below will help Hanna correctly retrieve the secret? (Select two)

```
$ curl \
--header "X-Vault-Token:hvs.lzrmRe5Y3LMcDRmOttEjWoag" \
--request GET \
https://vault.example.com:8200/v1/secret/data/my-secret
```

- A. `$ curl --header "X-Vault-Token:hvs.lzrmRe5Y3LMcDRmOttEjWoag" --request GET\integration https://vault.example.com:8200/v1/secret/data/my-secret`
- B. `$ curl --header "X-Vault-Token:hvs.lzrmRe5Y3LMcDRmOttEjWoag" --request GET --namespace "integration" https://vault.example.com:8200/v1/secret/data/my-secret`
- C. `$ curl --header "X-Vault-Token:hvs.lzrmRe5Y3LMcDRmOttEjWoag" --request GET https://vault.example.com:8200/v1/integration/secret/data/my-secret`
- D. `$ curl --header "X-Vault-Token:hvs.lzrmRe5Y3LMcDRmOttEjWoag" --header "X-Vault-Namespace:integration" --request GET https://vault.example.com:8200/v1/secret/data/my-secret`

Answer: CD

Explanation:

Comprehensive and Detailed In-Depth

Vault namespaces require specifying the integration namespace to access secrets:

C. Path-Based: "Modifying the API request URL to include the namespace 'integration' before the path to the secret ensures that Hanna is accessing the secret from the correct namespace." The URL `https://vault.example.com:8200/v1/integration/secret/data/my-secret` correctly prepends the namespace.

D. Header-Based: "Adding the 'X-Vault-Namespace:integration' header specifies the namespace where the secrets are stored." This header method keeps the base path unchanged while targeting the integration namespace.

Incorrect Options:

A: Incorrect syntax; `\integration` is malformed. "The API request URL structure is incorrect."

B: `--namespace` is not a curl flag. "The '`--namespace`' flag is not a valid option."

"To invoke an API on a specific namespace, you can pass the target namespace in the X-Vault-`Namespace` header or make the namespace as a part of the API endpoint."

Reference: <https://developer.hashicorp.com/vault/docs/enterprise/namespaces>

QUESTION 148

True or False? Although `AppRole` is designed for machines, humans can use it to authenticate to Vault if you wish.

A. True

B. False

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

AppRole's flexibility allows human use:

A. True: "Although AppRole is primarily designed for machine-to-machine authentication, it can also be used by humans to authenticate to Vault if needed." It uses a `role_id` and `secret_id`, which, while less convenient for humans, are technically usable. "Yeah, absolutely. Although it's not super friendly for us humans to remember the values, you could use it if you wanted to."

Incorrect Option:

B. False: Incorrect; it's not restricted to machines only.

This adaptability broadens AppRole's applicability.

Reference: <https://developer.hashicorp.com/vault/docs/auth/approle>

QUESTION 149

You are enabling a secrets engine in Vault using the CLI. What subcommands are available when using the `vault secrets` command? (Select five)

- A. update
- B. migrate
- C. tune
- D. enable
- E. move
- F. disable
- G. list

Answer: C, D, E, F, G

Explanation:

Comprehensive and Detailed In-Depth

The `vault secrets` command supports:

C. `tune`: "Tune a secrets engine configuration."

D. `enable`: "Enable a secrets engine."

E. `move`: "Move a secrets engine to a new path."

F. `disable`: "Disable a secrets engine."

G. `list`: "List enabled secrets engines."

Incorrect Options:

A. update: Not a subcommand.

B. migrate: Not applicable here.

"The vault secrets command has several subcommands to use when working with secrets engines."

Reference: <https://developer.hashicorp.com/vault/docs/commands/secrets#usage>

QUESTION 150

Kyle enabled the database secrets engine for dynamic credentials. Amy, the senior DBA, accidentally deleted the database users created by Vault, disrupting client applications. How can Kyle manually remove the leases in Vault?

A. No action is required since the leases will eventually expire and be revoked

B. Obtain the individual lease IDs from the application logs and remove them using the vault lease revoke command

C. Use the command vault lease revoke -force flag to delete the leases

D. Revoke all of the leases associated with the entire database secrets engine to be sure they are all removed

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

To clean up disrupted leases:

C. vault lease revoke -force: "Using the vault lease revoke -force flag is the correct way to manually remove leases in Vault." With -prefix, it targets specific leases (e.g., vault lease revoke -force -prefix database/creds/<role>). "This is meant for recovery situations where the secret was manually removed."

Incorrect Options:

A: Waiting risks ongoing issues. "May take time and could cause disruptions."

B: Inaccurate; -force is needed. "Not a valid approach without -force."

D: Too broad, affects other leases. "May impact other valid credentials."

Reference: <https://developer.hashicorp.com/vault/docs/commands/lease/revoke>

QUESTION 151

What command would you use to enable the Kubernetes secrets engine at the path of /k8s-cluster?

A. vault secrets enable -path=k8s-cluster kubernetes

B. vault kv put k8s-cluster type=kubernetes

- C. vault write sys/mounts/k8s-cluster
- D. vault secrets enable kubernetes -path=k8s-cluster

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

Correct syntax is:

A. vault secrets enable -path=k8s-cluster kubernetes: "The secrets enable command enables a secrets engine at a given path." The -path flag precedes the engine type.

Incorrect Options:

B: kv put is for key-value data, not enabling engines.

C: Incorrect CLI syntax; API-focused.

D: Reversed order; path must come first.

Reference: <https://developer.hashicorp.com/vault/docs/v1.16.x/commands/secrets/enable>

QUESTION 152

True or False? Once the lease for a dynamic secret has expired, Vault revokes the credentials on the backend platform for which they were created (i.e., database, AWS, Kubernetes).

- A. True
- B. False

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

Dynamic secrets are managed actively:

A. True: "Once the lease for a dynamic secret has expired, Vault automatically revokes the credentials on the backend platform for which they were created." This cleanup reduces technical debt.

Incorrect Option:

B. False: Incorrect; revocation is automatic.

"When a lease expires, Vault does indeed revoke the credentials on the platform."

Reference: <https://developer.hashicorp.com/vault/docs/concepts/lease>

QUESTION 153

In Vault, there are two main types of tokens, batch and service. Which of the following is true about

the renewable capabilities of each?

- A. Batch tokens cannot be renewed, but service can be renewed up to the max TTL
- B. Tokens cannot be renewed without reauthenticating to Vault
- C. Service tokens cannot be renewed, but batch can be renewed up to the max TTL
- D. Both batch and service tokens can be renewed up to the max TTL

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

Token renewability differs:

A. Correct: "Batch tokens cannot be renewed by Vault, but service tokens can be renewed up to the Max TTL of the token."

Incorrect Options:

B: Service tokens renew without reauth.

C: Reverses the truth.

D: Batch tokens are non-renewable.

Reference: <https://developer.hashicorp.com/vault/tutorials/tokens/tokens>

QUESTION 154

You need a simple and self-contained HashiCorp Vault cluster deployment with minimal dependencies. Which storage backend is best suited for this use case, providing all configuration within Vault and avoiding external services?

- A. Local File Storage Backend
- B. Integrated Storage (raft) Backend
- C. Consul Backend
- D. In-Memory Backend

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

For self-contained deployment:

B. Integrated Storage (raft): "The best choice for a simple and self-contained Vault cluster deployment with minimal dependencies." Uses Raft for consistency, no external services needed.

Incorrect Options:

A: Less reliable for production.

C: Requires Consul.

D: Non-persistent, for testing.

Reference: <https://developer.hashicorp.com/vault/docs/v1.16.x/internals/integrated-storage>

QUESTION 155

Tom needs to set the proper environment variable so he doesn't need to first authenticate to Vault to retrieve dynamically generated credentials for a database server. What environment variable does Tom need to set first before running commands?

A. VAULT_NAMESPACE

B. VAULT_TOKEN

C. VAULT_CAPATH

D. VAULT_CLIENT_KEY

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

To bypass manual auth:

B. VAULT_TOKEN: "The VAULT_TOKEN environment variable holds the contents of the token," enabling seamless access.

Incorrect Options:

A: Sets namespace, not auth.

C, D: TLS-related, not auth.

Reference: https://developer.hashicorp.com/vault/docs/commands#vault_token

QUESTION 156

Tom is authenticating to Vault using the CLI. Which of the following commands allows Tom to authenticate using the userpass method WITHOUT logging his password to the shell history?

A. vault login tom

B. vault login -method=userpass username=tom

C. vault login userpass username=tom password=jerry

D. vault login -method=userpass username=tom password=jerry

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

To avoid logging passwords:

B. Correct: "If you enter the command `vault login -method=userpass username=tom` and press enter, you will be prompted to enter your credentials but they will be hidden."

Incorrect Options:

A: Incomplete.

C, D: Expose password in history.

Reference: <https://developer.hashicorp.com/vault/docs/auth/userpass>

QUESTION 157

Which of the following secrets engines can store static secrets in Vault for future retrieval?

- A. KV
- B. PKI (certificates)
- C. Database
- D. Transit

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

For static secrets:

A. KV: "The KV secrets engine is the ONLY secrets engine that will store static data in Vault for future retrieval."

Incorrect Options:

B, C, D: Generate or encrypt, don't store static secrets.

Reference: <https://developer.hashicorp.com/vault/docs/secrets#secrets-engines>

QUESTION 158

Your organization operates active/active applications across multiple data centers for high availability. Which Vault feature should be used in the secondary data centers to provide local access to secrets?

- A. Performance standby nodes
- B. Customized plugins for the Vault cluster
- C. Disaster recovery cluster
- D. Performance replication cluster

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

For active/active setups:

D. Performance replication cluster: "Should be used in an active/active scenario to ensure applications in both data centers can easily access Vault secrets."

Incorrect Options:

A: Scales single cluster, not multi-DC.

B, C: Not suited for local access.

Reference: <https://developer.hashicorp.com/vault/docs/enterprise/replication#performancereplication-and-disaster-recovery-dr-replication>

QUESTION 159

Which of the following auth methods are intended for machine-to-machine authentication, and not necessarily human (operator) authentication? (Select four)

A. Okta

B. Tokens

C. TLS Certificates

D. Cloud-based Auth methods (AWS, Azure, GCP)

E. LDAP

F. AppRole

Answer: B, C, D, F

Explanation:

Comprehensive and Detailed In-Depth

Machine-oriented methods:

B, C, D, F: "Machine-oriented: AppRole, TLS, tokens, platform-specific methods (cloud, k8s)."

Incorrect Options:

A, E: "Operator-oriented: LDAP, Okta."

Reference: <https://developer.hashicorp.com/vault/tutorials/get-started/why-use-vault#human-andmachine-authentication>

QUESTION 160

Which of the following are valid types of tokens available in Vault? (Select five)

A. Primary token

- B. Batch token
- C. Orphan service token
- D. Service token
- E. Root token
- F. Periodic service token

Answer: B, C, D, E, F

Explanation:

Comprehensive and Detailed In-Depth

Vault supports several token types, each with distinct characteristics:

B. Batch token: "Batch tokens are encrypted binary large objects (blobs) that carry just enough information for authentication." They are lightweight and non-renewable.

C. Orphan service token: "Orphan tokens are not children of their parent; therefore, do not expire when their parent does." A valid subtype of service tokens.

D. Service token: "Service token is the general token that most people talk about when referring to a token in Vault." The standard token type.

E. Root token: "Root tokens are the most powerful tokens in Vault and have full control." Created during initialization.

F. Periodic service token: "Periodic service tokens have a TTL, but no max TTL," renewing automatically for long-running tasks.

Incorrect Option:

A. Primary token: "Not a valid token type in Vault." No such term exists in Vault's documentation. These token types cater to various use cases, from ephemeral to privileged access.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/tokens>

QUESTION 161

True or False? The following policy permits a user to read secrets contained in the path secrets/cloud/apps/jenkins?

text

CollapseWrapCopy

```
path "secrets/cloud/apps/jenkins/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}
```

- A. True
- B. False

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

The policy's path syntax determines access:

B. False: "This policy will NOT permit access to secrets stored under secrets/cloud/apps/jenkins." The wildcard * applies to paths after jenkins/, e.g., secrets/cloud/apps/jenkins/config, but not the exact path secrets/cloud/apps/jenkins. "Notice that in the policy, the wildcard (*) is AFTER the path jenkins, and not AT the jenkins path."

Incorrect Option:

A. True: Incorrect; the policy requires an additional segment to match.

To permit secrets/cloud/apps/jenkins, the policy should be path "secrets/cloud/apps/jenkins" {} or include a broader wildcard like secrets/cloud/apps/*.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/policies>

QUESTION 162

Thomas has authenticated to Vault using the API and has received the following response. What data must Thomas parse from the response in order to continue making requests to Vault?

text

CollapseWrapCopy

```
{
  "request_id": "65897160-fd8b-1f87-c24e-fdba14c9728e",
  "lease_id": "",
  "renewable": false,
  "lease_duration": 0,
  "data": null,
  "wrap_info": null,
  "warnings": null,
  "auth": {
    "client_token": "hvss.lzrmRe5Y3LMcDRmOttEjWoagd92fD29fxakwej_38djs",
    "accessor": "EMX0nv4nr0Y1wXoaN7i0WDW1",
    "policies": ["bryan", "default"],
    "token_policies": ["bryan", "default"],
    "metadata": {"username": "bryan"},
    "lease_duration": 2764800,
    "renewable": true,
    "entity_id": "40e203e8-818e-b6ad-4cb3-0befdbf9b598",
    "token_type": "service",
```

```
"orphan": true
}
}
```

- A. accessor
- B. request_id
- C. client_token
- D. entity_id

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

To continue API requests:

C. client_token: "When you authenticate to Vault using the API, the response will include the client_token, which is required for subsequent responses." This token, found at .auth.client_token, must be included in the X-Vault-Token header.

Incorrect Options:

- A. accessor: Used for token management, not requests.
- B. request_id: Tracks the request, not for auth.
- D. entity_id: Identifies the entity, not for requests.

Reference: <https://developer.hashicorp.com/vault/api-docs/auth/userpass#login>

QUESTION 163

When you are unsealing Vault using unseal keys, what are you actually doing?

- A. Creating the recovery keys
- B. Exporting the encryption key
- C. Reconstructing the root key
- D. Decrypting the Vault data

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

Unsealing involves:

C. Reconstructing the root key: "Unsealing is the process of obtaining the plaintext root key necessary to read the decryption key to decrypt the data, allowing access to the Vault." The unseal

keys reconstruct this root key via Shamir's Secret Sharing.

Incorrect Options:

A: Recovery keys are separate.

B: Keys aren't exported during unseal.

D: Data decryption is a result, not the action.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/seal#seal-unseal>

QUESTION 164

You need to create a limited-privileged token that isn't impacted by the TTL of its parent. What type of token should you create?

A. Service token with a use limit

B. Orphan token

C. Periodic token

D. Root token

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

For independence from parent TTL:

B. Orphan token: "Orphan tokens are not children of their parent; therefore, orphan tokens do not expire when their parent does."

Incorrect Options:

A: Use limit doesn't affect TTL linkage.

C: Periodic tokens renew but follow parent TTL.

D: Root tokens are unrestricted.

Reference: <https://developer.hashicorp.com/vault/tutorials/tokens/tokens#orphan-tokens>

QUESTION 165

Which of the following actions can be performed if you only had access to a token's accessor? (Select four)

A. Look up a token's properties

B. Renew the token

C. Retrieve the actual token ID

D. Revoke the token

E. Look up a token's capabilities on a path

Answer: A, B, D, E

Explanation:

Comprehensive and Detailed In-Depth

A token accessor allows:

A, B, D, E: "This accessor can only be used to perform limited actions: Look up a token's properties, Look up a token's capabilities on a path, Renew the token, Revoke the token." The calling token needs permissions.

Incorrect Option:

C: "Not including the actual token ID."

Reference: <https://developer.hashicorp.com/vault/docs/concepts/tokens#token-accessors>

QUESTION 166

Julie is a developer who needs to ensure an application can properly renew its lease for AWS credentials it uses to access data in an S3 bucket. Although the application would generally use the API, what is the equivalent CLI command to perform this action?

- A. vault renew aws/roles/s3-read-onlye6b9a2-296-83d9-2fe0-c11e846bdc99
- B. vault lease renew aws/creds/s3-read-onlye6b9a2-296-83d9-2fe0-c11e846bdc99
- C. vault lease renew aws/roles/s3-read-onlye6b9a2-296-83d9-2fe0-c11e846bdc99
- D. vault lease renew aws/creds/s3-read-only

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

To renew AWS credential leases:

B. Correct: "The proper command would be vault lease renew aws/creds/s3-read-onlye6b9a2-296-83d9-2fe0-c11e846bdc99." Targets the credential lease ID.

Incorrect Options:

A, C: Wrong path (roles vs. creds).

D: Missing lease ID.

Reference: <https://developer.hashicorp.com/vault/docs/commands/lease/renew>

QUESTION 167

Which of the following are supported auth methods for Vault? (Select six)

- A. AWS
- B. Kubernetes

- C. Token
- D. OIDC/JWT
- E. Userpass
- F. Cubbyhole
- G. AppRole

Answer: A, B, C, D, E,
G

Explanation:

Comprehensive and Detailed In-Depth

Supported auth methods:

A, B, C, D, E, G: "All of the options are valid auth methods except for Cubbyhole." Detailed in Vault docs.

Incorrect Option:

F: "Cubbyhole is a secrets engine."

Reference: <https://developer.hashicorp.com/vault/docs/auth>

QUESTION 168

Which of the following tokens are representative of a batch token? (Select two)

A.

hvr.AAAAAQL_tyer_gNuQqvQYPVQgsNxjap_YW1NB2m4CDHHadQo7rF2XLFGdwNJplAZNKbfloOvifrb
pRCGdgG1taTqmC7Da_qftN64zeL10SmNwEoDTiPzC_1aS1KExbtVftU3Sx16cBVqaynwsYRDfVnfTAffE

B.

hvb.CAESIKOOSODDNGUJQe3EmsS8EQthulLjxRDhan_Axte2OrmPGiAKHGH2cy5KVnNhM25JdG82cDB
0a1ZDbWhPTlAyeKMQHg

C.

hvb.AAAAAQJnAGuRT_z8FD_jOwP26zYaNzJ456_SVqse0oXtaqrpaLUC3LIHrUoJhQPylGX7A6K_dcS0shi
ql6g5-BVpz0QIkCm7ePFQVjDT2HcIf8C6FNgkW313vYgBGP8lzQHebtspC0pqK64cfyU_qPKIka2u4ngjsoy

D. hvs.493n55sZp2lX2zyQfpkHTkL4

Answer: BC

Explanation:

Comprehensive and Detailed In-Depth

Batch tokens are identified by:

B, C: "In newer versions of Vault (Vault 1.10+), batch tokens are prepended with hvb."

Incorrect Options:

A: hvr prefix is invalid.

D: hvs indicates service token.

Reference: <https://developer.hashicorp.com/vault/tutorials/tokens/tokens>

QUESTION 169

True or False? Once you authenticate to Vault using the API, subsequent requests will automatically be permitted without further interaction.

- A. True
- B. False

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

API auth requires ongoing token use:

B. False: "Once you authenticate using the API, subsequent requests are not automatically permitted without further interaction. Each request to Vault requires authentication using the token returned by Vault."

Incorrect Option:

A. True: Incorrect; token must be provided.

Reference: <https://developer.hashicorp.com/vault/docs/auth/approle>

QUESTION 170

You have multiple Kubernetes pods that need frequent access to Vault to retrieve credentials for establishing connectivity to a backend database. You enable the Kubernetes auth method in Vault. What resource do you need to create within Kubernetes to complete this configuration?

- A. Username and password for kubectl
- B. k8s service account token
- C. A Vault token for authentication
- D. An AppRole role_id and secret_id

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

Kubernetes auth requires:

B. k8s service account token: "The kubernetes auth method can be used to authenticate with Vault

using a Kubernetes Service Account Token."

Incorrect Options:

A, C, D: Not specific to Kubernetes auth.

Reference: <https://developer.hashicorp.com/vault/docs/auth/kubernetes>

QUESTION 171

After setting up a new HashiCorp Vault server with the default configurations, which method can be used to unseal Vault?

- A. Log on to each Vault node and provide the root token
- B. Running vault operator init to regenerate unseal keys and automatically unseal the Vault
- C. Submit a threshold of unseal keys to reconstruct the root key
- D. Restart the Vault service, which will automatically unseal it

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

Unsealing a new Vault:

C. Correct: "When a Vault server is started, it starts in a sealed state. Unsealing is the process of obtaining the plaintext root key necessary to read the decryption key to decrypt the data."

Incorrect Options:

A, B, D: Misrepresent unsealing process.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/seal>

New Topic: Topic 4, Exam Pool D

QUESTION 172

A large organization uses Vault for various use cases with multiple auth methods enabled. A user can authenticate via LDAP, OIDC, or a local userpass account, but they receive different policies for each method and often need to log out and back in for different actions. What can be configured in Vault to ensure users have consistent policies regardless of their authentication method?

- A. Enable the SSH secrets engine and instruct the user to obtain credentials using the new secrets engine
- B. Create a new entity and map the aliases from each of the available auth methods
- C. Assign the default policy to the user's policy used by each auth method
- D. Provide the user with an AppRole role-id and secret-id for authentication

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

In HashiCorp Vault, when a user authenticates via multiple methods (e.g., LDAP, OIDC, userpass), each authentication method generates a distinct token with its own set of policies based on the configuration of that auth method. This can lead to inconsistent access levels depending on how the user logs in. To address this and ensure consistent policies across all authentication methods, Vault's Identity system can be utilized. Specifically, creating an entity and mapping aliases from each authentication method to that entity allows Vault to associate a single logical identity with the user, regardless of how they authenticate.

An entity in Vault represents a single identity (e.g., a user or application) and can have multiple aliases tied to different auth methods. Each alias links the authentication method's identifier (e.g., LDAP username, OIDC subject) to the entity. Policies can then be assigned directly to the entity, ensuring that all tokens generated for that entity”across any auth method”inherit the same set of policies. This eliminates the need for users to log out and back in to switch contexts, as their access remains consistent.

Option A (SSH secrets engine) is unrelated, as it manages SSH credentials, not policy consistency across auth methods. Option C (assigning the default policy) doesn't guarantee consistency, as the default policy might not include all required permissions and doesn't unify policies across methods. Option D (AppRole) is a machine-oriented auth method and doesn't solve the multi-method human user scenario. The correct approach, as per Vault's Identity documentation, is to leverage entities and aliases.

Reference:

Vault Identity Documentation

Vault Entities and Aliases Tutorial

QUESTION 173

Your organization has applications in a primary data center and a secondary warm-standby site. You want to configure Vault replication between the primary and secondary clusters. If the primary fails over to the secondary, the applications must interact with Vault without re-authenticating. What type of Vault replication would you use?

- A. Performance Replication
- B. Integrated Storage
- C. Disaster Recovery Replication
- D. Vault Secrets Operator

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

Vault Enterprise supports two replication types: Performance Replication and Disaster Recovery (DR) Replication. The key requirement here is that applications must continue interacting with Vault without re-authenticating during a failover from the primary to the secondary cluster. DR Replication is designed for this exact scenario. It replicates all data, including tokens and leases, from the primary cluster to the secondary cluster. When the secondary is promoted to primary during a failover, the existing tokens remain valid, allowing applications to seamlessly continue operations without re-authentication.

Performance Replication, while improving scalability and performance by replicating data across clusters, manages its own tokens and leases on each secondary cluster. Tokens from the primary are not replicated, so a failover would invalidate existing tokens, requiring applications to reauthenticate” failing the requirement. Integrated Storage is a storage backend, not a replication type, and doesn't address failover behavior. The Vault Secrets Operator is a Kubernetes tool for secret management, unrelated to cluster replication. According to Vault's DR Replication documentation, it ensures continuity of token validity, making it the correct choice.

Reference:

Disaster Recovery Replication Tutorial

Performance Replication Tutorial

Vault Replication Overview

QUESTION 174

You have enabled the Transit secrets engine and want to start encrypting data to store in Azure Blob storage. What is the next step that needs to be completed before you can encrypt data? (Select two)

- A. Export the encryption key and upload it to the application server
- B. Enable the Transit secrets engine API
- C. Create an encryption key for the application to use
- D. Write a policy that permits the application to use the encryption key

Answer: C, D

Explanation:

Comprehensive and Detailed In-Depth

The Transit secrets engine in Vault is designed for encryption as a service, allowing applications to encrypt data without managing keys locally. After enabling the engine, two critical steps are required before encryption can begin: creating an encryption key and defining a policy to allow its use.

Option C: You must create an encryption key using a command like `vault write -f`

transit/keys/<key_name>. This key is stored in Vault and used for encryption/decryption operations. Without it, no encryption can occur, as the Transit engine relies on named keys to perform cryptographic operations.

Option D: A policy must be written to grant the application permissions to use the key, such as path "transit/encrypt/<key_name>" { capabilities = ["update"] } and path "transit/decrypt/<key_name>" { capabilities = ["update"] }. Vault's access control ensures that only authorized entities can perform encryption, making this step essential.

Option A (exporting the key) contradicts Vault's security model, as keys should remain in Vault, not be exported to application servers. Option B (enabling the Transit API) is unnecessary, as enabling the engine automatically exposes its API endpoints. The official Transit documentation confirms that key creation and policy configuration are the next steps post-enablement.

Reference:

Transit Secrets Engine Tutorial

Transit Secrets Engine Docs

QUESTION 175

Your team uses the Transit secrets engine to encrypt all data before writing it to a MySQL database server. During testing, you manually retrieve ciphertext from the database and decrypt it to ensure the data can be read. After decrypting the data, you are worried something is wrong because the plaintext data isn't legible. Why can you not read the original plaintext data after decrypting the ciphertext?

```
$ vault write transit/decrypt/krausen-key ciphertext=vault:v1:8SDd3WHDOjf7mq69C.....
```

Key Value

--- -----

```
plaintext Zml2ZSBzdGFyIHByYWN0aWNlIGV4YW1zIGJ5IGJyeWFuIGtyYXVzZW4=
```

- A. The incorrect key was selected when decrypting the ciphertext. Use the correct key to successfully read the data
- B. The incorrect key version was used to decrypt the data. Update the ciphertext and change the v1 to v3 to use the latest key version
- C. The plaintext is Base64 encoded. Decode the plaintext to see the original data
- D. The data was also encrypted on the database. Therefore Vault cannot decrypt the original data

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

When using the Transit secrets engine, Vault encrypts data and returns ciphertext (e.g., vault:v1:<ciphertext>). Upon decryption (e.g., vault write transit/decrypt/<key_name>

ciphertext=<value>), Vault returns the plaintext as a Base64-encoded string. This is because the Transit engine supports arbitrary data, including binary files (e.g., PDFs, images), and Base64 encoding ensures safe transport within JSON payloads. If the decrypted output (e.g., Zml2ZSBzdGFyIHByYWN0aWNlIGV4YW1zIGJ5IGJyeWFuIGtyYXVzZW4=) isn't legible, it's not an error—it's Base64 encoded. Decoding it (e.g., using a Base64 decoder) reveals the original plaintext (e.g., "five star practice exams by bryan krausen").

Option A (incorrect key) would cause a decryption failure, not illegible plaintext. Option B (incorrect key version) is irrelevant, as Vault automatically uses the correct version based on the ciphertext's vault:v# prefix, and changing it manually wouldn't produce Base64 output. Option D (database encryption) isn't indicated in the scenario and would also cause a failure, not Base64 output. The Transit documentation explicitly states that plaintext is returned Base64-encoded, requiring the user to decode it.

Reference:

Transit Secrets Engine Docs

Transit Usage Section

QUESTION 176

You are working on a new project and need to retrieve a secret from Vault. You log into the Vault UI and browse to the path where the secret is stored. Based on the screenshot below, what is true about the secrets stored in this path? (Select four)

The screenshot shows the Vault UI interface. On the left is a dark sidebar with navigation links: Vault, Dashboard, Secrets Engines (highlighted), Secrets Sync, Access, Policies, Tools, Monitoring, Replication, Raft Storage, and Client Count. The main content area shows the path 'secrets / developers / apps / eCommerce' and the title 'apps/eCommerce'. Below the title are tabs for 'Secret', 'Metadata', 'Paths', and 'Version History'. The 'Secret' tab is active, showing a toggle for 'JSON' and buttons for 'Delete', 'Destroy', 'Copy', 'Version 4', and 'Create new version'. A table lists secrets with columns 'Key' and 'Value':

Key	Value
connection_string	
database	
host	
username	

A. The secrets are stored in a KV v1 secrets engine

- B. The user does not have permission to delete the secret
- C. The secrets are stored in a KV v2 secrets engine
- D. The secrets engine is mounted at the path developers/
- E. There are four previous versions of the secret
- F. The user has additional permissions on the path beyond just list and read

Answer: C, D, E, F

Explanation:

Comprehensive and Detailed In-Depth

Assuming the screenshot shows a KV secrets engine at developers/ with version 5 of a secret and options for delete/create:

C: KV v2 is indicated by versioning (version 5 and four previous versions). KV v1 doesn't support versioning, per the KV v2 documentation.

D: The path developers/ is the mount point, as secrets are accessed under this path, consistent with Vault's mount structure.

E: Four previous versions (v1“v4) exist if v5 is current, a feature of KV v2's versioning.

F: Delete and create options in the UI imply permissions beyond list and read, such as delete and create or update, per Vault's UI behavior reflecting policy capabilities.

A: KV v1 lacks versioning, so this is incorrect.

B: The delete option's presence suggests permission exists, though UI visibility isn't a definitive policy check”still, it's typically indicative.

Reference:

KV Secrets Engine v2 Docs

Vault UI Tutorial

QUESTION 177

You are planning to deploy a new Vault cluster for your organization and notice that Vault supports a wide variety of storage backends. You need high availability since you will have multiple applications relying on the Vault service. When building your cluster, can you choose any of the available storage backends?

- A. Yes, because all backends provide similar functionality
- B. No, because not all storage backends provide similar functionality

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

Vault supports various storage backends (e.g., Consul, Raft, DynamoDB), but not all provide high availability (HA). HA ensures that Vault remains operational across multiple nodes, with automatic failover if a node fails—an essential feature for applications relying on Vault. The Vault documentation lists each backend's capabilities, noting that only certain ones (e.g., Consul, Raft Integrated Storage, etcd) support HA through features like leader election and data replication. Others, like Filesystem or MySQL, don't support HA natively, making them unsuitable for this requirement. Thus, you cannot choose any backend arbitrarily; the choice must align with HA needs, disproving option A and confirming option B.

Reference:

[Storage Backends Overview](#)

[HA Considerations](#)

QUESTION 178

Before data is written to the storage backend, the data is encrypted by which Vault feature?

- A. TLS certificate
- B. Cryptographic barrier
- C. Unseal keys
- D. Transit secrets engine

Answer: B

Explanation:

[Comprehensive and Detailed In-Depth](#)

Vault's architecture includes a cryptographic barrier that encrypts all data before it's written to the storage backend. This ensures that the backend (e.g., Consul, Filesystem) only stores encrypted data, enhancing security even if the backend is compromised. The barrier uses a master key (split into unseal keys via Shamir's Secret Sharing) to encrypt a keyring, which in turn encrypts the data. TLS certificates secure network communication, not storage encryption. Unseal keys unlock the master key, not encrypt data directly. The Transit engine is for application-level encryption, not storage backend protection. The Vault architecture does confirm the cryptographic barrier's role.

Reference:

[Vault Architecture Overview](#)

[Data Encryption](#)

QUESTION 179

Your Azure Subscription ID is stored in Vault and you need to retrieve it via Vault API for an automated job. The Subscription ID is stored at secret/cloud/azure/subscription. The secret is stored on a KV Version 2 secrets engine. What curl command below would successfully retrieve the latest

version of the secret?

- A. curl https://vault.krausen.com:8200/v1/secret/data/cloud/azure/subscription
- B. curl --header "X-Vault-Token: hvs.CbzCNJCVWt63jyzyaJakgDwz"
https://vault.krausen.com:8200/v1/secret/cloud/azure/subscription
- C. curl --header "X-Vault-Token: hvs.CbzCNJCVWt63jyzyaJakgDwz"
https://vault.krausen.com:8200/v1/secret/data/cloud/azure/subscription
- D. curl --header "X-Vault-Token: hvs.CbzCNJCVWt63jyzyaJakgDwz"
https://vault.krausen.com:8200/secret/data/cloud/azure/subscription/latest

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

For a KV v2 secrets engine, the API path to retrieve a secret's data is /v1/<mount>/data/<path>. Here, the mount is secret/, and the path is cloud/azure/subscription, making the correct endpoint /v1/secret/data/cloud/azure/subscription. Authentication requires the X-Vault-Token header with a valid token. Option C matches this exactly and retrieves the latest version by default, as per KV v2 API behavior. Option A lacks the token. Option B omits the /data/ segment, invalid for KV v2. Option D adds /latest, which isn't a valid KV v2 endpoint. The KV v2 API docs confirm this structure.

Reference:

KV v2 API Docs

Vault API Overview

QUESTION 180

All Vault instances, or clusters, include two built-in policies that are created automatically. Choose the two policies below and the correct information regarding each policy. (Select two)

- A. The root policy is created automatically. This policy provides superuser privileges and cannot be deleted
- B. The admin policy is created automatically. It provides administrative permissions but can be deleted if needed
- C. The default policy is created automatically. This policy can be modified but not deleted
- D. The default policy is created automatically. This policy cannot be modified but it can be deleted

Answer: A, C

Explanation:

Comprehensive and Detailed In-Depth

Vault automatically creates two built-in policies: root and default.

A: The root policy is created at initialization, granting superuser privileges (full access to all paths and operations). It's attached to root tokens and cannot be deleted or modified, per the policies documentation.

C: The default policy is also created automatically, providing basic permissions (e.g., token management). It's attached to all non-root tokens by default, can be modified, but cannot be deleted, as stated in the docs.

B: No admin policy is automatically created; administrative policies must be defined manually.

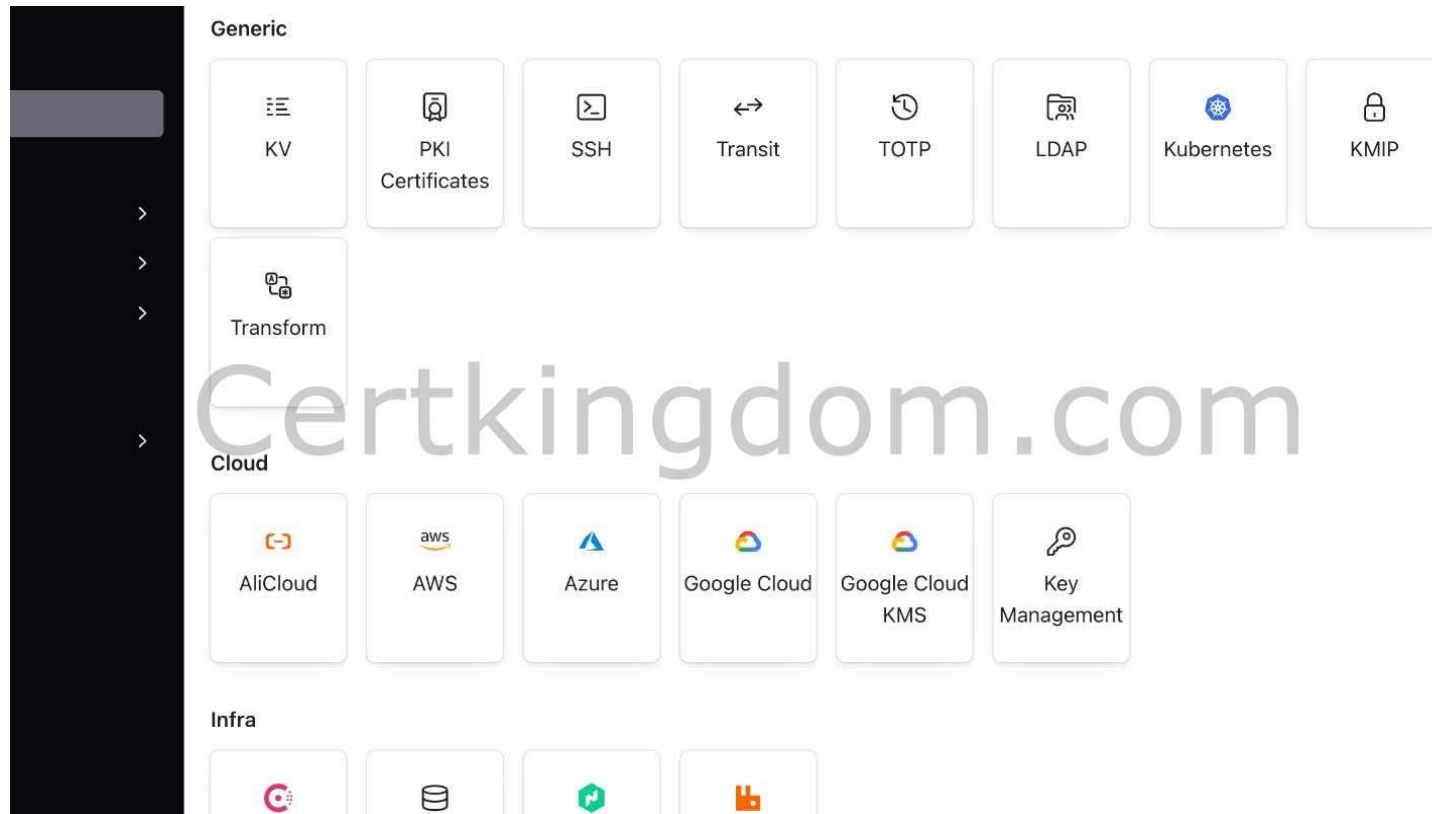
D: The default policy can be modified, contradicting this option.

Reference:

Built-in Policies

QUESTION 181

You have logged into the Vault UI and see this screen. What Vault component is being enabled in the screenshot below?



A. Storage Backends

B. Secrets Engine

C. Auth Methods
D. Audit Devices

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

In the Vault UI, the `Secrets` tab lists enabled secrets engines and includes an `Enable new engine` option to add a new one. Secrets engines manage secrets (e.g., KV, Transit), and enabling one configures it at a specific path. Storage backends (e.g., Raft) are set in the config file, not the UI. Auth methods (e.g., LDAP) are enabled under the `Access` tab. Audit devices (e.g., file logging) are under `Tools`. The screenshot context and UI workflow align with enabling a secrets engine, per the getting-started tutorial.

Reference:

Secrets Engines Tutorial

Secrets Engines Docs

QUESTION 182

An Active Directory admin created a service account for an internal application. You want to store these credentials in Vault, allowing a CI/CD pipeline to read and configure the application with them during provisioning. Vault should maintain the last 3 versions of this secret. Which Vault secrets engine should you use?

- A. The KV secrets engine
- B. The LDAP secrets engine
- C. The Identity secrets engine
- D. The KV v2 secrets engine

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

The requirement is to store static credentials (from Active Directory) in Vault with versioning (last 3 versions) for a CI/CD pipeline. The KV v2 secrets engine is designed for this: it stores arbitrary keyvalue pairs and supports versioning, allowing configuration of a maximum version count (e.g., `vault kv metadata put -max-versions=3 kv/path`). KV v1 (option A) lacks versioning. The LDAP engine (B) is for dynamic LDAP credentials, not static storage. The Identity engine (C) manages identities, not secrets. KV v2's versioning capability meets all needs, per its documentation.

Reference:

QUESTION 183

Sara uses the Vault CLI for administrative tasks on the production cluster. However, she encounters permission-denied errors when making changes and needs to check which policies are attached to her token to view and adjust permissions. What command can she run on the Vault node to see the attached policies?

- A. vault operator diagnose
- B. vault policy list
- C. vault token capabilities
- D. vault token lookup

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

To view policies attached to her token, Sara needs vault token lookup. This command displays token details, including the policies field (e.g., [default, training]), revealing what permissions she has. vault operator diagnose troubleshoots server issues, not tokens. vault policy list lists all policies in Vault, not those tied to a specific token. vault token capabilities checks capabilities on a path, not policy attachment. The token lookup command, per Vault docs, is the correct tool for inspecting token metadata like policies.

Reference:

[Token Lookup Docs](#)

[Token Concepts](#)

QUESTION 184

What of the following features are true about batch tokens in Vault? (Select two)

- A. Batch tokens are not persisted (written) to storage
- B. Batch tokens can be renewed
- C. Batch tokens are valid across all clusters when using Vault Enterprise replication
- D. Batch tokens can create child tokens

Answer: A, C

Explanation:

Comprehensive and Detailed In-Depth

Batch tokens are lightweight tokens in Vault, designed for high-performance use cases.

A: They are not persisted to storage, reducing backend load, as confirmed by the batch token tutorial.

C: In Vault Enterprise with DR Replication, batch tokens are replicated and remain valid across clusters when the secondary is promoted, per replication docs.

B: Batch tokens cannot be renewed; they have a fixed TTL, per the service vs. batch token comparison.

D: They cannot create child tokens, lacking features of service tokens.

Reference:

[Batch Tokens Tutorial](#)

[Tokens Docs](#)

QUESTION 185

To protect the sensitive data stored in Vault, what key is used to encrypt the data before it is written to the storage backend?

A. Recovery key

B. Encryption key

C. Unseal key

D. Root key

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

Vault encrypts all data before writing it to the storage backend using an encryption key within its cryptographic barrier. This key, stored in a keyring, is itself encrypted by the master key (split into unseal keys). The recovery key (A) is for emergency recovery, not data encryption. Unseal keys (C) unlock the master key, not encrypt data directly. The root key (D) isn't a term used in Vault's encryption flow; the master key is the closest analog, but it protects the encryption key, not the data itself. The architecture docs clarify the encryption key's role.

Reference:

[Vault Architecture](#)

[Keyring Details](#)

QUESTION 186

You have a new team member on the Vault operations team. Their first task is to rotate the encryption key in Vault as part of the organization's security policy. However, when they log in, they get an access denied error when attempting to rotate the key. The policy being used is below. Why

can't the user rotate the encryption key?

```
path "auth/*" {  
  capabilities = ["create", "read", "update", "delete", "list"]  
}  
path "sys/rotate" {  
  capabilities = ["read", "update"]  
}
```

- A. The policy requires sudo privileges since it is a root-protected path
- B. The policy doesn't include create privileges so a new encryption key can't be created
- C. The policy should include sys/rotate/<name of key> as part of the path
- D. The encryption key has a minimum TTL, therefore the key cannot be rotated until that time expires

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

Rotating the encryption key in Vault is done via the sys/rotate endpoint, a root-protected path requiring sudo capability in addition to update. The provided policy grants read and update on sys/rotate, but lacks sudo, resulting in an access denied error. Option B (create) isn't required for rotation, per the API docs. Option C is incorrect; sys/rotate is the fixed endpoint, not key-specific. Option D (TTL) isn't a Vault restriction for key rotation. The policies tutorial confirms sudo is needed for root-protected paths like this.

Reference:

Root-Protected Endpoints

Sys/Rotate API

QUESTION 187

Why are short-lived, dynamic secrets in Vault more secure than long-lived, static credentials?

- A. They provide better performance by caching credentials for longer durations
- B. They are created on-demand and expire after a short period, minimizing the risk of credential leakage
- C. They eliminate the need for authentication, allowing seamless access to Vault-managed systems
- D. They automatically rotate on a set schedule, reducing the need for manual intervention

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

Short-lived, dynamic secrets in Vault enhance security by being generated on-demand and expiring after a short, configurable time-to-live (TTL). This reduces the window of opportunity for credential leakage or misuse. Unlike long-lived, static credentials, which persist indefinitely and increase exposure risk if compromised, dynamic secrets are ephemeral—once they expire, they're automatically revoked by Vault, rendering them useless to attackers. For example, a database credential might last 5 minutes, limiting its attack surface compared to a static password stored indefinitely.

Option A (performance via caching) is unrelated to security and inaccurate, as dynamic secrets aren't cached longer. Option C (eliminating authentication) is false; authentication is still required to obtain dynamic secrets. Option D (automatic rotation) applies to some dynamic secrets (e.g., database roles), but the core security benefit is their short lifespan, not just rotation. Vault's documentation on dynamic secrets emphasizes their ephemerality as the key security advantage.

Reference:

Dynamic Secrets Tutorial

Dynamic Secrets Concepts

QUESTION 188

True or False? After rotating a transit encryption key, all data encrypted with the previous version must be rewrapped or re-encrypted with the new key.

A. True

B. False

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

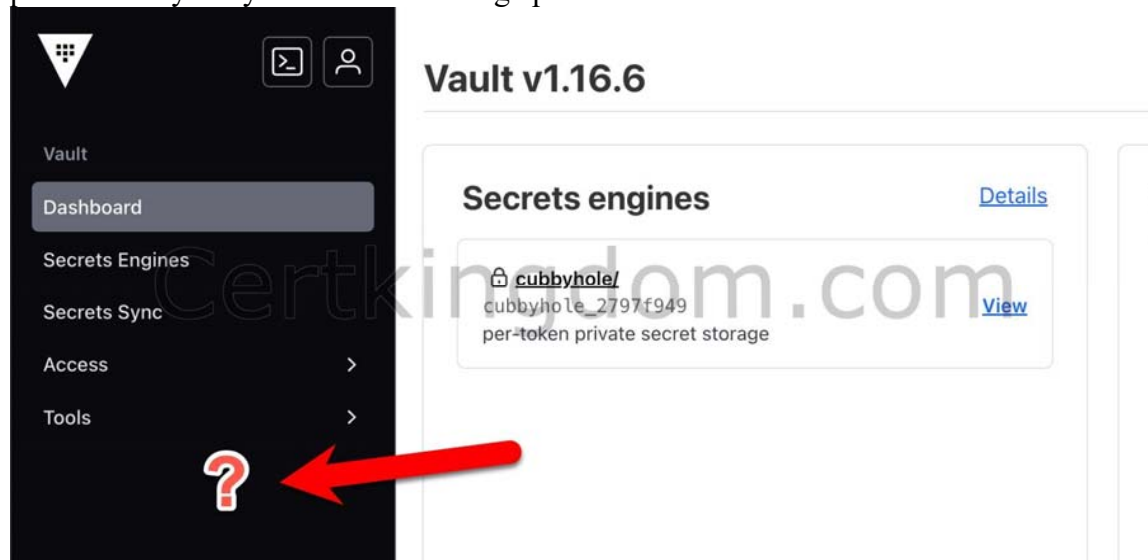
False. When a transit encryption key is rotated in Vault (e.g., via `vault write -f transit/keys/<key_name>/rotate`), the new key version becomes the default for future encryptions, but data encrypted with previous versions remains decryptable without rewrapping or re-encryption. Vault maintains a keyring with all versions, and the ciphertext prefix (e.g., `vault:v1:`) indicates which version was used, allowing automatic decryption with the corresponding key. This seamless handling simplifies key management and avoids mandatory data re-encryption post-rotation. Only if you set a `min_decryption_version` to archive older keys would rewrapping be needed, but that's optional, not default behavior.

Option A is incorrect per Vault's Transit documentation, which notes that old data can still be decrypted without immediate action after rotation.

Reference:

QUESTION 189

Your supervisor has requested that you log into Vault and update a policy for one of the development teams. You successfully authenticated to Vault via OIDC but do not see a way to manage the Vault policies. Why are you unable to manage policies in the Vault UI?



- A. Policies are only available on Vault Enterprise
- B. The Vault node is sealed, and therefore you cannot manage policies
- C. Policies cannot be managed in the UI, only the CLI and API
- D. The policy associated with your login does not permit access to manage policies

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

In the Vault UI, the Policies tab is visible only if your token's policy grants access to policy management endpoints (e.g., sys/policy in Vault OSS or sys/policies/acl in Enterprise). If the tab is missing after OIDC authentication, it's because your policy lacks permissions like read and list on these paths, preventing UI navigation to policy management. For example, a minimal policy to view policies in OSS is path "sys/policy/*" { capabilities = ["read", "list"] }. Without this, the UI hides the tab, aligning with Vault's least-privilege model.

Option A is false; policies exist in both OSS and Enterprise, with UI support in both. Option B is incorrect; a sealed Vault prevents login entirely, not just policy access. Option C is wrong; the UI does

support policy management when permitted. Vault's policy docs confirm that UI visibility depends on policy permissions.

Reference:

Policy Management OSS

Policy Management Enterprise

QUESTION 190

A Fintech company is using Vault to store its static long-lived credentials so automated processes can quickly retrieve secrets. A user needs to add a new static secret for a new automated job. What CLI commands can be used to store a new static credential? (Select two)

- A. `vault kv put kv/training/certification/vault @secrets.txt`
- B. `vault kv write kv/training/certification/vault key=username value=bryan`
- C. `vault kv create kv/training/certification/vault @secrets.txt`
- D. `vault kv put -mount=secret creds passcode=my-long-passcode`

Answer: A, D

Explanation:

Comprehensive and Detailed In-Depth

To store static credentials in Vault's KV secrets engine via CLI, the `vault kv put` command is used.

A: `vault kv put kv/training/certification/vault @secrets.txt` writes data from a file (`secrets.txt`) to the path `kv/training/certification/vault`. The `@` syntax reads key-value pairs from the file, a valid method per the KV docs.

D: `vault kv put -mount=secret creds passcode=my-long-passcode` specifies the mount (`secret/`) and stores `passcode=my-long-passcode` at `secret/creds`, a correct inline syntax.

B: `vault kv write` isn't a valid command; `put` is the correct verb. The `key=value` syntax is right but needs `put`.

C: `vault kv create` isn't a command; `put` is used to create or update secrets.

The KV CLI docs confirm `vault kv put` as the standard method, supporting both file input and inline key-value pairs.

Reference:

KV Put Command

KV Secrets Engine Docs

QUESTION 191

Your co-worker has asked you to perform certain operations in Vault and has provided you with a token accessor (not the token itself). What Vault operations would you be allowed to perform using only the provided accessor? (Select three)

- A. Renew the token to extend the TTL
- B. Revoke the token in Vault to make it invalid
- C. Create child tokens associated with the same policies as the related token
- D. Lookup properties of the token, such as the TTL, policies, and metadata

Answer: A, B, D

Explanation:

Comprehensive and Detailed In-Depth

A token accessor is a reference to a token, not the token itself, and supports limited operations:

A: vault token renew -accessor <accessor> extends the token's TTL if renewable, per the token docs.

B: vault token revoke -accessor <accessor> revokes the token, making it invalid, a supported accessor action.

D: vault token lookup -accessor <accessor> displays token properties (e.g., TTL, policies), a key accessor use case.

C: Creating child tokens requires the parent token, not just its accessor, as it involves authentication and policy inheritance, which accessors can't perform.

Accessors can't authenticate to Vault for secret access; they're for management tasks like these, per the tokens documentation.

Reference:

Token Accessors

Token Commands

QUESTION 192

You are using Vault CLI and enable the database secrets engine on the default path of database/.

However, the DevOps team wants to enable another database secrets engine for testing but receives an error stating the path is already in use. How can you enable a second database secrets engine using the CLI?

- A. vault secrets enable database database2/
- B. vault secrets enable -force database
- C. vault secrets enable -path=database2 database
- D. vault secrets enable database2/

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

Vault mounts secrets engines at unique paths, and only one engine can occupy a given path (e.g., database/). To enable a second database secrets engine, you must specify a different path using the -path flag: vault secrets enable -path=database2 database mounts a new instance at database2/. The type (database) defines the engine, and -path customizes its location, avoiding conflicts.

A: Incorrect syntax; lacks -path and misplaces database2/.

B: -force doesn't create a new path; it overwrites an existing engine, which isn't the goal.

D: Omits -path and engine type, making it invalid.

The secrets engine tutorial confirms -path is required for multiple instances of the same engine type.

Reference:

Secrets Engines Tutorial

Secrets Enable Command

QUESTION 193

To secure your applications, your organization uses certificates generated by a public C

A. However, this strategy has proven expensive and you have to revoke certificates even though they have additional time left. What Vault plugin can be used to quickly generate X.509 certificates to secure your internal applications?

A. Identity secrets engine

B. PKI secrets engine

C. SSH secrets engine

D. Transit secrets engine

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

The PKI secrets engine in Vault generates dynamic X.509 certificates, acting as a certificate authority (CA) or intermediate C

A. It allows quick, cost-effective certificate creation for internal applications, with configurable TTLs and revocation capabilities, avoiding reliance on expensive public CAs. For example, vault write pki/issue/<role> generates a certificate instantly. The Identity engine (A) manages identities, not certificates. The SSH engine (C) handles SSH credentials, not X.509. The Transit engine (D) is for encryption, not certificate generation. The PKI docs highlight its suitability for this use case.

Reference:

PKI Secrets Engine Docs

PKI Tutorial

QUESTION 194

You have successfully authenticated using the Kubernetes auth method, and Vault has provided a token. What HTTP header can be used to specify your token when you request dynamic credentials? (Select two)

- A. X-Vault-Token: <token>
- B. Token: <token>
- C. Authentication: <token>
- D. Authorization: Bearer <token>

Answer: A, D

Explanation:

Comprehensive and Detailed In-Depth

After authenticating with the Kubernetes auth method, Vault returns a token that must be included in subsequent API requests to retrieve dynamic credentials. The Vault documentation specifies two valid HTTP headers for this purpose:

"Once authenticated, most Vault operations require a client token to be set either via the X-Vault-Token header or via the Authorization header using the Bearer type. For example:

X-Vault-Token: <token>

Authorization: Bearer <token>"

” Vault API Documentation: Authentication

A: X-Vault-Token: <token> is the primary Vault-specific header for token authentication:

"The X-Vault-Token header is used to specify the token when requesting dynamic credentials from Vault. This header is commonly used to authenticate and authorize requests to Vault services."

” Vault API Documentation

D: Authorization: Bearer <token> is a standard HTTP authentication header supported by Vault:

"The Authorization header with the Bearer token format is another common way to specify the token when requesting dynamic credentials from Vault. This header is widely used for authentication purposes in HTTP requests."

” Vault API Documentation

B: Token: <token> is not a recognized Vault header.

C: Authentication: <token> is not a standard or supported header in Vault; the correct header is Authorization.

These headers ensure the token is passed securely to Vault for authorizing credential requests.

Reference:

Vault API Documentation: Authentication

Vault Tokens

QUESTION 195

Which of the following capabilities can be used when writing a Vault policy? (Select four)

- A. list
- B. deny
- C. apply
- D. root
- E. create
- F. write

Answer: A, B, E, F

Explanation:

Comprehensive and Detailed In-Depth

Vault policies define access control using specific capabilities. The Vault documentation lists the valid capabilities:

"When creating a policy, only the following capabilities are available in Vault:

create
read
update
delete
list
sudo
deny"

” Vault Policies: Capabilities

A: list is valid:

"The list capability enables the user to view a list of available resources or entities within Vault."

” Vault Policies: Capabilities

B: deny is valid:

"The deny capability is used to explicitly deny access to specific resources or operations within Vault."

” Vault Policies: Capabilities

E: create is valid:

"The create capability allows the user to create new policies, roles, tokens, and other entities within Vault."

” Vault Policies: Capabilities

F: write is a common shorthand for update in Vault's context and is valid:

"The update capability (often referred to as write in CLI contexts) allows the user to modify or update existing resources or entities within Vault."

” Vault Policies: Capabilities

Note: While write isn't explicitly listed, it's synonymous with update in practice, as confirmed by CLI usage and community convention.

C: apply is not a Vault policy capability.

D: root is not a capability; it's a policy name for superuser access.

Reference:

Vault Policies: Capabilities

QUESTION 196

You are configuring your application to retrieve a new PKI certificate upon provisioning. The Vault admins have given you an AppRole role-id and secret-id to inject into the CI/CD pipeline job that provisions your app. The application uses the credentials to successfully authenticate to Vault using the API. Which of the following is true about the step next required after authenticating to Vault?

- A. The client token needs to be retrieved from the API response before requesting the new PKI certificate
- B. The initial API response should include the new PKI certificate and no further action is required
- C. The app still needs to use the role-id and secret-id to request the new PKI certificate via API
- D. Now that the app is authenticated, it can simply make another API request for the PKI certificate

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

After authenticating with AppRole using the role-id and secret-id via the API (e.g., POST /v1/auth/approle/login), Vault returns a response containing a client_token. This token must be extracted for subsequent requests, such as retrieving a PKI certificate. The Vault documentation states:

"When you use the Vault API to authenticate, the Vault API response will include a client_token that is tied to a specific policy. Once you receive that response, it is up to the user (or application) to parse that response and retrieve the token. Once the token is retrieved, a second API request needs to be sent to Vault to request the new PKI certificate."

” Vault API: AppRole

A: Correct. The client_token from the response (e.g., under .auth.client_token) is required for the next request (e.g., POST /v1/pki/issue/<role>):

"The client token is necessary to make subsequent requests to Vault, including requesting the new PKI certificate."

” Vault API Documentation

B: Incorrect. Authentication doesn't return a PKI certificate; a separate request is needed.

C: Incorrect. The role-id and secret-id are for authentication, not certificate retrieval:
"Authentication and interaction with a secrets engine are separate actions."

” Vault API: AppRole

D: Partially true but vague; it omits the critical step of retrieving the token first.

Reference:

Vault API: AppRole

Vault PKI Secrets Engine

QUESTION 197

Vault is configured with the oidc auth method and you need to log in using the CLI. What command would you use to authenticate so you can make configuration changes to Vault?

A. vault login -method=oidc username=bryan

B. vault auth oidc

C. vault login auth/oidc/users/bryan

D. vault login username=bryan

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

To authenticate via the OIDC auth method using the CLI, the vault login command with the -method flag is used. The Vault documentation states:

"To authenticate using the CLI, you could use the command vault login and specify the auth method you wish to use by using the -method flag. For example, if you wanted to authenticate using OIDC, you could use vault login -method=oidc [options]."

” Vault Commands: login

A: vault login -method=oidc username=bryan is correct, specifying the OIDC method and username:

"The correct command to authenticate using the oidc auth method in Vault is vault login -method=oidc username=bryan."

” Vault Auth: OIDC

B: vault auth oidc is invalid; auth is not a login command.

C: vault login auth/oidc/users/bryan is incorrect syntax; it mimics an API path, not a CLI command.

D: vault login username=bryan lacks the method specification, defaulting to token auth.

Reference:

Vault Commands: login

Vault Auth: OIDC

QUESTION 198

Which core component of Vault can store, generate, or encrypt data for organizations?

- A. auth method
- B. storage backend
- C. secrets engine
- D. audit device

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

Secrets engines are Vault's core components for managing data. The Vault documentation states:

"Secrets engines are components that store, generate, or encrypt data. Secrets engines are incredibly flexible, so it is easiest to think about them in terms of their function. Secrets engines are provided some set of data, they take some action on that data, and they return a result."

” Vault Secrets Engines

C: Correct. Secrets engines (e.g., KV, Transit) handle storing, generating, or encrypting data:

"The secrets engine is a core component of Vault that is responsible for storing, generating, and encrypting data for organizations."

” Vault Secrets Engines

A: Auth methods authenticate, not manage data.

B: Storage backends persist encrypted data, not generate or encrypt it directly.

D: Audit devices log actions, not handle data.

Reference:

Vault Secrets Engines

QUESTION 199

Frapps, Inc. is a coffee startup specializing in frozen caffeinated beverages. Their new customer loyalty web app uses Vault to store sensitive information, choosing Integrated Storage for its benefits. Select the benefits the organization would see by using Integrated Storage over other storage backends (Select four)

- A. Eliminates network communication between hosts, requiring no open ports between hosts
- B. Uses the SERF gossip protocol to enable communication between cluster nodes
- C. Eliminates the requirement to deploy and manage a separate platform for storing encrypted data
- D. Simplified troubleshooting since Integrated Storage is a built-in solution
- E. Reduces operational overhead since all configuration is within Vault itself
- F. Immediate access to storage since the data is stored locally on disk

Answer: C, D, E, F

Explanation:

Comprehensive and Detailed In-Depth

Integrated Storage (Raft) offers several benefits over external storage backends. The Vault documentation states:

"Introduced in Vault 1.4, Integrated Storage is a built-in solution that provides a highly available, durable storage backend without relying on any external systems. All Vault data is stored locally on each node, and replicated to all other nodes in the cluster for high availability. It also reduces complexity since all configuration is done within Vault."

” Vault Configuration: Raft Storage

C: Correct.

"Eliminates the requirement to deploy and manage a separate platform for storing encrypted data."

” Vault Configuration: Raft Storage

D: Correct.

"Troubleshooting is simplified when using Integrated Storage because it is a built-in solution within Vault."

” Vault Configuration: Raft Storage

E: Correct.

"Reduces operational overhead by keeping all configuration and data storage within Vault itself."

” Vault Configuration: Raft Storage

F: Correct.

"Integrated Storage provides immediate access to stored data since it is stored locally on disk within Vault."

” Vault Configuration: Raft Storage

A: Incorrect; Raft requires port 8201 for replication:

"The Vault cluster nodes still need to communicate over port 8201 for replication and RPC forwarding."

” Vault Configuration: Raft Storage

B: Incorrect; Raft uses the RAFT protocol, not SERF:

"Integrated Storage uses the same underlying consensus protocol (RAFT) as Consul to handle cluster leadership and log management."

” Vault Configuration: Raft Storage

Reference:

Vault Configuration: Raft Storage

QUESTION 200

You are using Vault to generate dynamic credentials for a Microsoft SQL server to perform queries for

a month-end report. The report seems to be taking much longer than expected due to degradation on the underlying server, and you are afraid that Vault might automatically revoke the credentials. How can you extend the time the credentials are valid to ensure your month-end query is successful?

- A. Renew the lease
- B. Generate a new lease
- C. Create a new role within the secrets engine for the database
- D. Revoke the lease

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

Dynamic credentials have a lease with a TTL, after which Vault revokes them. To extend their validity, you renew the lease. The Vault documentation states:

"If a lease has been created in Vault, it has an associated TTL in which it will expire and be revoked. If the lease needs to be extended for some reason, you can use the command `vault lease renew <lease_id>` to extend the TTL of the lease so it will not expire at its original TTL and will be extended by the time specified in seconds from the current time the lease renewal was issued."

” Vault Commands: lease renew

A: Correct. Renewing the lease (e.g., `vault lease renew <lease_id>`) extends the TTL:

"Renewing the lease of the dynamic credentials in Vault allows you to extend the validity period without having to generate new credentials."

” Vault Commands: lease renew

B: Generating a new lease creates new credentials, disrupting the query.

C: Creating a new role doesn't extend existing credentials' TTL.

D: Revoking the lease terminates the credentials, halting the query.

Reference:

Vault Commands: lease renew

Vault Concepts: Leases

QUESTION 201

Your organization is integrating its legacy application with Vault to improve its security. However, you have discovered that the application has issues when the token changes for authentication during testing. What type of token could be used to help alleviate this issue without compromising security?

- A. Periodic Service Token
- B. Root Token
- C. Orphan Service Token

D. Batch Token

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

Periodic Service Tokens allow renewal without changing the token, addressing the application's issue. The Vault documentation states:

"In some cases, having a token be revoked would be problematic -- for instance, if a long-running service needs to maintain its SQL connection pool over a long period of time. In this scenario, a periodic token can be used. The idea behind periodic tokens is that it is easy for systems and services to perform an action relatively frequently -- for instance, every two hours, or even every five minutes. Therefore, as long as a system is actively renewing this token -- in other words, as long as the system is alive -- the system is allowed to keep using the token and any associated leases."

” Vault Concepts: Tokens

A: Correct. Periodic tokens maintain stability with renewal:

"A Periodic Service Token is a type of token in Vault that can be renewed periodically without the need for the application to re-authenticate every time the token changes."

” Vault Concepts: Tokens

B: Root tokens are insecure for applications due to unlimited access:

"Root tokens should not be used for application authentication due to their high level of access and security risks."

” Vault Concepts: Tokens

C: Orphan tokens don't support periodic renewal inherently.

D: Batch tokens cannot be renewed:

"Batch tokens cannot be renewed."

” Vault Tutorials: Batch Tokens

Reference:

Vault Concepts: Tokens

QUESTION 202

How does the instance updates feature work when using the Vault Secrets Operator?

- A. By monitoring the Vault audit logs to watch for changes to the target path
- B. By constantly validating the current secret stored in Vault
- C. By continuously launching an init container to check for updates
- D. By subscribing to event notifications from Vault

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

The Vault Secrets Operator (VSO) uses event notifications for instant updates. The Vault documentation states:

"Vault Secrets Operator (VSO) supports instant updates for VaultStaticSecrets by subscribing to event notifications from Vault. This allows the Vault Secrets Operator to receive real-time updates and changes to secrets, ensuring that the application always has access to the latest secret values without the need for manual intervention."

” Vault Secrets Operator: Instant Updates

D: Correct. Subscribing to Vault's event notifications enables real-time updates.

A: Audit logs track actions, not real-time updates.

B: Constant validation isn't the mechanism; it's notification-driven.

C: Continuous init containers are inefficient and not used by VSO.

Reference:

Vault Secrets Operator: Instant Updates

QUESTION 203

Your organization uses a CI/CD pipeline to deploy its applications on Azure. During testing, you generate new credentials to validate Vault can create new credentials. The result of this command is below:

text

CollapseWrapCopy

```
$ vault read azure/creds/bryan-krausen
```

Key Value

```
lease_id azure/creds/bryan-krauseneed0373-ca92-99b6-b914-779b7bb0e1d9
```

```
lease_duration 60m
```

```
lease_renewable true
```

```
client_id 532bf678-ee4e-6be1-116b-4e4221e445dd
```

```
client_secret be60395b-4e6b-2b7e-a4b3-c449a5c00973
```

What commands can be used to revoke this secret after you have finished testing? (Select three)

A. vault lease revoke azure/

B. vault lease revoke -prefix azure/

C. vault lease revoke azure/creds/bryan-krauseneed0373-ca92-99b6-b914-779b7bb0e1d9

D. vault lease revoke azure/creds/bryan-krausen

E. vault lease revoke -prefix azure/creds/bryan-krausen

Answer: B, C, E

Explanation:

Comprehensive and Detailed In-Depth

Dynamic credentials are tracked by leases, revocable via vault lease revoke. The Vault documentation states:

"The vault lease revoke command is used to revoke a lease/secret created by a Vault secrets engine. Each lease that is created is tracked using a unique lease ID, which can be used to renew or revoke a lease.

You can revoke an individual lease using the command vault lease revoke <lease_id>

You can also revoke ALL leases from a secrets engine using the -prefix flag, such as vault lease revoke -prefix azure/

You can also revoke leases created from a specific role by using the -prefix flag but specifying the path all the way to the role like this: vault lease revoke -prefix azure/creds/<role_name>"

” Vault Commands: lease revoke

B: Correct. vault lease revoke -prefix azure/ revokes all leases under azure/.

C: Correct. vault lease revoke azure/creds/bryan-krauseneed0373-ca92-99b6-b914-779b7bb0e1d9 targets the specific lease ID.

E: Correct. vault lease revoke -prefix azure/creds/bryan-krausen revokes all leases for that role.

A: Incorrect; lacks the -prefix flag, making it invalid syntax.

D: Incorrect; lacks the -prefix flag and isn't a full lease ID.

Reference:

Vault Commands: lease revoke

QUESTION 204

Which of the following are considered benefits of using policies in Vault? (Select three)

- A. Policies are assigned to a token on a 1:1 basis to eliminate conflicting policies
- B. Provides granular access control to paths within Vault
- C. Policies have an implicit deny, meaning that policies are deny by default
- D. Policies provide Vault operators with role-based access control

Answer: B, C, D

Explanation:

Comprehensive and Detailed In-Depth

Vault policies offer several benefits for access control. The Vault documentation states:

"There are many benefits to using Vault policies, including:

Provides granular access control to paths within Vault to control who can access certain paths inside

Vault

Policies have an implicit deny, meaning that policies are deny by default - no policy means no authorization

Policies provide Vault operators with role-based access control so you can ensure users only have access to the paths required"

” Vault Tutorials: Policies

B: Correct. Granular control is a core feature.

C: Correct. Implicit deny enhances security:

"Policies in Vault follow the principle of least privilege by having an implicit deny."

” Vault Policies

D: Correct. Role-based access simplifies management.

A: Incorrect; tokens can have multiple policies:

"Policies are indeed attached to tokens, but tokens can be assigned more than one policy if needed.

Policies are cumulative and capabilities are additive."

” Vault Tutorials: Policies

Reference:

Vault Tutorials: Policies

Vault Policies

QUESTION 205

A MySQL server has been deployed on Google Cloud Platform (GCP) to support a legacy application.

You want to generate dynamic credentials against this MySQL server rather than use static credentials. What Vault secrets engine would you use to accomplish this?

A. The GCP secrets engine

B. The Identity secrets engine

C. The database secrets engine

D. The Cubbyhole secrets engine

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

The Database secrets engine generates dynamic credentials for databases like MySQL, regardless of the platform. The Vault documentation states:

"Regardless of where a database server is deployed, you can use the database secrets engine to generate dynamic credentials against it. It doesn't matter what platform that server is deployed on, you would still use the database secrets engine. The database secrets engine generates database credentials dynamically based on configured roles."

” Vault Secrets: Databases

C: Correct. The database engine supports MySQL:

"Supported databases include PostgreSQL, MySQL, MongoDB, and more."

” Vault Secrets: Databases

A: GCP engine is for GCP services, not databases.

B: Identity engine manages identities, not credentials.

D: Cubbyhole is for temporary storage, not dynamic credentials.

Reference:

Vault Secrets: Databases

QUESTION 206

Your organization has enabled the LDAP auth method on the path of corp-auth/. When you access the Vault UI, you cannot log in despite providing the correct credentials. Based on the screenshot below, what action should you take to log in?

Sign in to Vault

Method

LDAP

Username

bryan.krausen

Password

.....

[More options](#)

Sign In

Contact your administrator for login credentials

- A. Select corp-auth from the dropdown list
- B. Enter the username as corp-auth/bryan.krausen
- C. Select More Options and enter the Mount path that LDAP was enabled on (corp-auth/)
- D. Change to the Namespace of corp-auth before trying to authenticate

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

When an auth method like LDAP is mounted at a non-default path (e.g., corp-auth/), the Vault UI requires specifying that path. The Vault documentation implies this via CLI examples, and UI behavior confirms it:

"If a backend was mounted using a non-default path, you need to provide it under the Mount Path option under More Options."

” Vault Tutorials: Getting Started UI (Implied)

C: Correct. Clicking æMore Options and entering corp-auth/ directs the UI to the LDAP method:

"By entering the mount path, you are directing Vault to use the LDAP auth method configured on that specific path for authentication."

” Vault Auth: LDAP

A: Dropdowns typically list methods, not paths; incorrect assumption.

B: Username doesn't include the path in this context.

D: Namespace is unrelated to auth mount paths.

Reference:

Vault Tutorials: Getting Started UI

Vault Auth: LDAP

QUESTION 207

You have enabled the Transit secrets engine on your Vault cluster to provide an "encryption as a service" service as your team develops new applications. What is a prime use case for the Transit secrets engine?

- A. Encrypting data before being written to an Amazon S3 bucket
- B. Storing the encrypted data in Vault for easy retrieval
- C. Generating dynamic SSH credentials for access to local systems
- D. Creating X.509 certificates for a new fleet of containers

Answer: A

Explanation:

Comprehensive and Detailed In-Depth

The Transit secrets engine provides encryption as a service. The Vault documentation states:

"The Transit secrets engine is used to encrypt data in transit. It does NOT store the data locally. It simply encrypts the data and returns the ciphertext to the requester. A prime use case is encrypting data before being written to an external storage service like Amazon S3."

” Vault Secrets: Transit

A: Correct. Encrypting data for S3 is a key use case:

"Encrypting data before being written to an Amazon S3 bucket ensures that sensitive data is protected both in transit and at rest."

” Transit Tutorial

B: Incorrect; Transit doesn't store data long-term.

C: SSH credentials are handled by the SSH engine.

D: X.509 certificates are managed by the PKI engine.

Reference:

Vault Secrets: Transit

Transit Tutorial

QUESTION 208

By default, what methods of authentication does Vault support? (Select four)

A. SSH

B. Kubernetes

C. VMware

D. LDAP

E. AppRole

F. JWT

Answer: B, D, E, F

Explanation:

Comprehensive and Detailed In-Depth

Vault supports multiple auth methods by default. The Vault documentation states:

"Auth methods are the components in Vault that perform authentication and are responsible for assigning identity and a set of policies to a user. Available auth methods include AppRole, JWT/OIDC, Kubernetes, LDAP, and more."

” Vault Auth Methods

B: Kubernetes is supported:

"Kubernetes authentication method in Vault allows Kubernetes service accounts to authenticate with Vault."

” Vault Auth: Kubernetes

D: LDAP is supported:

"LDAP authentication method allows users to authenticate against an LDAP directory."

” Vault Auth: LDAP

E: AppRole is supported:

"AppRole authentication method in Vault allows machines or applications to authenticate with Vault."

” Vault Auth: AppRole

F: JWT is supported:

"JWT authentication method in Vault allows users to authenticate using JSON Web Tokens (JWT)."

” Vault Auth: JWT

A: SSH is a secrets engine, not an auth method.

C: VMware is not a default auth method.

Reference:

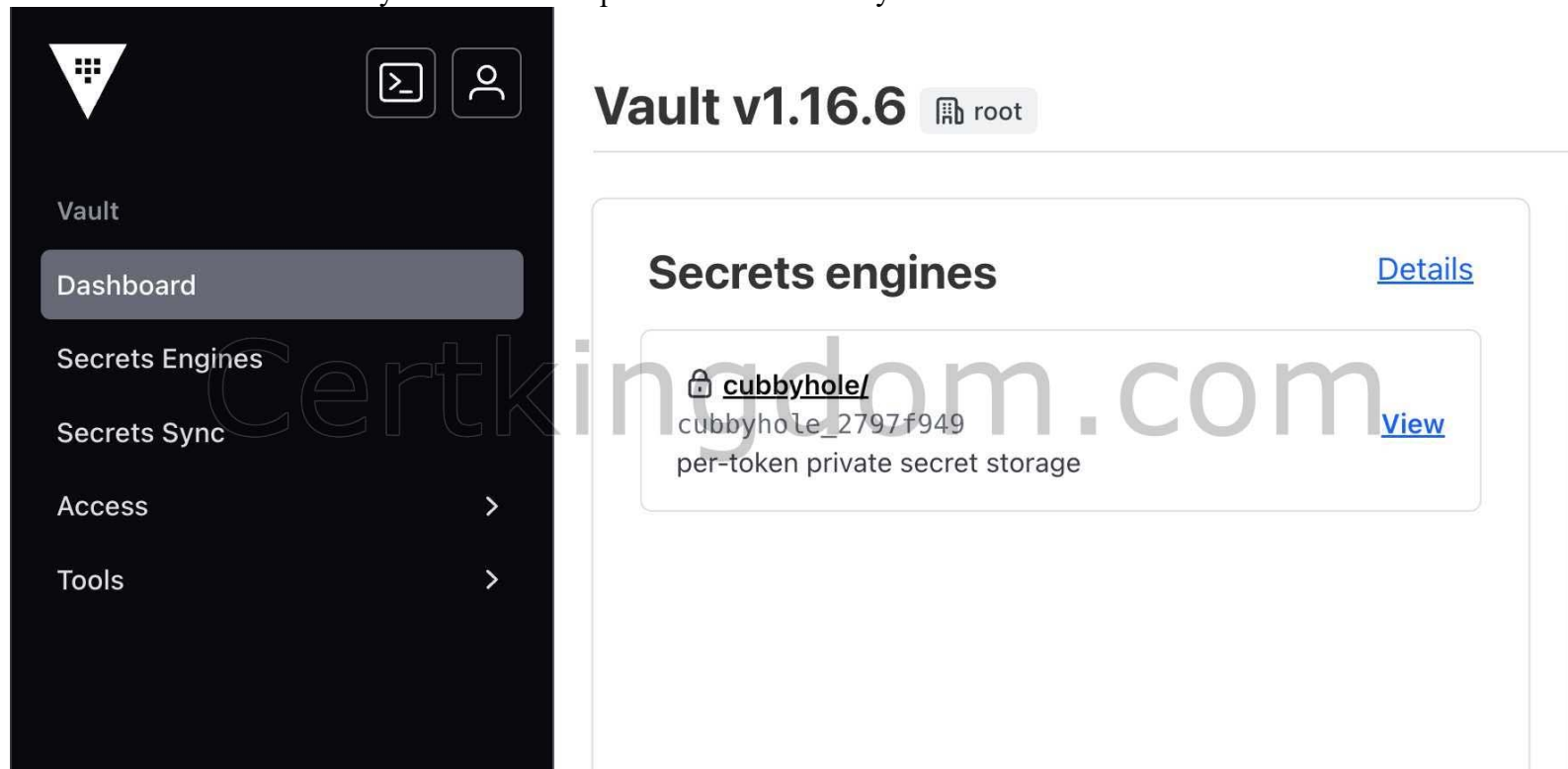
Vault Auth Methods

QUESTION 209

A developer has requested access to manage secrets at the path kv/apps/webapp01. You create the policy below which gives them the proper access:

```
path "kv/apps/webapp01" {  
  capabilities = ["read", "create", "update", "list"]  
}
```

However, when the developer logs in to the Vault UI, they see the following screenshot and cannot access the desired secret. Why can't the developer see the secrets they need?



A. The Vault UI isn't enabled for the developer, therefore they will only see the default options

- B. The key/value secrets engine isn't available in the Vault UI, therefore the developer should use a different Vault interface instead
- C. The policy doesn't permit list access to the paths prior to the secret so the Vault UI doesn't display the mount path
- D. The secrets are stored under the cubbyhole secrets engine, so the developer should browse to that secrets engine

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

The Vault UI requires list permissions on parent paths to navigate mounts. The Vault documentation states:

"When you are using the UI, you will likely need to add additional LIST permissions to the mount (sys/mounts) and then LIST for every path up to the desired secret."

” Vault API: sys/mounts

C: Correct. The policy lacks list on kv/ or kv/apps/, so the UI can't display kv/:

"The policy doesn't permit list access to the paths prior to the secret so the Vault UI doesn't display the mount path."

” Vault Tutorials: Policies

A: Incorrect; the UI isn't user-specific.

B: Incorrect; KV is available in the UI.

D: Incorrect; the path is kv/, not cubbyhole.

Reference:

Vault API: sys/mounts

Vault Tutorials: Policies

QUESTION 210

A developer team requests integration of their legacy application with Vault to encrypt and decrypt data for a backend database. They cannot modify the application for Vault authentication. What is the best way to achieve this integration?

- A. Enable the Transit secrets engine and configure the secrets engine to send data directly to the legacy app
- B. Have the app team call the Vault API to encrypt and decrypt the required data
- C. Enable and configure the Kubernetes auth method to allow the application to authenticate to Vault using a JWT
- D. Run the Vault Agent on the application server(s) and use the Auto Auth feature to manage the tokens

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

The Vault Agent with Auto-Auth is ideal for legacy apps unable to modify for authentication. The Vault documentation states:

"Legacy applications often suffer from the ability to integrate with modern platforms such as Vault.

To assist with this, you can use the Vault Agent to authenticate and manage a Vault token automatically. The token is written to a sink (local file) that the application can pick up and use. The Vault Agent Auto Auth feature will manage the lifecycle of the token to ensure there is always a valid token that the application can use."

” Vault Agent Auto Auth

D: Correct. The Agent handles tokens for Transit encryption:

"Running the Vault Agent on the application server(s) and utilizing the Auto Auth feature is the best way to integrate Vault with the legacy application."

” Vault Agent Auto Auth

A: Transit doesn't send data directly.

B: Requires app modification, not feasible.

C: Kubernetes auth requires app changes and Kubernetes context.

Reference:

Vault Agent Auto Auth

Vault Secrets: Transit

QUESTION 211

You have a CI/CD pipeline using Terraform to provision AWS resources with static privileged credentials. Your security team requests that you use Vault to limit AWS access when needed. How can you enhance this process and increase pipeline security?

A. Enable the SSH secrets engine and have Terraform generate dynamic credentials when deploying resources in AWS

B. Enable the Transit secrets engine to encrypt the AWS credentials and have Terraform retrieve these credentials when needed

C. Store the AWS credentials in the Vault KV store and use the Vault provider to obtain these credentials on each terraform apply

D. Enable the aws secrets engine and configure Terraform to dynamically generate a short-lived AWS credential on each terraform apply

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

The AWS secrets engine generates dynamic credentials, enhancing security. The Vault documentation states:

"The best bet here is to use the AWS secrets engine to generate dynamic credentials for your AWS account(s) when Terraform is executed. You can use the Vault provider to grab these credentials for Vault and then use the credentials as inputs for your AWS provider. In this scenario, Terraform would generate credentials only when executed, and the credentials would automatically expire when the lease expires."

” Vault Secrets: AWS

D: Correct. Dynamic, short-lived credentials limit exposure:

"Enabling the aws secrets engine in Vault allows you to dynamically generate short-lived AWS credentials for each terraform apply."

” Vault Secrets: AWS

A: SSH engine is unrelated to AWS.

B: Transit encrypts data, not credentials.

C: KV stores static credentials, less secure.

Reference:

Vault Secrets: AWS

Vault Provider for Terraform

QUESTION 212

What is the primary role of the Vault Security Operator (VSO) in a Kubernetes environment?

- A. Managing Vault server deployments and auto-scaling Vault instances in Kubernetes
- B. Enforcing Kubernetes network policies for Vault communication
- C. Automating the injection and lifecycle management of Vault secrets for Kubernetes workloads
- D. Replacing Kubernetes Secrets with a built-in alternative that does not require Vault

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

The VSO automates secret management in Kubernetes. The Vault documentation states:

"The Vault Security Operator (VSO) is designed to streamline the integration of Vault with Kubernetes by automating the retrieval, injection, and lifecycle management of secrets for workloads running in a Kubernetes cluster. It enables Kubernetes applications to securely consume Vault secrets without requiring direct interaction with Vault, improving security and operational efficiency."

” Vault Security Operator

C: Correct.

"Automating the injection and lifecycle management of Vault secrets for Kubernetes workloads."

” Vault Security Operator

A: Server management is not VSO's role.

B: Network policies are separate.

D: VSO enhances, doesn't replace, Kubernetes Secrets.

Reference:

Vault Security Operator

QUESTION 213

Your organization recently suffered a security breach on a specific application, and the security response team believes that MySQL database credentials were likely obtained during the event. The application generated the credentials using the database secrets engine in Vault mounted at the path database/. How can you quickly revoke all of the secrets generated by this secrets engine?

A. vault token revoke database/*

B. vault secrets disable mysql

C. vault lease renew database/creds/mysql

D. vault lease revoke -prefix database/

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

To revoke all secrets from the database/ engine, use vault lease revoke -prefix. The Vault documentation states:

"If you need to revoke many leases, you can use vault lease revoke -prefix <prefix> and Vault will revoke all leases associated with the specified path. For example, you can revoke all leases associated with an entire database secrets engine by using vault lease revoke -prefix database/."

” Vault Commands: lease revoke

D: Correct. Revokes all leases under database/:

"Using the command vault lease revoke -prefix database/ will revoke all the leases that have a prefix matching the specified path database/."

” Vault Commands: lease revoke

A: Revokes tokens, not leases.

B: Disables the engine, not existing secrets.

C: Renews a specific lease, not revokes all.

Reference:

Vault Commands: lease revoke

Vault Secrets: Databases

QUESTION 214

You need to write a new policy for Vault for a group of users on the automation team. The requirements stipulate that each user (and all future users) get access to their own private section of a KV secrets engine at the path kv/team/ and be able to manage their own secrets. Which policy below meets these requirements while minimizing the administrative effort and following the principle of least privilege?

- A. path "secret/data/groups/{{identity.groups.ids.2f62-9503-42aa7A869741.name}}/" { capabilities = ["list"] }
- B. path "kv/team/frank/" { capabilities = ["create", "update", "read", "delete"] } path "kv/team/steve/" { capabilities = ["create", "update", "read", "delete"] } path "kv/team/bryan/" { capabilities = ["create", "update", "read", "delete"] }
- C. path "kv/team/" { capabilities = ["create", "update", "read", "delete"] }
- D. path "kv/team/{{identity.entity.id}}/" { capabilities = ["create", "update", "read", "delete"] } path "kv/team/{{identity.entity.id}}/" { capabilities = ["create", "update", "read", "delete"] }

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

Templated policies with {{identity.entity.id}} provide user-specific access. The Vault documentation states:

"This policy would permit all current and future users with a custom path based on their entity ID when they log into Vault using a variable replacement within the path. Templated policies allow policy authors to create policies that can dynamically adjust based on attributes of the identity requesting access."

” Vault Policies: Templated Policies

D: Correct. Uses entity ID for private sections with minimal effort:

"By using {{identity.entity.id}}, each user gets access to their own private section, minimizing administrative effort as new users automatically get their own path."

” Vault Policies: Templated Policies

A: Group-based and only lists, not manages.

B: Hardcodes users, not scalable.

C: Grants all users access to all secrets, violating least privilege.

Reference:

Vault Policies: Templated Policies

QUESTION 215

A security architect is designing a solution to address the "Secret Zero" problem for a Kubernetes-based application that needs to authenticate to HashiCorp Vault. Which approach correctly leverages Vault features to solve this challenge?

- A. Store the Vault root token in a ConfigMap and mount it to all containers that require access to sensitive information
- B. Generate a long-lived token during deployment and store it as an environment variable within each container that needs to access Vault
- C. Configure the Kubernetes auth method in Vault and enable applications to authenticate without pre-shared secrets
- D. Implement a custom sidecar container that uses AppRole role-id and secret-id each time the application needs to access Vault

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

The Kubernetes auth method addresses Secret Zero by using service account tokens. The Vault documentation states:

"The 'Secret Zero' problem refers to the bootstrapping challenge of how applications can authenticate to a secrets management system without requiring an initial secret. In a Kubernetes environment, the Kubernetes Auth Method in Vault allows applications to authenticate using their Kubernetes service account tokens, which are automatically provided to pods. The Vault server validates these tokens against the Kubernetes API server, establishing a chain of trust where applications can authenticate to Vault without pre-shared secrets."

” Vault Auth Methods

C: Correct. Eliminates pre-shared secrets:

"Configuring the Kubernetes auth method in Vault allows applications running in Kubernetes to authenticate to Vault without the need for pre-shared secrets."

” Vault Auth: Kubernetes

A, B: Introduce static secrets, worsening Secret Zero.

D: Retains pre-shared secrets (role-id/secret-id).

Reference:

Vault Auth Methods

Vault Auth: Kubernetes

QUESTION 216

Over a few years, you have a lot of data that has been encrypted by older versions of a Transit encryption key. Due to compliance regulations, you have to re-encrypt the data using the newest version of the encryption key. What is the easiest way to complete this task without putting the data at risk?

- A. Rotate the encryption key used to encrypt the data
- B. Decrypt the data manually and encrypt it with the latest version
- C. Use the transit rewrap feature
- D. Create a new master key used by Vault

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

The Transit rewrap feature re-encrypts data safely. The Vault documentation states:

"Luckily, Vault provides an easy way of re-wrapping encrypted data when a key is rotated. Using the rewrap API endpoint, a non-privileged Vault entity can send data encrypted with an older version of the key to have it re-encrypted with the latest version. The application performing the re-wrapping never interacts with the decrypted data."

” Transit Rewrap Tutorial

C: Correct. Rewrap avoids decryption risks:

"Using the transit rewrap feature in Vault allows you to re-encrypt the data without decrypting it first."

” Transit Rewrap Tutorial

A: Rotation doesn't re-encrypt existing data.

B: Manual decryption exposes data.

D: Master key changes don't affect Transit data.

Reference:

Transit Rewrap Tutorial

QUESTION 217

True or False? Your organization currently runs all of its workloads on Google Cloud Platform (GCP). Recently, Vault has been deployed, and you need to select an auth method to authenticate your workloads with Vault. Based on this information, GCP is the only auth method that can be used in your environment.

- A. True
- B. False

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

False. Vault supports multiple auth methods, not just platform-specific ones. The Vault documentation states:

"Just because you are using a certain platform does not mean you need to use the related auth method. Vault offers a variety of auth methods that can be used based on the organization's needs and existing infrastructure, allowing for flexibility and customization in authentication processes."

” Vault Auth Concepts

B: Correct. Options like AppRole, LDAP, or JWT can be used on GCP:

"GCP auth MIGHT be the best option, but it's not the ONLY option that you can use."

” Vault Auth Concepts

A: Incorrect; Vault isn't limited to GCP auth on GCP.

Reference:

Vault Auth Concepts

QUESTION 218

There are a few ways in Vault that can be used to obtain a root token. Select the valid methods from the answers below. (Select three)

- A. Generating a root token using a quorum of recovery keys when using Vault auto unseal
- B. Initializing Vault when first creating the cluster by using vault operator init
- C. Using a batch DR operation token to create a new root token in the event of an emergency
- D. Running the command vault token create when using a valid root token

Answer: A, B, D

Explanation:

Comprehensive and Detailed In-Depth

Root tokens are restricted in creation. The Vault documentation states:

"Root tokens are tokens that have the root policy attached to them. In fact, there are only three ways to create root tokens:

The initial root token generated at vault operator init -- this token has no expiration

By using another root token; a root token with an expiration cannot create a root token that never expires

By using vault operator generate-root with the permission of a quorum of unseal/recovery key holders"

” Vault Concepts: Tokens

A, B, D: Correct per the above.

C: Incorrect; DR tokens are for replication, not root creation:

"DR operation tokens are typically used for disaster recovery operations and may not be directly related to generating a root token in Vault."

” Vault Replication

Reference:

Vault Concepts: Tokens

QUESTION 219

Vault enables the generation of dynamic credentials against many different platforms. When generating these credentials, what Vault feature is used to track the credentials?

A. namespace

B. role

C. token

D. lease_id

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

Dynamic credentials are tracked via leases. The Vault documentation states:

"With every dynamic secret and service type authentication token, Vault creates a lease. A lease is metadata containing information such as time duration, renewability, and more. Vault promises that the data will be valid for the given period, or Time To Live (TTL). The lease_id is a unique identifier assigned to each dynamically generated credential by Vault."

” Vault Concepts: Leases

D: Correct. lease_id tracks credential lifecycle:

"It is used to track the lifecycle of the credential, including its creation, renewal, and revocation."

” Vault Concepts: Leases

A: Namespaces organize, not track.

B: Roles define generation, not tracking.

C: Tokens authenticate, not track credentials.

Reference:

Vault Concepts: Leases

QUESTION 220

You have multiple Vault clusters in your environment, one for test and one for production. You have

the CLI installed on your local machine and need to target the production cluster to make configuration changes. What environment variable can you set to target the production cluster?

- A. VAULT_REDIRECT_ADDR
- B. VAULT_CLUSTER_ADDR
- C. VAULT_ADDR
- D. VAULT_CAPATH

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

The VAULT_ADDR variable specifies the target Vault server. The Vault documentation states:

"VAULT_ADDR is the environment variable that is used to specify the address of the Vault server expressed as a URL and port, for example: <https://vault.bryankrausen.com:8200/>. You can easily modify the value of the environment variable whenever you want to target a different Vault node/cluster."

” Vault Environment Variables

C: Correct. Sets the production cluster address:

"Setting the VAULT_ADDR environment variable allows you to specify the address of the Vault server you want to target."

” Vault Environment Variables

A, B, D: Incorrect; unrelated to CLI targeting.

Reference:

Vault Environment Variables

QUESTION 221

Your organization audited an essential application and found it isn't securely storing data

a. For added security, auditors recommended encrypting all data before storing it in a backend database, and the application server should not store encryption keys locally. Which secrets engine meets these requirements?

- A. PKI secrets engine
- B. SSH secrets engine
- C. Transit secrets engine
- D. Cubbyhole secrets engine

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

The Transit secrets engine encrypts data without local key storage. The Vault documentation states:

"The Transit secrets engine allows you to send cleartext data to Vault to be encrypted. Vault will encrypt the data with the referenced encryption key, which is stored locally, and returns the ciphertext to the application. Although the encryption keys in the Transit secrets engine are exportable, they are generally kept in Vault."

” Transit Tutorial

C: Correct. Meets encryption and key security needs:

"It allows applications to encrypt data before storing it in a backend database and decrypt it when needed, without storing encryption keys locally."

” Vault Secrets: Transit

A: PKI is for certificates.

B: SSH is for SSH credentials.

D: Cubbyhole is for temporary storage.

Reference:

Transit Tutorial

Vault Secrets: Transit

QUESTION 222

True or False? Performing a rekey operation using the vault operator rekey command creates new unseal/recovery keys as well as a new root key?

A. True

B. False

Answer: B

Explanation:

Comprehensive and Detailed In-Depth

False. The vault operator rekey command updates unseal/recovery keys, not the master key (often confused with æroot key). The Vault documentation states:

"The operator rekey command generates a new set of unseal keys. This can optionally change the total number of key shares or the required threshold of those key shares to reconstruct the master key. This operation is zero downtime, but it requires that Vault is unsealed and a quorum of existing unseal keys are provided."

” Vault Commands: operator rekey

B: Correct. Only unseal keys are recreated:

"When performing a rekey operation using the vault operator rekey command, new unseal/recovery

keys are generated, but the root key remains the same."

” Vault Commands: operator rekey

A: Incorrect; the master key persists.

Reference:

Vault Commands: operator rekey

QUESTION 223

You are planning the deployment of your first Vault cluster and have decided to use Integrated Storage as the storage backend. Where do you configure the storage backend to be used by Vault?

- A. In the systemd service file
- B. Inside the Vault service once Vault is up and running
- C. In the Vault configuration file
- D. In the Vault Agent sink file

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

The storage backend is configured in the Vault configuration file. The Vault documentation states:

"The Vault configuration file includes different stanzas and parameters to define a variety of configuration options. These configurations include the storage backend, listener, TLS certificates, seal type, cluster name, log level, UI, cluster IP address, and a few more. Most of these are required to get Vault up and running in the first place, so they must be placed in the configuration file."

” Vault Configuration

C: Correct. For Integrated Storage:

"Configuring the storage backend to be used by Vault is done in the Vault configuration file."

” Vault Configuration: Raft Storage

A: systemd manages the service, not storage.

B: Backend must be set before running.

D: Agent sink is for client tokens.

Reference:

Vault Configuration

Vault Configuration: Raft Storage

QUESTION 224

Which of the following is true about the token authentication method in Vault? (Select three)

- A. The token auth method is automatically enabled in Vault and cannot be disabled

- B. External authentication mechanisms, such as GitHub, are used to dynamically create tokens
- C. The token auth method is used as the first method of authentication for Vault for a newly initialized Vault node/cluster
- D. Tokens cannot be used directly; they must be used in conjunction with one of Vault's many auth methods

Answer: A, B, C

Explanation:

Comprehensive and Detailed In-Depth

The token auth method is foundational to Vault. The Vault documentation states:

"Tokens are the core method for authentication within Vault. It is also the only auth method that cannot be disabled. If you've gone through the getting started guide, you probably noticed that vault server -dev (or vault operator init for a non-dev server) outputs an initial 'root token.' This is the first method of authentication for Vault. All external authentication mechanisms, such as GitHub, map down to dynamically created tokens."

” Vault Concepts: Tokens

A, B, C: Correct per the above.

D: Incorrect; tokens can be used directly:

"Tokens can be used directly or auth methods can be used to dynamically generate tokens based on external identities."

” Vault Concepts: Tokens

Reference:

Vault Concepts: Tokens

QUESTION 225

You are the primary Vault operator. During a routine audit, an auditor requested the ability to display all secrets under a specific path in Vault without seeing the actual stored data

a. Which policy permits the auditor to display the stored secrets without revealing their contents?

- A. path "kv/apps/production/" { capabilities = ["list"] }
- B. path "kv/apps/+" { capabilities = ["list"] }
- C. path "kv/+production" { capabilities = ["list"] }
- D. path "kv/apps/*" { capabilities = ["list", "read"] }

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

The list capability allows viewing secret names without data. The Vault documentation states:
"The list capability is required to list keys at a path without necessarily being able to read the data at those paths. The + symbol is a directory replacement and ANY value would be permitted in that path segment."

” Vault Policies: Capabilities

” Vault Policies: Policy Syntax

C: Correct. Lists all secrets under kv/<anything>/production:

"This policy allows the auditor to list all secrets under the specified path kv/+ /production without being able to read the actual stored data."

” Vault Policies: Capabilities

A, B: Too narrow, missing some secrets.

D: Includes read, exposing data.

Reference:

Vault Policies: Capabilities

Vault Policies: Policy Syntax

QUESTION 226

You are using the Vault API to test authentication before modifying your CI/CD pipeline to properly authenticate to Vault. You manually authenticate to Vault and receive the response below. Based on the provided options, which of the following are true? (Select four)

```
$ curl \
```

```
--request POST \
```

```
--data @payload.json \
```

```
https://vault.krausen.com:8200/v1/auth/userpass/login/bryan.krausen | jq
```

```
*****
```

```
***** RESPONSE BELOW *****
```

```
*****
```

```
{
```

```
"request_id": "f758e8da-11b6-8341-d404-56f0c370a7fa",
```

```
"lease_id": "",
```

```
"renewable": false,
```

```
"lease_duration": 0,
```

```
"data": null,
```

```
"wrap_info": null,
```

```
"warnings": null,
```

```
"auth": {
```

```
"client_token": "hvs.CbzCNJCVWt63jyzyaJakgDwz",
```

```
"accessor": "rffwXzKFcxvaQi6Vgo8tY4Lt",
```

```
"policies": [
```

```
"training",
"default"
],
"token_policies": [
"training",
"default"
],
"metadata": {
"username": "bryan.krausen"
},
"lease_duration": 84600,
"renewable": true,
"entity_id": "f1795f6a-c576-d619-b2d5-74c0ace08edb",
"token_type": "service",
"orphan": true
}
}
```

- A. The token required to retrieve a secret is hvs.CbzCNJCVWt63jyzyaJakgDwz
- B. The returned token is a batch token
- C. The user needs to retrieve .auth.client_token in order to perform other actions
- D. The accessor will be used to authenticate to Vault to retrieve secrets
- E. The user is using the userpass auth method
- F. The user's password is stored in a file named payload.json

Answer: A, C, E, F

Explanation:

Comprehensive and Detailed In-Depth

The API response provides authentication details. The Vault documentation states:

"When executing an authentication request to Vault, you will need to provide the credentials that will be used for authentication. Once successfully authenticated, Vault will return a bunch of information. The primary value that you need to retrieve from this response is the client_token, which can be queried from a JSON parsing tool (such as jq) by grabbing the value of .auth.client_token."

” Vault API Docs

A, C, E, F: Correct per the response and endpoint (/auth/userpass).

B: Incorrect; token_type is service, not batch:

"The returned token is a service token used for interacting with Vault's API on behalf of the

authenticated user."

” Vault Concepts: Tokens

D: Incorrect; accessors don't authenticate:

"The accessor value provided in the response is not typically used for direct authentication to Vault to retrieve secrets."

” Vault Concepts: Tokens

Reference:

Vault API Docs

Vault Auth: Userpass

QUESTION 227

A new application is being provisioned in your environment. The application requires the generation of dynamic credentials against the Oracle database in order to read reporting data

a. Which is the best auth method to use to permit the application to authenticate to Vault?

A. OIDC

B. GitHub

C. Userpass

D. AppRole

Answer: D

Explanation:

Comprehensive and Detailed In-Depth

AppRole is optimal for machine authentication. The Vault documentation states:

"AppRole is an auth method that is better suited for machine-to-machine authentication. The

AppRole auth method allows machines or applications to authenticate with Vault using a rolespecific secret ID and role ID."

” Vault Auth: AppRole

D: Correct. Ideal for dynamic Oracle credentials:

"AppRole is the best auth method to use in this scenario because it allows machines or applications to authenticate with Vault."

” Vault Auth: AppRole

A, B, C: Human-oriented, not machine-suited.

Reference:

Vault Auth: AppRole

QUESTION 228

Your organization runs workloads on both AWS and Azure for production applications. The security

team has requested that a single Vault authentication mechanism be enabled to support applications on both public cloud platforms. Which of the following would be a valid auth method you can use?

- A. AWS
- B. GitHub
- C. AppRole
- D. Azure

Answer: C

Explanation:

Comprehensive and Detailed In-Depth

AppRole is platform-agnostic. The Vault documentation states:

"Auth methods are commonly grouped into machine-based and human-based auth methods. In this case, AWS and Azure cannot be used since you can't authenticate with a single auth method across both platforms. AppRole is a Vault authentication method that allows machines or applications to authenticate with Vault using a role-specific secret ID and role ID."

” Vault Auth Methods

C: Correct. Works across AWS and Azure:

"It is a flexible and secure method that can be used across different cloud platforms like AWS and Azure."

” Vault Auth: AppRole

A, D: Platform-specific.

B: User-based, not cross-platform.

Reference:

Vault Auth Methods

Vault Auth: AppRole

New Topic: Topic 5, Exam Pool E

QUESTION 229

You are using the Vault userpass auth method mounted at auth/userpass. How do you create a new user named "sally" with password "h0wN0wB4r0wnC0w"? This new user will need the power-users policy.

A.

```
vault put auth/userpass/users/sally \  
password=h0wN0wB4r0wnC0w \  
policies=power-users
```

B.

```
vault write userpass/sally \  
password=h0wN0wB4r0wnC0w \  
policies=power-users
```

C.

```
vault kv write userpass/sally \  
password=h0wN0wB4r0wnC0w \  
policies=power-users
```

D.

```
vault write auth/userpass/users/sally \  
password=h0wN0wB4r0wnC0w \  
policies=power-users
```

Answer: D

Explanation:

To create a new user named æsally with password æh0wN0wB4r0wnC0w and the power-users policy, you would use the Vault userpass auth method mounted at auth/userpass. You would use the following command: ævault write auth/userpass/users/sally password=h0wN0wB4r0wnC0w policies=power-users . This command would create a new user named æsally with the specified password and policy. Reference:

[Userpass Auth Method | Vault | HashiCorp Developer]

[Create Vault policies | Vault | HashiCorp Developer]

QUESTION 230

The vault lease renew command increments the lease time from:

- A. The current time
- B. The end of the lease

Answer: A

Explanation:

The vault lease renew command increments the lease time from the current time, not the end of the lease. This means that the user can request a specific amount of time they want remaining on the lease, termed the increment. This is not an increment at the end of the current TTL; it is an increment from the current time. For example, vault lease renew -increment=3600 my-lease-id would request that the TTL of the lease be adjusted to 1 hour (3600 seconds) from now. Having the increment be rooted at the current time instead of the end of the lease makes it easy for users to reduce the length of leases if they don't actually need credentials for the full possible lease period, allowing those credentials to expire sooner and resources to be cleaned up earlier. The requested increment is completely advisory. The backend in charge of the secret can choose to completely ignore it¹. Reference:

Lease, Renew, and Revoke | Vault | HashiCorp Developer

QUESTION 231

HOTSPOT

Where do you define the Namespace to log into using the Vault UI?

To answer this question

Use your mouse to click on the screenshot in the location described above. An arrow indicator will mark where you have clicked. Click the "Answer" button once you have positioned the arrow to answer the question. You may need to scroll down to see the entire screenshot.

Sign in to Vault

Namespace

finance

Method

LDAP



Username

jake

Password

.....

 [Hide options](#)

Mount path

ldap-mo

 If this backend was mounted using a non-default path, enter it here.

Sign In

Answer:

Explanation:

The namespace can be defined in the `Mount path` field in the `Advanced options` section of the login screen. The mount path is the path where the auth method is enabled, and it can include a namespace prefix. For example, if the LDAP auth method is enabled at the path `ns1/auth/ldap`, where `ns1` is the namespace, then the mount path field should be set to `ns1/auth/ldap`. This way, the Vault UI will log in to the correct namespace and auth method. Alternatively, the namespace can also be specified in the URL of the Vault UI, such as `https://vault.example.com/ui/vault/auth/ns1/auth/ldap/login`.

QUESTION 232

You have a 2GB Base64 binary large object (blob) that needs to be encrypted. Which of the following best describes the transit secrets engine?

- A. A data key encrypts the blob locally, and the same key decrypts the blob locally.
- B. To process such a large blob, Vault will temporarily store it in the storage backend.
- C. Vault will store the blob permanently. Be sure to run Vault on a compute optimized machine
- D. The transit engine is not a good solution for binaries of this size.

Answer: D

Explanation:

The transit secrets engine is not a good solution for binaries of this size, because it is designed to handle cryptographic functions on data in-transit, not data at-rest. The transit secrets engine does not store any data sent to it, so it would require sending the entire 2GB blob to Vault for encryption or decryption, which would be inefficient and impractical. A better solution would be to use the transit secrets engine to generate a data key, which is a high-entropy key that can be used to encrypt or decrypt data locally. The data key can be returned in plaintext or wrapped by another key, depending on the use case. This way, the transit secrets engine only handles the encryption or decryption of the data key, not the data itself, and the data can be stored in any primary data store. Reference: [Transit - Secrets Engines | Vault | HashiCorp Developer](#), [Encryption as a service: transit secrets engine | Vault | HashiCorp Developer](#)

QUESTION 233

How would you describe the value of using the Vault transit secrets engine?

- A. Vault has an API that can be programmatically consumed by applications
- B. The transit secrets engine ensures encryption in-transit and at-rest is enforced enterprise wide

- C. Encryption for application data is best handled by a storage system or database engine, while storing encryption keys in Vault
- D. The transit secrets engine relieves the burden of proper encryption/decryption from application developers and pushes the burden onto the operators of Vault

Answer: D

Explanation:

The transit secrets engine relieves the burden of proper encryption/decryption from application developers and pushes the burden onto the operators of Vault. The transit secrets engine provides encryption as a service, which means that it performs cryptographic operations on data in-transit without storing any data. This allows developers to delegate the responsibility of managing encryption keys and algorithms to Vault operators, who can define and enforce policies on the transit secrets engine. This way, developers can focus on their application logic and data, while Vault handles the encryption and decryption of data in a secure and scalable manner. Reference: Transit - Secrets Engines | Vault | HashiCorp Developer, Encryption as a service: transit secrets engine | Vault | HashiCorp Developer

QUESTION 234

What is the Vault CLI command to query information about the token the client is currently using?

- A. vault lookup token
- B. vault token lookup
- C. vault lookup self
- D. vault self-lookup

Answer: B

Explanation:

The Vault CLI command to query information about the token the client is currently using is vault token lookup. This command displays information about the token or accessor provided as an argument, or the locally authenticated token if no argument is given. The information includes the token ID, accessor, policies, TTL, creation time, and metadata. This command can be useful for debugging and auditing purposes, as well as for renewing or revoking tokens. Reference: token lookup - Command | Vault | HashiCorp Developer, Tokens | Vault | HashiCorp Developer

QUESTION 235

Which of the following is a machine-oriented Vault authentication backend?

- A. Okta
- B. AppRole
- C. Transit
- D. GitHub

Answer: B

Explanation:

AppRole is a machine-oriented authentication method that allows machines or applications to authenticate with Vault using a role ID and a secret ID. The role ID is a unique identifier for the application, and the secret ID is a single-use credential that can be delivered to the application securely. AppRole is designed to provide secure introduction of machines and applications to Vault, and to support the principle of least privilege by allowing fine-grained access control policies to be attached to each role¹.

Okta, GitHub, and Transit are not machine-oriented authentication methods. Okta and GitHub are user-oriented authentication methods that allow users to authenticate with Vault using their Okta or GitHub credentials²³. Transit is not an authentication method at all, but a secrets engine that provides encryption as a service⁴.

Reference:

AppRole Auth Method | Vault | HashiCorp Developer

Okta Auth Method | Vault | HashiCorp Developer

GitHub Auth Method | Vault | HashiCorp Developer

Transit Secrets Engine | Vault | HashiCorp Developer

QUESTION 236

Security requirements demand that no secrets appear in the shell history. Which command does not meet this requirement?

- A. generate-password | vault kv put secret/password value
- B. vault kv put secret/password value-itsasecret
- C. vault kv put secret/password value=@data.txt
- D. vault kv put secret/password value-SSECRET_VALUE

Answer: B

Explanation:

The command that does not meet the security requirement of not having secrets appear in the shell history is B. `vault kv put secret/password value-itsasecret`. This command would store the secret value `œitsasecret` in the key/value secrets engine at the path `secret/password`, but it would also

expose the secret value in the shell history, which could be accessed by other users or malicious actors. This is not a secure way of storing secrets in Vault.

The other commands are more secure ways of storing secrets in Vault without revealing them in the shell history.

A. `generate-password | vault kv put secret/password value` would use a pipe to pass the output of the `generate-password` command, which could be a script or a tool that generates a random password, to the `vault kv put` command, which would store the password in the key/value secrets engine at the path `secret/password`. The password would not be visible in the shell history, only the commands. C. `vault kv put secret/password value=@data.txt` would use the `@` syntax to read the secret value from a file named `data.txt`, which could be encrypted or protected by file permissions, and store it in the key/value secrets engine at the path `secret/password`. The file name would be visible in the shell history, but not the secret value. D. `vault kv put secret/password value=$SECRET_VALUE` would use the `-S` syntax to read the secret value from the environment variable `SECRET_VALUE`, which could be set and unset in the shell session, and store it in the key/value secrets engine at the path `secret/password`. The environment variable name would be visible in the shell history, but not the secret value.

Reference:

[Write Secrets | Vault | HashiCorp Developer]

QUESTION 237

You can build a high availability Vault cluster with any storage backend.

A. True

B. False

Answer: B

Explanation:

Not all storage backends support high availability mode for Vault. Only the storage backends that support locking can enable Vault to run in a multi-server mode where one server is active and the others are standby. Some examples of storage backends that support high availability mode are Consul, Integrated Storage, and ZooKeeper. Some examples of storage backends that do not support high availability mode are Filesystem, MySQL, and PostgreSQL. Reference:

<https://developer.hashicorp.com/vault/docs/concepts/ha1>,

<https://developer.hashicorp.com/vault/docs/configuration/storage2>

QUESTION 238

What command creates a secret with the key "my-password" and the value "53cr3t" at path "mysecrets" within the KV secrets engine mounted at "secret"?

- A. vault kv put secret/my-secrets/my-password 53cr3t
- B. vault kv write secret/my-secrets/my-password 53cr3t
- C. vault kv write 53cr3t my-secrets/my-password
- D. vault kv put secret/my-secrets my-password-53cr3t

Answer: A

Explanation:

The vault kv put command writes the data to the given path in the K/V secrets engine. The command requires the mount path of the K/V secrets engine, the secret path, and the key-value pair to store.

The mount path can be specified with the -mount flag or as part of the secret path. The key-value pair can be given as an argument or read from a file or stdin. The correct syntax for the command is:

```
vault kv put -mount=secret my-secrets/my-password 53cr3t
```

or

```
vault kv put secret/my-secrets my-password=53cr3t
```

The other options are incorrect because they use the deprecated vault kv write command, or they have the wrong order or format of the arguments. Reference:

<https://developer.hashicorp.com/vault/docs/commands/kv/put3>,

<https://developer.hashicorp.com/vault/docs/commands/kv4>

QUESTION 239

What can be used to limit the scope of a credential breach?

- A. Storage of secrets in a distributed ledger
- B. Enable audit logging
- C. Use of a short-lived dynamic secrets
- D. Sharing credentials between applications

Answer: C

Explanation:

Using a short-lived dynamic secrets can help limit the scope of a credential breach by reducing the exposure time of the secrets. Dynamic secrets are generated on-demand by Vault and automatically revoked when they are no longer needed. This way, the credentials are not stored in plain text or in a static database, and they can be rotated frequently to prevent unauthorized access. Dynamic secrets also provide encryption as a service, which means that they perform cryptographic operations on data in-transit without storing any data. This adds an extra layer of security and reduces the risk of

data leakage or tampering. Reference: Dynamic secrets | Vault | HashiCorp Developer, What are dynamic secrets and why do I need them? - HashiCorp

QUESTION 240

What environment variable overrides the CLI's default Vault server address?

- A. VAULT_ADDR
- B. VAULT_HTTP_ADORESS
- C. VAULT_ADDRESS
- D. VAULT_HTTPS_ADDRESS

Answer: B

Explanation:

The environment variable VAULT_ADDR overrides the CLI's default Vault server address. The VAULT_ADDR environment variable specifies the address of the Vault server that is used to communicate with Vault from other applications or processes. By setting this variable, you can avoid hard-coding the Vault server address in your code or configuration files, and you can also use different addresses for different environments or scenarios. For example, you can use a local development server for testing purposes, and a production server for deploying your application. Reference: Commands (CLI) | Vault | HashiCorp Developer, Vault Agent - secrets as environment variables | Vault | HashiCorp Developer

QUESTION 241

Which of the following statements describe the CLI command below?

`S vault login -method=ldap username=mitchellh`

- A. Generates a token which is response wrapped
- B. You will be prompted to enter the password
- C. By default the generated token is valid for 24 hours
- D. Fails because the password is not provided

Answer: A

Explanation:

The CLI command `vault login -method=ldap username=mitchellh` generates a token that is response wrapped. This means that the token contains a base64-encoded response wrapper, which is a JSON object that contains information about the token, such as its policies, metadata, and expiration time. The response wrapper is used to verify the authenticity and integrity of the token, and to prevent

replay attacks. The response wrapper also allows Vault to automatically renew the token when it expires, or to revoke it if it is compromised. The `-method ldap` option specifies that the authentication method is LDAP, which requires a username and password to be provided. The username `mitchellh` is an example of an LDAP user name, and the password will be hidden when entered. Reference: [Vault CLI Reference](#) | [Vault](#) | HashiCorp Developer, [Vault CLI Reference](#) | [Vault](#) | HashiCorp Developer

QUESTION 242

The following three policies exist in Vault. What do these policies allow an organization to do?

app.hcl

```
path "transit/encrypt/my_app_key" {  
  capabilities = ["update"]  
}
```

callcenter.hcl

```
path "transit/decrypt/my_app_key" {  
  capabilities = ["update"]  
}
```

rewrap.hcl

```
path "transit/keys/my_app_key" {  
  capabilities = ["read"]  
}  
  
path "transit/rewrap/my_app_key" {  
  capabilities = ["update"]  
}
```

- A. Separates permissions allowed on actions associated with the transit secret engine
- B. Nothing, as the minimum permissions to perform useful tasks are not present
- C. Encrypt, decrypt, and rewrap data using the transit engine all in one policy
- D. Create a transit encryption key for encrypting, decrypting, and rewrapping encrypted data

Answer: C

Explanation:

The three policies that exist in Vault are:

`admins`: This policy grants full access to all secrets and operations in Vault. It can be used by administrators or operators who need to manage all aspects of Vault.

`default`: This policy grants access to all secrets and operations in Vault except for those that require

specific policies. It can be used as a fallback policy when no other policy matches.

transit: This policy grants access only to the transit secrets engine, which handles cryptographic functions on data in-transit. It can be used by applications or services that need to encrypt or decrypt data using Vault.

These policies allow an organization to perform useful tasks such as:

Encrypting, decrypting, and rewrapping data using the transit engine all in one policy: This policy grants access to both the transit secrets engine and the default policy, which allows performing any operation on any secret in Vault.

Creating a transit encryption key for encrypting, decrypting, and rewrapping encrypted data: This policy grants access only to the transit secrets engine and its associated keys, which are used for encrypting and decrypting data in transit using AES-GCM with a 256-bit AES key or other supported key types.

Separating permissions allowed on actions associated with the transit secret engine: This policy grants access only to specific actions related to the transit secrets engine, such as creating keys or wrapping requests. It does not grant access to other operations or secrets in Vault.

QUESTION 243

Your DevOps team would like to provision VMs in GCP via a CICD pipeline. They would like to integrate Vault to protect the credentials used by the tool. Which secrets engine would you recommend?

- A. Google Cloud Secrets Engine
- B. Identity secrets engine
- C. Key/Value secrets engine version 2
- D. SSH secrets engine

Answer: A

Explanation:

The Google Cloud Secrets Engine is the best option for the DevOps team to provision VMs in GCP via a CICD pipeline and integrate Vault to protect the credentials used by the tool. The Google Cloud Secrets Engine can dynamically generate GCP service account keys or OAuth tokens based on IAM policies, which can be used to authenticate and authorize the CICD tool to access GCP resources. The credentials are automatically revoked when they are no longer used or when the lease expires, ensuring that the credentials are short-lived and secure. The DevOps team can configure rolesets or static accounts in Vault to define the scope and permissions of the credentials, and use the Vault API or CLI to request credentials on demand. The Google Cloud Secrets Engine also supports generating access tokens for impersonated service accounts, which can be useful for delegating access to other service accounts without storing or managing their keys.

The Identity Secrets Engine is not a good option for this use case, because it does not generate GCP credentials, but rather generates identity tokens that can be used to access other Vault secrets engines or namespaces². The Key/Value Secrets Engine version 2 is also not a good option, because it does not generate dynamic credentials, but rather stores and manages static secrets that the user provides³. The SSH Secrets Engine is not a good option either, because it does not generate GCP credentials, but rather generates SSH keys or OTPs that can be used to access remote hosts via SSH⁴.

Reference:

Google Cloud - Secrets Engines | Vault | HashiCorp Developer

Identity - Secrets Engines | Vault | HashiCorp Developer

KV - Secrets Engines | Vault | HashiCorp Developer

SSH - Secrets Engines | Vault | HashiCorp Developer

QUESTION 244

Which of these is not a benefit of dynamic secrets?

- A. Supports systems which do not natively provide a method of expiring credentials
- B. Minimizes damage of credentials leaking
- C. Ensures that administrators can see every password used
- D. Replaces cumbersome password rotation tools and practices

Answer: C

Explanation:

Dynamic secrets are generated on-demand by Vault and have a limited time-to-live (TTL). They do not ensure that administrators can see every password used, as they are often encrypted and ephemeral. The benefits of dynamic secrets are:

They support systems that do not natively provide a method of expiring credentials, such as databases, cloud providers, SSH, etc. Vault can revoke the credentials when they are no longer needed or when the lease expires.

They minimize the damage of credentials leaking, as they are short-lived and can be easily rotated or revoked. If a credential is compromised, the attacker has a limited window of opportunity to use it before it becomes invalid.

They replace cumbersome password rotation tools and practices, as Vault can handle the generation and revocation of credentials automatically and securely. This reduces the operational overhead and complexity of managing secrets.

Reference: <https://developer.hashicorp.com/vault/tutorials/getting-started/getting-started-dynamicsecrets1>, <https://developer.hashicorp.com/vault/docs/concepts/lease2>

QUESTION 245

Which of the following cannot define the maximum time-to-live (TTL) for a token?

- A. By the authentication method that natively provide a method of expiring credentials
- B. By the client system if credentials leaking
- C. By the mount endpoint configuration very password used
- D. A parent token TTL if password rotation tools and practices
- E. System max TTL

Answer: B

Explanation:

The maximum time-to-live (TTL) for a token is defined by the lowest value among the following factors:

The authentication method that issued the token. Each auth method can have a default and a maximum TTL for the tokens it generates. These values can be configured by the auth method's mount options or by the auth method's specific endpoints.

The mount endpoint configuration that the token is accessing. Each secrets engine can have a default and a maximum TTL for the leases it grants. These values can be configured by the secrets engine's mount options or by the secrets engine's specific endpoints.

A parent token TTL. If a token is created by another token, it inherits the remaining TTL of its parent token, unless the parent token has an infinite TTL (such as the root token). A child token cannot outlive its parent token.

System max TTL. This is a global limit for all tokens and leases in Vault. It can be configured by the system backend's `max_lease_ttl` option.

The client system that uses the token cannot define the maximum TTL for the token, as this is determined by Vault's configuration and policies. The client system can only request a specific TTL for the token, but this request is subject to the limits imposed by the factors above.

Reference: <https://developer.hashicorp.com/vault/docs/concepts/tokens3>,

<https://developer.hashicorp.com/vault/docs/concepts/lease2>,

<https://developer.hashicorp.com/vault/docs/commands/auth/tune4>,

<https://developer.hashicorp.com/vault/docs/commands/secrets/tune5>,

<https://developer.hashicorp.com/vault/docs/commands/token/create6>

QUESTION 246

What are orphan tokens?

- A. Orphan tokens are tokens with a use limit so you can set the number of uses when you create them

- B. Orphan tokens are not children of their parent; therefore, orphan tokens do not expire when their parent does
- C. Orphan tokens are tokens with no policies attached
- D. Orphan tokens do not expire when their own max TTL is reached

Answer: D

Explanation:

Orphan tokens are tokens that are root of their own token tree. This means that they do not have any parent token associated with them, and they do not expire when their parent token expires. Orphan tokens are useful for scenarios where you need a short-lived and independent token, such as for testing or debugging purposes. Orphan tokens can also be used to create temporary access tokens for applications or services that need to communicate with Vault without using a long-lived root token. Reference: Tokens | Vault | HashiCorp Developer, Vault cli: how to create orphan token with role - HashiCorp Discuss

QUESTION 247

To give a role the ability to display or output all of the end points under the `/secrets/apps/*` end point it would need to have which capability set?

- A. update
- B. read
- C. sudo
- D. list
- E. None of the above

Answer: C

Explanation:

To give a role the ability to display or output all of the end points under the `/secrets/apps/*` end point, it would need to have the list capability set. The list capability allows a role to perform any operation on any path in Vault, including reading, writing, deleting, and listing. The list capability is required for roles that need to access sensitive data or perform administrative tasks in Vault. The other capabilities are not relevant for this scenario, as they only allow specific operations on specific paths or secrets engines. Reference: Policies | Vault | HashiCorp Developer, token capabilities - Command | Vault | HashiCorp Developer

QUESTION 248

You have been tasked with writing a policy that will allow read permissions for all secrets at path

secret/bar. The users that are assigned this policy should also be able to list the secrets. What should this policy look like?

A.

```
path "secret/bar/*" {  
  capabilities = ["read", "list"]  
}
```

B.

```
path "secret/bar/*" {  
  capabilities = ["list"]  
}  
  
path "secret/bar/" {  
  capabilities = ["read"]  
}
```

C.

```
path "secret/bar/*" {  
  capabilities = ["read"]  
}  
  
path "secret/bar/" {  
  capabilities = ["list"]  
}
```

D.

```
path "secret/bar/+" {  
  capabilities = ["read", "list"]  
}
```

Answer: C

Explanation:

This policy would allow read permissions for all secrets at path secret/bar, as well as list permissions for the secret/bar/ path. The list permission is required to be able to see the names of the secrets under a given path1. The wildcard () character matches any number of characters within a single path segment, while the slash (/) character matches the end of the path2. Therefore, the policy would grant read access to any secret that starts with secret/bar/, such as secret/bar/foo or secret/bar/baz, but not to secret/bar itself. To grant list access to secret/bar, the policy needs to specify the exact path with a slash at the end. This policy follows the principle of least privilege, which means that it only grants the minimum permissions necessary for the users to perform their tasks3.

The other options are not correct because they either grant too much or too little permissions.

Option A would grant both read and list permissions to all secrets under secret/bar, which is more than what is required. Option B would grant list permissions to all secrets under secret/bar, but only read permissions to secret/bar itself, which is not what is required. Option D would use an invalid character (+) in the policy, which would cause an error.

Reference:

Policy Syntax | Vault | HashiCorp Developer

Policy Syntax | Vault | HashiCorp Developer

Policies | Vault | HashiCorp Developer

QUESTION 249

When using Integrated Storage, which of the following should you do to recover from possible data loss?

- A. Failover to a standby node
- B. Use snapshot
- C. Use audit logs
- D. Use server logs

Answer: B

Explanation:

Integrated Storage is a Raft-based storage backend that allows Vault to store its data internally without relying on an external storage system. It also enables Vault to run in high availability mode with automatic leader election and failover. However, Integrated Storage is not immune to data loss or corruption due to hardware failures, network partitions, or human errors. Therefore, it is recommended to use the snapshot feature to backup and restore the Vault data periodically or on demand. A snapshot is a point-in-time capture of the entire Vault data, including the encrypted secrets, the configuration, and the metadata. Snapshots can be taken and restored using the vault operator raft snapshot command or the sys/storage/raft/snapshot API endpoint. Snapshots are encrypted and can only be restored with a quorum of unseal keys or recovery keys. Snapshots are also portable and can be used to migrate data between different Vault clusters or storage backends. Reference: <https://developer.hashicorp.com/vault/docs/concepts/integrated-storage1>, <https://developer.hashicorp.com/vault/docs/commands/operator/raft/snapshot2>, <https://developer.hashicorp.com/vault/api-docs/system/storage/raft/snapshot3>

QUESTION 250

How many Shamir's key shares are required to unseal a Vault instance?

- A. All key shares
- B. A quorum of key shares
- C. One or more keys
- D. The threshold number of key shares

Answer: D

Explanation:

Shamir's Secret Sharing is a cryptographic algorithm that allows a secret to be split into multiple parts, called key shares, such that a certain number of key shares are required to reconstruct the secret. The number of key shares and the threshold number are configurable parameters that depend on the desired level of security and availability. Vault uses Shamir's Secret Sharing to protect its master key, which is used to encrypt and decrypt the data encryption key that secures the Vault data. When Vault is initialized, it generates a master key and splits it into a configured number of key shares, which are then distributed to trusted operators. To unseal Vault, the threshold number of key shares must be provided to reconstruct the master key and decrypt the data encryption key. This process ensures that no single operator can access the Vault data without the cooperation of other key holders. Reference: <https://developer.hashicorp.com/vault/docs/concepts/seal4>, <https://developer.hashicorp.com/vault/docs/commands/operator/init5>, <https://developer.hashicorp.com/vault/docs/commands/operator/unseal6>

QUESTION 251

Which of these are a benefit of using the Vault Agent?

- A. Vault Agent allows for centralized configuration of application secrets engines
- B. Vault Agent will auto-discover which authentication mechanism to use
- C. Vault Agent will enforce minimum levels of encryption an application can use
- D. Vault Agent will manage the lifecycle of cached tokens and leases automatically

Answer: D

Explanation:

Vault Agent is a client daemon that provides the following features:

Auto-Auth - Automatically authenticate to Vault and manage the token renewal process for locally retrieved dynamic secrets.

API Proxy - Allows Vault Agent to act as a proxy for Vault's API, optionally using (or forcing the use of) the Auto-Auth token.

Caching - Allows client-side caching of responses containing newly created tokens and responses containing leased secrets generated off of these newly created tokens. The agent also manages the renewals of the cached tokens and leases.

Templating - Allows rendering of user-supplied templates by Vault Agent, using the token generated by the Auto-Auth step.

Process Supervisor Mode - Runs a child process with Vault secrets injected as environment variables.

One of the benefits of using the Vault Agent is that it will manage the lifecycle of cached tokens and leases automatically. This means that the agent will handle the token renewal and revocation logic, as well as the lease renewal and revocation logic for the secrets that are cached by the agent. This reduces the burden on the application developers and operators, and ensures that the tokens and secrets are always valid and up-to-date. Reference: Vault Agent | Vault | HashiCorp

Developer, Caching - Vault Agent | Vault | HashiCorp Developer

QUESTION 252

Which of the following describes usage of an identity group?

- A. Limit the policies that would otherwise apply to an entity in the group
- B. When they want to revoke the credentials for a whole set of entities simultaneously
- C. Audit token usage
- D. Consistently apply the same set of policies to a collection of entities

Answer: D

Explanation:

An identity group is a collection of entities that share some common attributes. An identity group can have one or more policies attached to it, which are inherited by all the members of the group. An identity group can also have subgroups, which can further refine the policies and attributes for a subset of entities.

One of the use cases of an identity group is to consistently apply the same set of policies to a collection of entities. For example, an organization may have different teams or departments, such as engineering, sales, or marketing. Each team may have its own identity group, with policies that grant access to the secrets and resources that are relevant to their work. By creating an identity group for each team, the organization can ensure that the entities belonging to each team have the same level of access and permissions, regardless of which authentication method they use to log in to Vault. Reference: Identity: entities and groups | Vault | HashiCorp Developer, vault_identity_group | Resources | hashicorp/vault | Terraform | Terraform Registry

QUESTION 253

Vault supports which type of configuration for source limited token?

- A. Cloud-bound tokens
- B. Domain-bound tokens
- C. CIDR-bound tokens
- D. Certificate-bound tokens

Answer: C

Explanation:

Vault supports CIDR-bound tokens, which are tokens that can only be used from a specific set of IP addresses or network ranges. This is a way to limit the scope and exposure of a token in case it is compromised or leaked. CIDR-bound tokens can be created by specifying the bound_cidr_list parameter when creating or updating a token role, or by using the -bound-cidr option when creating a token using the vault token create command. CIDR-bound tokens can also be created by some auth methods, such as AWS or Kubernetes, that can automatically bind the tokens to the source IP or network of the client. Reference: Token - Auth Methods | Vault | HashiCorp Developer, vault token create - Command | Vault | HashiCorp Developer

QUESTION 254

Where does the Vault Agent store its cache?

- A. In a file encrypted using the Vault transit secret engine

- B. In the Vault key/value store
- C. In an unencrypted file
- D. In memory

Answer: D

Explanation:

The Vault Agent stores its cache in memory, which means that it does not persist the cached tokens and secrets to disk or any other storage backend. This makes the cache more secure and performant, as it avoids exposing the sensitive data to potential attackers or unauthorized access. However, this also means that the cache is volatile and will be lost if the agent process is terminated or restarted. To mitigate this, the agent can optionally use a persistent cache file to restore the tokens and leases from a previous agent process. The persistent cache file is encrypted using a key derived from the agent's auto-auth token and a nonce, and it is stored in a user-specified location on disk. Reference: Caching - Vault Agent | Vault | HashiCorp Developer, Vault Agent Persistent Caching | Vault | HashiCorp Developer

QUESTION 255

Your organization has an initiative to reduce and ultimately remove the use of long lived X.509 certificates. Which secrets engine will best support this use case?

- A. PKI
- B. Key/Value secrets engine version 2, with TTL defined
- C. Cloud KMS
- D. Transit

Answer: A

Explanation:

The PKI secrets engine is designed to support the use case of reducing and ultimately removing the use of long lived X.509 certificates. The PKI secrets engine can generate dynamic X.509 certificates on demand, with short time-to-live (TTL) and automatic revocation. This eliminates the need for manual processes of generating, signing, and rotating certificates, and reduces the risk of certificate compromise or misuse. The PKI secrets engine can also act as a certificate authority (CA) or an intermediate CA, and can integrate with external CAs or CRLs. The PKI secrets engine can issue certificates for various purposes, such as TLS, SSH, code signing, email encryption, etc. Reference: <https://developer.hashicorp.com/vault/docs/secrets/pki1>, <https://developer.hashicorp.com/vault/tutorials/getting-started/getting-started-dynamic-secrets>

QUESTION 256

When unsealing Vault, each Shamir unseal key should be entered:

- A. Sequentially from one system that all of the administrators are in front of
- B. By different administrators each connecting from different computers
- B. While encrypted with each administrators PGP key
- D. At the command line in one single command

Answer: B

Explanation:

When unsealing Vault, each Shamir unseal key should be entered by different administrators each connecting from different computers. This is because the Shamir unseal keys are split into shares that are distributed to trusted operators, and no single operator should have access to more than one share. This way, the unseal process requires the cooperation of a quorum of key holders, and enhances the security and availability of Vault. The unseal keys can be entered via multiple mechanisms from multiple client machines, and the process is stateful. The order of the keys does not matter, as long as the threshold number of keys is reached. The unseal keys should not be entered at the command line in one single command, as this would expose them to the history and compromise the security. The unseal keys should not be encrypted with each administrator's PGP key, as this would prevent Vault from decrypting them and reconstructing the master key. Reference: <https://developer.hashicorp.com/vault/docs/concepts/seal3>, <https://developer.hashicorp.com/vault/docs/commands/operator/unseal>

QUESTION 257

As a best practice, the root token should be stored in which of the following ways?

- A. Should be revoked and never stored after initial setup
- B. Should be stored in configuration automation tooling
- C. Should be stored in another password safe
- D. Should be stored in Vault

Answer: A

Explanation:

The root token is the initial token created when initializing Vault. It has unlimited privileges and can perform any operation in Vault. As a best practice, the root token should be revoked and never stored after initial setup. This is because the root token is a single point of failure and a potential security risk if it is compromised or leaked. Instead of using the root token, Vault operators should

create other tokens with appropriate policies and roles that allow them to perform their tasks. If a new root token is needed in an emergency, the vault operator generate-root command can be used to create one on-the-fly with the consent of a quorum of unseal key holders. Reference: Tokens | Vault | HashiCorp Developer, Generate root tokens using unseal keys | Vault | HashiCorp Developer

QUESTION 258

When creating a policy, an error was thrown:

Create ACL policy

 Error

failed to parse policy: path "secret/webapp/*": invalid capability "write"

Name

webapp

Policy

☐ Upload file

```
1 path "secret/webapp/*" {  
2   capabilities = ["read", "write", "delete", "list", "sudo"]  
3 }
```

Certkingdom.com

Which statement describes the fix for this issue?

- A. Replace write with create in the capabilities list
- B. You cannot have a wildcard (" ") in the path
- C. sudo is not a capability

Answer: A

Explanation:

The error was thrown because the policy code contains an invalid capability, `write`. The valid capabilities for a policy are `create`, `read`, `update`, `delete`, `list`, and `sudo`. The `write` capability is not recognized by Vault and should be replaced with `create`, which allows creating new secrets or overwriting existing ones. The other statements are not correct, because the wildcard (*) and the sudo capability are both valid in a policy. The wildcard matches any number of characters within a path segment, and the sudo capability allows performing certain operations that require root privileges.

Reference:

[Policy Syntax | Vault | HashiCorp Developer]

[Policy Syntax | Vault | HashiCorp Developer]

QUESTION 259

Where can you set the Vault seal configuration? Choose two correct answers.

- A. Cloud Provider KMS
- B. Vault CLI
- C. Vault configuration file
- D. Environment variables
- E. Vault API

Answer: CD

Explanation:

The Vault seal configuration can be set in two ways: through the Vault configuration file or through environment variables. The Vault configuration file is a text file that contains the settings and options for Vault, such as the storage backend, the listener, the telemetry, and the seal. The seal stanza in the configuration file specifies the seal type and the parameters to use for additional data protection, such as using HSM or Cloud KMS solutions to encrypt and decrypt the root key. The seal configuration can also be set through environment variables, which will take precedence over the

values in the configuration file. The environment variables are prefixed with `VAULT_SEAL_` and followed by the seal type and the parameter name. For example, `VAULT_SEAL_AWSKMS_REGION` sets the region for the AWS KMS seal. Reference: [Seals - Configuration | Vault | HashiCorp Developer](#), [Environment Variables | Vault | HashiCorp Developer](#)

QUESTION 260

Which of the following vault lease operations uses a `lease_id` as an argument? Choose two correct answers.

- A. `renew`
- B. `revoke -prefix`
- C. `create`
- D. `describe`
- E. `revoke`

Answer: AE

Explanation:

The vault lease operations that use a `lease_id` as an argument are `renew` and `revoke`. The `renew` operation allows a client to extend the validity of a lease associated with a secret or a token. The `revoke` operation allows a client to terminate a lease immediately and invalidate the secret or the token. Both operations require a `lease_id` as an argument to identify the lease to be renewed or revoked. The `lease_id` can be obtained from the response of reading a secret or creating a token, or from the vault lease list command. The other operations, `revoke-prefix`, `create`, and `describe`, do not use a `lease_id` as an argument. The `revoke-prefix` operation allows a client to revoke all secrets or tokens generated under a given prefix. The `create` operation allows a client to create a new lease for a secret. The `describe` operation allows a client to view information about a lease, such as its TTL, policies, and metadata. Reference: [Lease, Renew, and Revoke | Vault | HashiCorp Developer](#), [vault lease - Command | Vault | HashiCorp Developer](#)

QUESTION 261

An organization wants to authenticate an AWS EC2 virtual machine with Vault to access a dynamic database secret. The only authentication method which they can use in this case is AWS.

- A. True
- B. False

Answer: B

Explanation:

The statement is false. An organization can authenticate an AWS EC2 virtual machine with Vault to access a dynamic database secret using more than one authentication method. The AWS auth method is one of the options, but not the only one. The AWS auth method supports two types of authentication: `ec2` and `iam`. The `ec2` type uses the signed EC2 instance identity document to authenticate the EC2 instance. The `iam` type uses the AWS Signature v4 algorithm to sign a request to the `sts:GetCallerIdentity` API and authenticate the IAM principal. However, the organization can also use other auth methods that are compatible with EC2 instances, such as `AppRole`, `JWT/OIDC`, or `Kubernetes`. These methods require the EC2 instance to have some sort of identity material, such as a role ID, a secret ID, a JWT token, or a service account token, that can be used to authenticate to Vault. The identity material can be provisioned to the EC2 instance using various mechanisms, such as user data, metadata service, or cloud-init scripts. The choice of the auth method depends on the use case, the security requirements, and the trade-offs between convenience and control. Reference: [AWS - Auth Methods | Vault | HashiCorp Developer](#), [AppRole - Auth Methods | Vault | HashiCorp Developer](#), [JWT/OIDC - Auth Methods | Vault | HashiCorp Developer](#), [Kubernetes - Auth Methods | Vault | HashiCorp Developer](#)

QUESTION 262

You are using Vault's Transit secrets engine to encrypt your data

a. You want to reduce the amount of content encrypted with a single key in case the key gets compromised. How would you do this?

- A. Use 4096-bit RSA key to encrypt the data
- B. Upgrade to Vault Enterprise and integrate with HSM
- C. Periodically re-key the Vault's unseal keys
- D. Periodically rotate the encryption key

Answer: D

Explanation:

The Transit secrets engine supports the rotation of encryption keys, which allows you to change the key that is used to encrypt new data without affecting the ability to decrypt data that was already encrypted. This reduces the amount of content encrypted with a single key in case the key gets compromised, and also helps you comply with the NIST guidelines for key rotation. You can rotate the encryption key manually by invoking the `/transit/keys/<name>/rotate` endpoint, or you can configure the key to automatically rotate based on a time interval or a number of encryption operations. When you rotate a key, Vault generates a new key version and increments the key's `latest_version` metadata. The new key version becomes the encryption key used for encrypting any new data. The previous key versions are still available for decrypting the existing data, unless you

specify a minimum decryption version to archive the old key versions. You can also delete or disable old key versions if you want to revoke access to the data encrypted with those versions. Reference: <https://developer.hashicorp.com/vault/docs/secrets/transit1>, <https://developer.hashicorp.com/vault/api-docs/secret/transit2>

QUESTION 263

What does the following policy do?

```
path "secret/data/{{identity.entity.id}}/*" {  
    capabilities = ["create", "update", "read", "delete"]  
}  
  
path "secret/metadata/{{identity.entity.id}}/*" {  
    capabilities = ["list"]  
}
```

- A. Grants access for each user to a KV folder which shares their id
- B. Grants access to a special system entity folder
- C. Allows a user to read data about the secret endpoint identity
- D. Nothing, this is not a valid policy

Answer: C

Explanation:

This policy allows a user to read data about the secret endpoint identity. The policy grants the user the ability to create, update, read, and delete data in the `secret/data/{identity.entity.id}` path. Additionally, the user is allowed to list data in the `secret/metadata/{identity.entity.id}` path. This policy is useful for users who need to access information about the secret endpoint identity. The secret endpoint identity is a feature of the Identity Secrets Engine, which allows Vault to generate identity tokens that can be used to access other Vault secrets engines or namespaces. The identity tokens are based on the entity and group information of the user or machine that authenticates with Vault. The entity is a unique identifier for the user or machine, and the group is a collection of entities that share some common attributes. The identity tokens can carry metadata and policies that are associated with the entity and group. The `secret/data/{identity.entity.id}` path is where the user can store and retrieve data that is related to the secret endpoint identity. For example, the user can store some configuration or

preferences for the secret endpoint identity in this path. The `æsecret/metadata/{identity.entity.id}` path is where the user can list the metadata of the data stored in the `æsecret/data/{identity.entity.id}` path. For example, the user can list the version, creation time, deletion time, and destroy time of the data in this path.

Reference:

[Identity - Secrets Engines | Vault | HashiCorp Developer]

[KV - Secrets Engines | Vault | HashiCorp Developer]

QUESTION 264

To make an authenticated request via the Vault HTTP API, which header would you use?

- A. The X-Vault-Token HTTP Header
- B. The x-Vault-Request HTTP Header
- C. The Content-Type HTTP Header
- D. The X-Vault-Namespace HTTP Header

Answer: A

Explanation:

To make an authenticated request via the Vault HTTP API, you need to use the X-Vault-Token HTTP Header or the Authorization HTTP Header using the Bearer <token> scheme. The token is a string that represents your identity and permissions in Vault. You can obtain a token by using an authentication method, such as userpass, approle, aws, etc. The token can also be a root token, which has unlimited access to Vault, or a wrapped token, which is a response-wrapping token that can be used to unwrap the actual token. The token must be sent with every request to Vault that requires authentication, except for the unauthenticated endpoints, such as `sys/init`, `sys/seal-status`, `sys/unseal`, etc. The token is used by Vault to verify your identity and enforce the policies that grant or deny access to various paths and operations. Reference:

<https://developer.hashicorp.com/vault/api-docs3>,

<https://developer.hashicorp.com/vault/docs/concepts/tokens4>,

<https://developer.hashicorp.com/vault/docs/concepts/auth5>

QUESTION 265

Which of the following are replication methods available in Vault Enterprise? Choose two correct answers.

- A. Cluster sharding
- B. Namespaces
- C. Performance Replication

D. Disaster Recovery Replication

Answer: CD

Explanation:

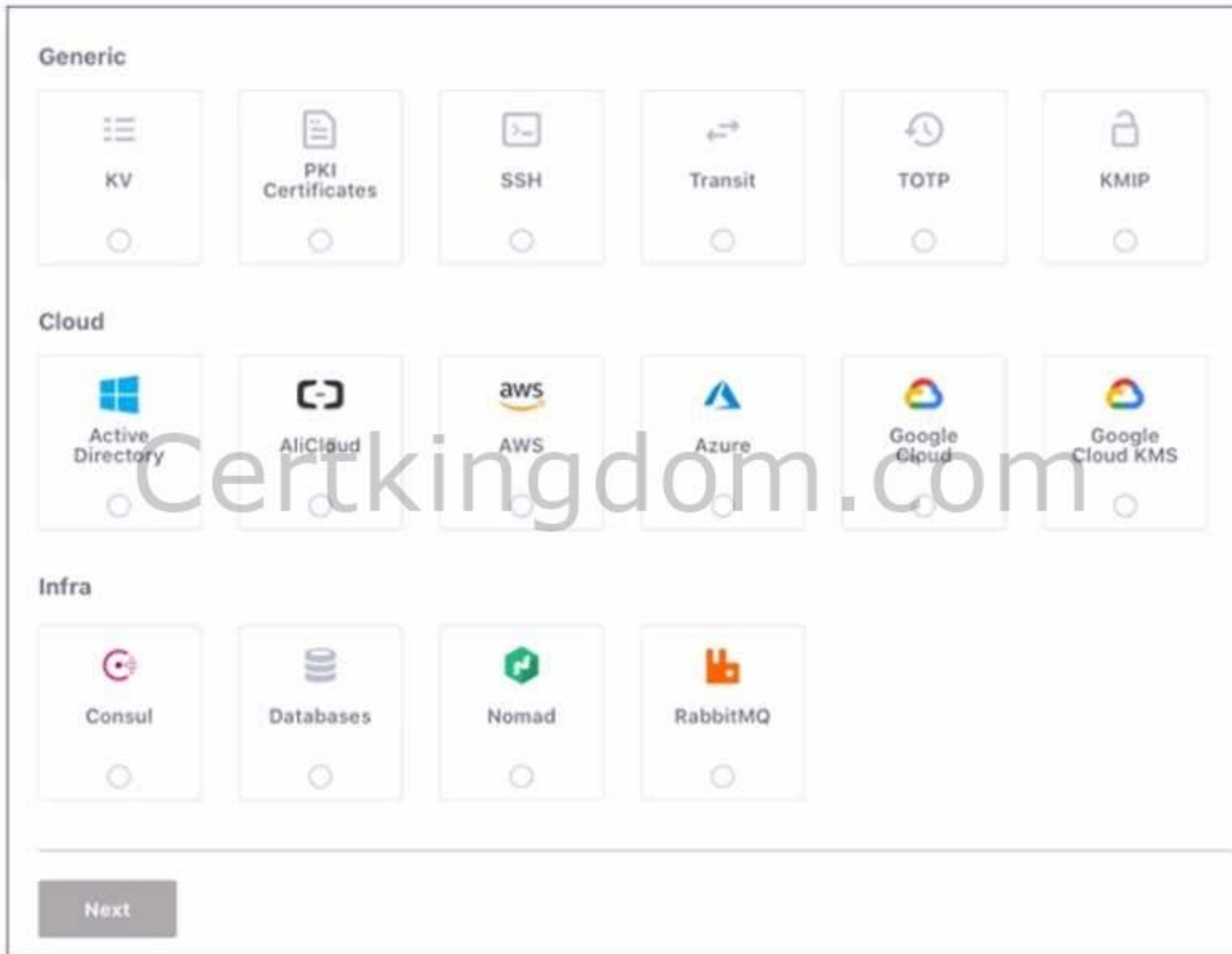
The replication methods available in Vault Enterprise are performance replication and disaster recovery replication. These methods allow critical data to be replicated across clusters to support horizontally scaling and disaster recovery workloads.

Performance replication enables a primary cluster to replicate data to one or more secondary clusters, which can handle client requests and improve performance and availability. Performance replication replicates most Vault data, such as secrets, policies, auth methods, and leases, but not tokens. Performance secondaries generate their own tokens and leases, which are not replicated back to the primary. Performance replication also supports filtering, which allows selective replication of data based on namespaces or paths.

Disaster recovery replication enables a primary cluster to replicate data to one or more secondary clusters, which act as standby clusters in case of a failure or outage of the primary. Disaster recovery replication replicates all Vault data, including tokens and leases, and maintains the same configuration and state as the primary. Disaster recovery secondaries do not handle client requests, but they can be promoted to a primary in a disaster recovery scenario. Reference: Replication - Vault Enterprise | Vault | HashiCorp Developer, Performance Replication - Vault Enterprise | Vault | HashiCorp Developer, Disaster Recovery Replication - Vault Enterprise | Vault | HashiCorp Developer

QUESTION 266

Use this screenshot to answer the question below:



When are you shown these options in the GUI?

- A. Enabling policies
- B. Enabling authentication engines
- C. Enabling secret engines

D. Enabling authentication methods

Answer: D

Explanation:

This screenshot is shown when you are enabling authentication methods in the GUI. Authentication methods are the ways users and applications authenticate with Vault. Vault supports many different authentication methods, including username and password, GitHub, and more. You can enable one or more authentication methods from the grid of options, which are divided into three categories: Generic, Cloud, and Infra. Each option has a name, a description, and a logo. You can also enable authentication methods using the Vault CLI or API.

Enabling policies, authentication engines, and secret engines are different tasks that are not related to this screenshot. Policies are rules that govern the access to Vault resources, such as secrets, authentication methods, and audit devices. Authentication engines are components of Vault that perform authentication and assign policies to authenticated entities. Secret engines are components of Vault that store, generate, or encrypt data. These tasks have different GUI pages and options than the screenshot.

Reference:

[Authentication | Vault | HashiCorp Developer]

[Policies | Vault | HashiCorp Developer]

[Authentication | Vault | HashiCorp Developer]

[Secrets Engines | Vault | HashiCorp Developer]

QUESTION 267

Examine the command below. Output has been trimmed.

```

$ vault write auth/approle/login \
  role_id="debb8f13-79ea-3e3d-8100-10711d85c1fb" \
  secret_id="31d52faa-5b0b-711d-2ea2-c197cff6081b"Key Value
---
token            b.AAAAAQI1WH-DExezQvz-ZGWMhzy8uWXEoQYHH60...trimmed...
token_accessor    n/a
token_duration    1m
token_renewable   false
token_policies    ["shipping"]
identity_policies []
policies          ["shipping"]
token_meta_role_name shipping

```

Which of the following statements describe the command and its output?

- A. Missing a default token policy
- B. Generated token's TTL is 60 hours
- C. Generated token is an orphan token which can be renewed indefinitely
- D. Configures the AppRole auth method with user specified role ID and secret ID

Answer: BC

Explanation:

The command shown in the image is:

```

vault token create -policy=approle -orphan -period=60h

```

This command creates a new token with the following characteristics:

It has the policy `approle` attached to it, which grants or denies access to certain paths and operations in Vault according to the policy rules. The policy can be defined by using the vault policy write command or the sys/policy API endpoint¹².

It is an orphan token, which means it has no parent token and it will not be revoked when its parent token is revoked. Orphan tokens can be useful for creating long-lived tokens that are not affected by the token hierarchy³.

It has a period of 60 hours, which means it has a renewable TTL of 60 hours. This means that the token can be renewed indefinitely as long as it does not go past the 60-hour mark from the last renewal time. The token's TTL will be reset to 60 hours upon each renewal. Periodic tokens are useful

for creating tokens that have a fixed lifetime and can be easily revoked⁴.

Reference: [1]¹,

[2]², 3(<https://developer.hashicorp.com/vault/docs/secrets/kv>), 4(<https://developer.hashicorp.com/vault/docs/secrets/kv>)

QUESTION 268

The key/value v2 secrets engine is enabled at secret/. See the following policy:

```
path "secret/data/*" {  
  capabilities = ["create", "read", "update", "delete", "list"]  
}  
  
path "secret/data/super-secret" {  
  capabilities = ["deny"]  
}
```

Which of the following operations are permitted by this policy? Choose two correct answers.

- A. vault kv get secret/webapp1
- B. vault kv put secret/webapp1 apikey-"ABCDEFGHJI" K123M"
- C. vault kv metadata get secret/webapp1
- D. vault kv delete secret/super-secret
- E. vault kv list secret/super-secret

Answer: AC

Explanation:

The policy shown in the image is:

```
path "secret/data/webapp1" { capabilities = [create, read, update, delete, list] }  
path "secret/data/super-secret" { capabilities = [deny] }
```

This policy grants or denies access to the key/value v2 secrets engine mounted at secret/ according to the following rules:

The path "secret/data/webapp1" has the capabilities of create, read, update, delete, and list. This means that the policy allows performing any of these operations on the secrets stored under this path. The data/ prefix is used to access the actual secret data in the key/value v2 secrets

engine5. Therefore, the policy permits the operation of `vault kv get secret/webapp1`, which reads the secret data at `secret/data/webapp1`6.

The path `secret/data/super-secret` has the capability of `deny`. This means that the policy denies performing any operation on the secrets stored under this path. The policy overrides any other policy that might grant access to this path. Therefore, the policy does not permit the operations of `vault kv delete secret/super-secret` and `vault kv list secret/super-secret`, which delete and list the secret data at `secret/data/super-secret` respectively6.

The policy does not explicitly define any rules for the path `secret/metadata`. The `metadata/` prefix is used to access the metadata of the secrets in the key/value v2 secrets engine, such as the number of versions, the deletion status, the creation time, etc5. By default, if the policy grants any of the capabilities of `create`, `read`, `update`, or `delete` on the `data/` path, it also grants the same capabilities on the corresponding `metadata/` path7. Therefore, the policy permits the operation of `vault kv metadata get secret/webapp1`, which reads the metadata of the secret at `secret/metadata/webapp1`8.

Reference: 5(<https://developer.hashicorp.com/vault/docs/secrets/kv/kv-v2>), [6]6, 7(<https://developer.hashicorp.com/vault/docs/secrets/kv/kv-v2>), [8]8

QUESTION 269

You are performing a high number of authentications in a short amount of time. You're experiencing slow throughput for token generation. How would you solve this problem?

- A. Increase the time-to-live on service tokens
- B. Implement batch tokens
- C. Establish a rate limit quota
- D. Reduce the number of policies attached to the tokens

Answer: B

Explanation:

Batch tokens are a type of tokens that are not persisted in Vault's storage backend, but are encrypted blobs that carry enough information to perform Vault actions. Batch tokens are extremely lightweight and scalable, and can improve the throughput for token generation. Batch tokens are suitable for high-volume and ephemeral workloads, such as containers or serverless functions, that require short-lived and non-renewable tokens. Batch tokens can be created by using the `-type=batch` flag in the `vault token create` command, or by configuring the `token_type` parameter in the `auth` method's role or mount options. Batch tokens have some limitations compared to service tokens, such as the lack of renewal, revocation, listing, accessor, and cubbyhole features. Therefore, batch tokens should be used with caution and only when the trade-offs are acceptable. Reference:

<https://developer.hashicorp.com/vault/tutorials/tokens/batch-tokens1>,
<https://developer.hashicorp.com/vault/docs/commands/token/create2>,
<https://developer.hashicorp.com/vault/docs/concepts/tokens#token-types3>

QUESTION 270

When looking at Vault token details, which key helps you find the paths the token is able to access?

- A. Meta
- B. Path
- C. Policies
- D. Accessor

Answer: C

Explanation:

When looking at Vault token details, the policies key helps you find the paths the token is able to access. Policies are a declarative way to grant or forbid access to certain paths and operations in Vault. Policies are written in HCL or JSON and are attached to tokens by name. Policies are deny by default, so an empty policy grants no permission in the system. A token can have one or more policies associated with it, and the effective policy is the union of all the individual policies. You can view the token details by using the vault token lookup command or the auth/token/lookup API endpoint. The output will show the policies key with a list of policy names that are attached to the token. You can also view the contents of a policy by using the vault policy read command or the sys/policy API endpoint. The output will show the rules key with the HCL or JSON representation of the policy. The rules will specify the paths and the capabilities (such as create, read, update, delete, list, etc.) that the policy allows or denies. Reference:

<https://developer.hashicorp.com/vault/docs/concepts/policies4>,
<https://developer.hashicorp.com/vault/docs/commands/token/lookup5>,
<https://developer.hashicorp.com/vault/api-docs/auth/token#lookup-a-token6>,
<https://developer.hashicorp.com/vault/docs/commands/policy/read7>,
<https://developer.hashicorp.com/vault/api-docs/system/policy8>

QUESTION 271

A developer mistakenly committed code that contained AWS S3 credentials into a public repository. You have been tasked with revoking the AWS S3 credential that was in the code. This credential was created using Vault's AWS secrets engine and the developer received the following output when requesting a credential from Vault.

Key	Value
lease_id	aws/creds/s3-access/f3e92392-7d9c-09c8-c921-575d62fe80d8
lease_duration	768h
lease_renewable	true
access_key	AKIAIOWQXTLW36DV7IEA
secret_key	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

Which Vault command will revoke the lease and remove the credential from AWS?

- A. vault lease revoke aws/creds/s3-access/f3e92392-7d9c-99c8-c921-57Sd62fe89d8
- B. vault lease revoke AKIAI0WQXTLW36DV7IEA
- C. vault lease revoke f3e92392-7d9c-09c8-c921-575d62fe80d8
- D. vault lease revoke access_key-AKIAI0WQXTLW36DV7IEA

Answer: A

Explanation:

The correct answer is A because the lease ID is the unique identifier for the credential. The lease ID is used to revoke the credential using the vault lease revoke command. This command will invalidate the credential immediately and prevent any further renewals. It will also delete the access key and secret key from AWS, rendering them useless. The access key and secret key are not sufficient to revoke the credential, as they are not recognized by Vault. The lease ID is composed of the path of the secrets engine, the role name, and a random UUID. In this case, the path is aws/creds, the role name is s3-access, and the UUID is f3e92392-7d9c-99c8-c921-57Sd62fe89d8.

Reference:

lease revoke - Command | Vault | HashiCorp Developer

QUESTION 272

When an auth method is disabled all users authenticated via that method lose access.

- A. True
- B. False

Answer: A

Explanation:

The statement is true. When an auth method is disabled, all users authenticated via that method lose access. This is because the tokens issued by the auth method are automatically revoked when the auth method is disabled. This prevents the users from performing any operation in Vault using the revoked tokens. To regain access, the users have to authenticate again using a different auth method that is enabled and has the appropriate policies attached. Reference: [Auth Methods | Vault | HashiCorp Developer](#), [auth disable - Command | Vault | HashiCorp Developer](#)

QUESTION 273

A user issues the following cURL command to encrypt data using the transit engine and the Vault AP:

```
curl \
--header "X-Vault-Token: c4f280f6-fdb2-18eb-89d3-589e2e834cdb" \
--request POST \<
--data @payload.json \
http://127.0.0.1:8200/v1/transit/encrypt/my-key
```

Which payload.json file has the correct contents?

A.

```
{
  "plaintext": "dG81IHFlawNrIGJyb3duIGZveA=="
}
```

B.

```
{
  "ciphertext": "vault:v1:abcdefgh"
}
```

C.

```
{
  "data": {
    "plaintext": "dGhlIHFlaWNrIGJyb3duIGZveA=="
  }
}
```

D.

```
{
  "data": {
    "ciphertext": "vault:sv1:abcdefgh"
  }
}
```

Answer: C

Explanation:

The payload.json file that has the correct contents is C. This file contains a JSON object with a single key, `plaintext`, and a value that is the base64-encoded string of the data to be encrypted. This is the format that the Vault API expects for the `transit encrypt` endpoint. The other files are not correct because they either have the wrong key name, the wrong value format, or the wrong JSON syntax.

Reference:

Encrypt Data - Transit Secrets Engine | Vault | HashiCorp Developer

QUESTION 274

An authentication method should be selected for a use case based on:

- A. The auth method that best establishes the identity of the client
- B. The cloud provider for which the client is located on
- C. The strongest available cryptographic hash for the use case
- D. Compatibility with the secret engine which is to be used

Answer: A

Explanation:

An authentication method should be selected for a use case based on the auth method that best establishes the identity of the client. The identity of the client is the basis for assigning a set of policies and permissions to the client in Vault. Different auth methods have different ways of verifying the identity of the client, such as using passwords, tokens, certificates, cloud credentials, etc. Depending on the use case, some auth methods may be more suitable or convenient than others. For example, for human users, the userpass or ldap auth methods may be easy to use, while for machines or applications, the approle or aws auth methods may be more secure and scalable. The choice of the auth method should also consider the trade-offs between security, performance, and usability. Reference: Auth Methods | Vault | HashiCorp Developer, Authentication - Concepts | Vault | HashiCorp Developer

QUESTION 275

A web application uses Vault's transit secrets engine to encrypt data in-transit. If an attacker intercepts the data in transit which of the following statements are true? Choose two correct answers.

- A. You can rotate the encryption key so that the attacker won't be able to decrypt the data
- B. The keys can be rotated and min_decryption_version moved forward to ensure this data cannot be decrypted
- C. The Vault administrator would need to seal the Vault server immediately
- D. Even if the attacker was able to access the raw data, they would only have encrypted bits (TLS in transit)

Answer: BD

Explanation:

A web application that uses Vault's transit secrets engine to encrypt data in-transit can benefit from the following security features:

Even if the attacker was able to access the raw data, they would only have encrypted bits (TLS in transit). This means that the attacker would need to obtain the encryption key from Vault in order to decrypt the data, which is protected by Vault's authentication and authorization mechanisms. The transit secrets engine does not store the data sent to it, so the attacker cannot access the data from Vault either.

The keys can be rotated and min_decryption_version moved forward to ensure this data cannot be decrypted. This means that the web application can periodically change the encryption key used to encrypt the data, and set a minimum decryption version for the key, which prevents older versions of the key from being used to decrypt the data. This way, even if the attacker somehow obtained an old version of the key, they would not be able to decrypt the data that was encrypted with a newer

version of the key.

The other statements are not true, because:

You cannot rotate the encryption key so that the attacker won't be able to decrypt the data. Rotating the key alone does not prevent the attacker from decrypting the data, as they may still have access to the old version of the key that was used to encrypt the data. You need to also move the `min_decryption_version` forward to invalidate the old version of the key.

The Vault administrator would not need to seal the Vault server immediately. Sealing the Vault server would make it inaccessible to both the attacker and the legitimate users, and would require unsealing it with the unseal keys or the recovery keys. Sealing the Vault server is a last resort option in case of a severe compromise or emergency, and is not necessary in this scenario, as the attacker does not have access to the encryption key or the data in Vault. Reference: Transit - Secrets Engines | Vault | HashiCorp Developer, Encryption as a service: transit secrets engine | Vault | HashiCorp Developer

QUESTION 276

The Vault encryption key is stored in Vault's backend storage.

- A. True
- B. False

Answer: B

Explanation:

The statement is false. The Vault encryption key is not stored in Vault's backend storage, but rather in Vault's memory. The Vault encryption key is the key that is used to encrypt and decrypt the data that is stored in Vault's backend storage, such as secrets, tokens, policies, etc. The Vault encryption key is derived from the master key, which is generated when Vault is initialized. The master key is split into unseal keys using Shamir's secret sharing algorithm, and the unseal keys are distributed to trusted operators. To start Vault, a quorum of unseal keys is required to reconstruct the master key and derive the encryption key. The encryption key is then kept in memory and used to protect the data in Vault's backend storage. The encryption key is never written to disk or exposed via the API. Reference: Seal/Unseal | Vault | HashiCorp Developer, Key Rotation | Vault | HashiCorp Developer

QUESTION 277

Which of the following statements describe the secrets engine in Vault? Choose three correct answers.

- A. Some secrets engines simply store and read data
- B. Once enabled, you cannot disable the secrets engine
- C. You can build your own custom secrets engine

- D. Each secrets engine is isolated to its path
- E. A secrets engine cannot be enabled at multiple paths

Answer: ACD

Explanation:

Secrets engines are components that store, generate, or encrypt data in Vault. They are enabled at a specific path in Vault and have their own API and configuration. Some of the statements that describe the secrets engines in Vault are:

Some secrets engines simply store and read data, such as the key/value secrets engine, which acts like an encrypted Redis or Memcached. Other secrets engines perform more complex operations, such as generating dynamic credentials, encrypting data, issuing certificates, etc1.

You can build your own custom secrets engine by using the plugin system, which allows you to write and run your own secrets engine as a separate process that communicates with Vault over gRPC. You can also use the SDK to create your own secrets engine in Go and compile it into Vault2.

Each secrets engine is isolated to its path, which means that the secrets engine cannot access or interact with other secrets engines or data outside its path. The path where the secrets engine is enabled can be customized and can have multiple segments. For example, you can enable the AWS secrets engine at `aws/` or `aws/prod/` or `aws/dev`.

The statements that are not true about the secrets engines in Vault are:

You can disable an existing secrets engine by using the `vault secrets disable` command or the `sys/mounts` API endpoint. When a secrets engine is disabled, all of its secrets are revoked and all of its data is deleted from the storage backend4.

A secrets engine can be enabled at multiple paths, with a few exceptions, such as the system and identity secrets engines. Each secrets engine enabled at a different path is independent and isolated from others. For example, you can enable the KV secrets engine at `kv/` and `secret/` and they will not share any data3.

Reference: 1(<https://developer.hashicorp.com/vault/docs/secrets>), 2(<https://developer.hashicorp.com/vault/docs/secrets>), 3(<https://developer.hashicorp.com/vault/docs/secrets>), 4(<https://developer.hashicorp.com/vault/docs/secrets>)

QUESTION 278

What is a benefit of response wrapping?

- A. Log every use of a secret
- B. Load balance secret generation across a Vault cluster
- C. Provide error recovery to a secret so it is not corrupted in transit
- D. Ensure that only a single party can ever unwrap the token and see what's inside

Answer: D

Explanation:

Response wrapping is a feature that allows Vault to take the response it would have sent to a client and instead insert it into the cubbyhole of a single-use token, returning that token instead. The client can then unwrap the token and retrieve the original response. Response wrapping has several benefits, such as providing cover, malfeasance detection, and lifetime limitation for the secret data. One of the benefits is to ensure that only a single party can ever unwrap the token and see what's inside, as the token can be used only once and cannot be unwrapped by anyone else, even the root user or the creator of the token. This provides a way to securely distribute secrets to the intended recipients and detect any tampering or interception along the way⁵.

The other options are not benefits of response wrapping:

Log every use of a secret: Response wrapping does not log every use of a secret, as the secret is not directly exposed to the client or the network. However, Vault does log the creation and deletion of the response-wrapping token, and the client can use the audit device to log the unwrapping operation⁶.

Load balance secret generation across a Vault cluster: Response wrapping does not load balance secret generation across a Vault cluster, as the secret is generated by the Vault server that receives the request and the response-wrapping token is bound to that server. However, Vault does support high availability and replication modes that can distribute the load and improve the performance of the cluster⁷.

Provide error recovery to a secret so it is not corrupted in transit: Response wrapping does not provide error recovery to a secret so it is not corrupted in transit, as the secret is encrypted and stored in the cubbyhole of the token and cannot be modified or corrupted by anyone. However, if the token is lost or expired, the secret cannot be recovered either, so the client should have a backup or retry mechanism to handle such cases.

Reference: ⁵(<https://developer.hashicorp.com/vault/docs/concepts/responsewrapping>), ⁶(<https://developer.hashicorp.com/vault/docs/secrets>), ⁷(<https://developer.hashicorp.com/vault/docs/secrets>), (<https://developer.hashicorp.com/vault/tutorials/secretsmanagement/cubbyhole-response-wrapping>)

QUESTION 279

Which of the following describes the Vault's auth method component?

- A. It verifies a client against an internal or external system, and generates a token with the appropriate policies attached
- B. It verifies a client against an internal or external system, and generates a token with root policy
- C. It is responsible for durable storage of client tokens
- D. It dynamically generates a unique set of secrets with appropriate permissions attached

Answer: A

Explanation:

The Vault's auth method component is the component that performs authentication and assigns identity and policies to a client. It verifies a client against an internal or external system, and generates a token with the appropriate policies attached. The token can then be used to access the secrets and resources that are authorized by the policies. Vault supports various auth methods, such as userpass, ldap, aws, kubernetes, etc., that can integrate with different identity providers and systems. The auth method component can also handle token renewal and revocation, as well as identity grouping and aliasing. Reference: [Auth Methods | Vault | HashiCorp Developer](#), [Authentication - Concepts | Vault | HashiCorp Developer](#)

QUESTION 280

Which Vault secret engine may be used to build your own internal certificate authority?

- A. Transit
- B. PKI
- C. PostgreSQL
- D. Generic

Answer: B

Explanation:

The Vault secret engine that can be used to build your own internal certificate authority is the PKI secret engine. The PKI secret engine generates dynamic X.509 certificates on-demand, without requiring manual processes of generating private keys and CSRs, submitting to a CA, and waiting for verification and signing. The PKI secret engine can act as a root CA or an intermediate CA, and can issue certificates for various purposes, such as TLS, code signing, email encryption, etc. The PKI secret engine can also manage the certificate lifecycle, such as rotation, revocation, renewal, and CRL generation. The PKI secret engine can also integrate with external CAs, such as Venafi or Entrust, to delegate the certificate issuance and management. Reference: [PKI - Secrets Engines | Vault | HashiCorp Developer](#), [Build Your Own Certificate Authority \(CA\) | Vault - HashiCorp Learn](#)

QUESTION 281

Which of the following statements are true about Vault policies? Choose two correct answers.

- A. The default policy can not be modified
- B. You must use YAML to define policies
- C. Policies provide a declarative way to grant or forbid access to certain paths and operations in Vault
- D. Vault must be restarted in order for a policy change to take an effect
- E. Policies deny by default (empty policy grants no permission)

Answer: CE

Explanation:

Vault policies are written in HCL or JSON format and are attached to tokens or roles by name. Policies define the permissions and restrictions for accessing and performing operations on certain paths and secrets in Vault. Policies are deny by default, which means that an empty policy grants no permission in the system, and any request that is not explicitly allowed by a policy is implicitly denied¹. Some of the features and benefits of Vault policies are:

Policies are path-based, which means that they match the request path to a set of rules that specify the allowed or denied capabilities, such as create, read, update, delete, list, sudo, etc².

Policies are additive, which means that if a token or a role has multiple policies attached, the effective policy is the union of all the individual policies. The most permissive capability is granted if there is a conflict³.

Policies can use glob patterns, such as `*` and `+`, to match multiple paths or segments with a single rule. For example, path `secret/*` matches any path starting with `secret/`, and path `secret+/config` matches any path with two segments after `secret/` and ending with `config`⁴.

Policies can use templating to interpolate certain values into the rules, such as identity information, time, randomness, etc. For example, path `secret/{{identity.entity.id}}/*` matches any path starting with `secret/` followed by the entity ID of the requester⁵.

Policies can be managed by using the vault policy commands or the `sys/policy` API endpoints. You can write, read, list, and delete policies by using these interfaces⁶.

The default policy is a built-in policy that is attached to all tokens by default and cannot be deleted.

However, the default policy can be modified by using the vault policy write command or the `sys/policy` API endpoint. The default policy provides common permissions for tokens, such as renewing themselves, looking up their own information, creating and managing response-wrapping tokens, etc⁷.

You do not have to use YAML to define policies, as Vault supports both HCL and JSON formats.

HCL is a human-friendly configuration language that is also JSON compatible, which means that JSON can be used as a valid input for policies as well⁸.

Vault does not need to be restarted in order for a policy change to take effect, as policies are stored and evaluated in memory. Any change to a policy is immediately reflected in the system, and any token or role that has that policy attached will be affected by the change.

Reference: 1(<https://developer.hashicorp.com/vault/docs/concepts/policies>), 2(<https://developer.hashicorp.com/vault/docs/concepts/policies>), 3(<https://developer.hashicorp.com/vault/docs/concepts/policies>), 4(<https://developer.hashicorp.com/vault/docs/concepts/policies>), 5(<https://developer.hashicorp.com/vault/docs/concepts/policies>), 6(<https://developer.hashicorp.com/vault/docs/commands/lease>), 7(<https://developer.hashicorp.com/vault/docs/concepts/policies>), 8(<https://developer.hashicorp.com/vault/docs/concepts/policies#policy-updates>)

QUESTION 282

Use this screenshot to answer the question below:

Secrets

Access

Policies

Tools

Status

Secrets Engines

Enable new engine +

 <u>aws/</u> aws_69555089		...
 <u>cubbyhole/</u> cubbyhole_17b97728		...
 <u>eu-secrets/</u> v2 kv_f1aa4381		...
 <u>secret/</u> v2 kv_253415a8		...
<u>transform/</u> transform_32a0740f		...

 © 2020 HashiCorp Vault 1.4.2+ent [Documentation](#)

Where on this page would you click to view a secret located at secret/my-secret?

- A. A
- B. B
- C. C
- D. D
- E. E

Answer: C

Explanation:

In the HashiCorp Vault UI, secrets are organized in a tree-like structure. To view a secret located at `secret/my-secret`, you would click on the `secret/` folder in the tree, then click on the `my-secret` file. In this screenshot, the `secret/` folder is located at option C. This folder contains the secrets that are stored in the key/value secrets engine, which is the default secrets engine in Vault. The key/value secrets engine allows you to store arbitrary secrets as key/value pairs. The key is the path of the secret, and the value is the data of the secret. For example, the secret located at `secret/my-secret` has a key of `my-secret` and a value of whatever data you stored there.

Reference:

[KV - Secrets Engines | Vault | HashiCorp Developer]

QUESTION 283

An organization would like to use a scheduler to track & revoke access granted to a job (by Vault) at completion. What auth-associated Vault object should be tracked to enable this behavior?

- A. Token accessor
- B. Token ID
- C. Lease ID
- D. Authentication method

Answer: C

Explanation:

A lease ID is a unique identifier that is assigned by Vault to every dynamic secret and service type authentication token. A lease ID contains information such as the secret path, the secret version, the secret type, etc. A lease ID can be used to track and revoke access granted to a job by Vault at completion, as it allows the scheduler to perform the following operations:

Look up the lease information by using the `vault lease lookup` command or the `sys/leases/lookup` API endpoint. This will return the metadata of the lease, such as the expire time, the issue time, the renewable status, and the TTL.

Renew the lease if needed by using the vault lease renew command or the sys/leases/renew API endpoint. This will extend the validity of the secret or the token for a specified increment, or reset the TTL to the original value if no increment is given.

Revoke the lease when the job is completed by using the vault lease revoke command or the sys/leases/revoke API endpoint. This will invalidate the secret or the token immediately and prevent any further renewals. For example, with the AWS secrets engine, the access keys will be deleted from AWS the moment a lease is revoked.

A lease ID is different from a token ID or a token accessor. A token ID is the actual value of the token that is used to authenticate to Vault and perform requests. A token ID should be treated as a secret and protected from unauthorized access. A token accessor is a secondary identifier of the token that is used for token management without revealing the token ID. A token accessor can be used to lookup, renew, or revoke a token, but not to authenticate to Vault or access secrets. A token ID or a token accessor can be used to revoke the token itself, but not the leases associated with the token.

To revoke the leases, a lease ID is required.

An authentication method is a way to verify the identity of a user or a machine and issue a token with appropriate policies and metadata. An authentication method is not an object that can be tracked or revoked, but a configuration that can be enabled, disabled, tuned, or customized by using the vault auth commands or the sys/auth API endpoints.

Reference: (<https://developer.hashicorp.com/vault/docs/commands/lease/lookup>),
(<https://developer.hashicorp.com/vault/docs/commands/lease/renew>),
(<https://developer.hashicorp.com/vault/docs/commands/lease/revoke>),
(<https://developer.hashicorp.com/vault/docs/concepts/tokens#token-accessors>),
(<https://developer.hashicorp.com/vault/docs/concepts/auth>)

QUESTION 284

Which statement describes the results of this command: `$ vault secrets enable transit`

- A. Enables the transit secrets engine at transit path
- B. Requires a root token to execute the command successfully
- C. Enables the transit secrets engine at secret path
- D. Fails due to missing -path parameter
- E. Fails because the transit secrets engine is enabled by default

Answer: A

Explanation:

The command `vault secrets enable transit` enables the transit secrets engine at the transit path. The transit secrets engine is a secrets engine that handles cryptographic functions on data in-transit, such as encryption, decryption, signing, verification, hashing, and random bytes generation. The transit

secrets engine does not store the data sent to it, but only performs the requested operations and returns the results. The transit secrets engine can also be viewed as encryption as a service or decryption as a service. The command `vault secrets enable transit` uses the default path of transit for the secrets engine, but this can be changed by using the `-path` option. For example, `vault secrets enable -path=my-transit transit` would enable the transit secrets engine at the `my-transit` path. Reference: Transit - Secrets Engines | Vault | HashiCorp Developer, `vault secrets enable -` Command | Vault | HashiCorp Developer

QUESTION 285

Running the second command in the GUI CLI will succeed.



```
> vault help
Usage: vault <command> [args]

Commands:
  read      Read data and retrieves secrets
  write     Write data, configuration, and secrets
  delete    Delete secrets and configuration
  list      List data or secrets

Web CLI Commands:
  api       Navigate to the Vault API explorer. Use 'api [filter]' to prefilter the list.
  clear     Clear output from the log
  clearall  Clear output and command history
  fullscreen Toggle fullscreen display
  refresh   Refresh the data on the current screen under the CLI window

> vault auth enable cert
```

- A. True
- B. False

Answer: B

Explanation:

Running the second command in the GUI CLI will fail. The second command is `vault kv put`

secret/creds passcode=my-long-passcode. This command attempts to write a secret named creds with the value passcode=my-long-passcode to the secret path, which is the default path for the kv secrets engine. However, the kv secrets engine is not enabled at the secret path, as shown by the first command vault secrets list, which lists the enabled secrets engines and their paths. The only enabled secrets engine is the transit secrets engine at the transit path. Therefore, the second command will fail with an error message saying that no secrets engine is mounted at the path secret/. To make the second command succeed, the kv secrets engine must be enabled at the secret path or another path, using the vault secrets enable command. For example, vault secrets enable -path=secret kv would enable the kv secrets engine at the secret path. Reference: kv - Command | Vault | HashiCorp Developer, vault secrets enable - Command | Vault | HashiCorp Developer